DEVOPS ENGINEERING MASTERY

INFRASTRUCTURE AS CODE & TERRAFORM ENGINEERING

Infrastructure Automation, Terraform Architecture, State Management & Production Provisioning

VOLUME 08 • FIRST EDITION

```
nahid@technova:~$ terraform plan -out=tfplan
✓ Plan: 12 to add, 0 to change, 0 to destroy
nahid@technova:~$ terraform apply tfplan
✓ Apply complete! Resources: 12 added, 0 changed, 0 destroyed
nahid@technova:~$ terraform state list | wc -l
```

Terraform • HCL • AWS • Azure • GCP • IBM Cloud

First Edition • Published 9 November 2026 • Dhaka, Bangladesh

24 Chapters • 24 Guided Labs • 3 Major Projects • 120 MCQs

Infrastructure as Code (IaC) & Terraform Engineering

DEVOPS ENGINEERING MASTERY — Volume 08

Infrastructure Automation, Terraform Architecture, State Management ও Production Provisioning

First Edition — Published 9 November 2026.

Hands-on environment: Ubuntu Server 24.04 LTS, Terraform 1.9+ (pinned >= 1.9, < 2.0), providers pinned and explained in every chapter (hashicorp/aws ~> 5.0, hashicorp/azurerm ~> 4.0, hashicorp/google ~> 6.0, IBM-Cloud/ibm ~> 1.0), with a MERN (MongoDB, Express, React, Node.js) example application called TechNova. All domains, IP addresses and credentials in examples are fictional (technova.example, 203.0.113.x, CHANGE-ME-...). Sample terminal outputs are illustrative. Cloud chapters name the resources that incur charges and always end with a `terraform destroy` cleanup step — never leave cloud resources running after a lab. Official provider documentation is the authority; no pricing or service limits are stated in this book.

How to use this book. Read the parts in order — each part builds on the previous one. Keep a Terraform-ready machine beside you and do every hands-on exercise and every lab in the companion Lab Workbook before moving on. Do not copy HCL blindly; read the explanation under each configuration field first. Destructive operations are always flagged with a WARNING box and a safe alternative. Never practise on production infrastructure: use local providers and disposable cloud accounts, never paste a real secret into an example, never commit a state file or a secret to Git, and never edit Terraform state by hand as a normal workflow — use the safe state operations in Chapter 10.

A note on honesty. Reading a book alone does not make anyone an expert. This book teaches the foundations and the practical judgment to progress toward professional engineering — but the judgment itself comes from doing the labs, breaking a test environment, fixing it, and repeating. There is no shortcut for that practice.

Conventions. Bangla is the explanatory language; technical terms, command names, APIs and configuration keys stay in English. `monospace` marks commands, file paths and code. **CONCEPT** explains an idea, **REAL WORLD** shows an industry scenario, **WARNING** marks risk, **TROUBLESHOOTING** helps you fix a failure, and **LAB** marks hands-on work.

© 2026 DevOps Engineering Mastery series. All rights reserved. This book is for personal learning. Infrastructure changes should first be practised in a disposable environment, never directly on production systems. The author and publisher accept no liability for data loss, service disruption or cloud charges caused by running commands without a backup, a plan review or an approval gate.

Who this book is for

A learner who knows JavaScript, React, Next.js, Node.js, Express and MongoDB, understands basic Git and Linux concepts, has studied Linux server administration, networking, CI/CD, Docker and Kubernetes, and

wants to automate real infrastructure with Terraform — from a first local project to a governed, multi-environment production platform with remote state, Policy as Code, CI/CD and disaster recovery.

Learning outcomes

By the end of this book you will be able to: explain why Infrastructure as Code exists and how declarative management differs from imperative scripting; describe Terraform Core, providers, the dependency graph and the plan/apply pipeline; write correct HCL for variables, outputs, locals, resources, data sources, meta-arguments, dynamic blocks and functions; design reusable, versioned modules and compose them into environments; manage state safely with remote backends and locking; import and migrate existing infrastructure; provision AWS, Azure, GCP and IBM Cloud infrastructure with Terraform; secure state and secrets, validate configuration and enforce Policy as Code; run Terraform in CI/CD with plan review and approval gates; and operate a production IaC platform with governance, drift detection, disaster recovery and cost management.

সূচিপত্র — Contents

PART I — Infrastructure as Code Foundations

Chapter 01 — Infrastructure as Code — Concepts, History, and Engineering Principles 7

Chapter 02 — Traditional Infrastructure vs Automated Infrastructure 13

Chapter 03 — Declarative vs Imperative Infrastructure Management 17

Chapter 04 — Terraform Architecture — Providers, Resources, Data Sources, and Plugins 21

Chapter 05 — Terraform Installation, CLI, Project Structure, and Workflow 26

PART II — Terraform Core Engineering

Chapter 06 — HCL Fundamentals — Blocks, Arguments, Expressions, and Comments 33

Chapter 07 — Variables, Outputs, Locals, and Type Constraints 39

Chapter 08 — Resources, Data Sources, and Resource Dependencies 45

Chapter 09 — Terraform State — Purpose, Lifecycle, and Internal Structure 51

Chapter 10 — State Management — Remote Backend, Locking, and State Operations 56

Chapter 11 — Terraform Plan, Apply, Destroy, Refresh, and Lifecycle Rules 62

PART III — Advanced Terraform Engineering

Chapter 12 — Meta-Arguments — count, for_each, depends_on, and lifecycle 69

Chapter 13 — Dynamic Blocks, Functions, Expressions, and Advanced HCL 76

Chapter 14 — Terraform Modules — Reusability, Design, and Module Composition 83

Chapter 15 — Module Versioning, Registry, and Dependency Management 88

Chapter 16 — Importing Existing Infrastructure and Infrastructure Migration 92

PART IV — Cloud Infrastructure Provisioning

Chapter 17 — AWS Infrastructure Provisioning with Terraform 98

Chapter 18 — Microsoft Azure Infrastructure Provisioning 109

Chapter 19 — Google Cloud Platform Infrastructure Provisioning 117

Chapter 20 — IBM Cloud Infrastructure Provisioning 123

Chapter 21 — Networking, Compute, Storage, Database, and IAM Automation 130

PART V — Production Infrastructure as Code

Chapter 22 — Terraform Security, Secrets, Validation, and Policy as Code 135

Chapter 23 — Terraform CI/CD, Testing, Collaboration, and GitOps Integration 139

Chapter 24 — Production IaC Architecture, Governance, Disaster Recovery, and Cost Management 143

Companion Volumes

Lab Workbook — 24 Guided Labs • 15 Assignments • 3 Major Projects • Capstone

Assessment and Answer Guide — 120 MCQs • Short Answers • Scenarios • Case Studies • Final Exam

Quick Reference Handbook — Glossary (150+ Terms) • HCL & Command References • Checklists • Decision Trees

Guided labs (24), major projects (3) and the capstone are in the separate *Terraform Lab Workbook*. MCQs, assignments, examinations and the answer key are in the separate *Assessment and Answer Guide*. Glossary, HCL and command references, checklists and decision trees are in the separate *Quick Reference Handbook*.

PART I • DEVOPS ENGINEERING MASTERY

Infrastructure as Code Foundations

Infrastructure as Code-এর ধারণা ও ইতিহাস, Traditional বনাম Automated Infrastructure, Declarative বনাম Imperative ব্যবস্থাপনা, Terraform আর্কিটেকচার, ইনস্টলেশন, CLI ও প্রজেক্ট কাঠামো।

অধ্যায় ১ • PART I — Infrastructure as Code Foundations

Chapter 1 — Infrastructure as Code — Concepts, History, and Engineering Principles

১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — **Infrastructure as Code (IaC)** বলতে ঠিক কী বোঝায় তা নিজের ভাষায় ব্যাখ্যা করতে; হাতে-কলমে সার্ভার বানানো থেকে শুরু করে ঘোষণামূলক কোডে অবকাঠামো লেখার ইতিহাসের ধাপগুলো বলতে; **Idempotence**, **Immutability**, **Reproducibility** ও **Reviewability**-এর মতো প্রকৌশল নীতিগুলো বাস্তব উদাহরণসহ প্রয়োগ করতে; এবং কেন IaC ছাড়া আধুনিক DevOps অসম্পূর্ণ, তা TechNova-র প্রেক্ষাপটে যুক্তি দিয়ে দেখাতে।

- IaC-এর সংজ্ঞা, পরিধি ও সীমা নির্ধারণ করা।
- অবকাঠামো পরিচালনার বিবর্তন — ম্যানুয়াল অপারেশন, শেল স্ক্রিপ্ট, কনফিগারেশন ম্যানেজমেন্ট থেকে IaC পর্যন্ত — ব্যাখ্যা করা।
- পাঁচটি মূল প্রকৌশল নীতি উদাহরণসহ প্রয়োগ করা।
- একটি কাল্পনিক কোম্পানির সমস্যায় IaC সমাধানটি চিহ্নিত করা।

১.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

DevOps-এর মূল প্রতিশ্রুতি হলো — সফটওয়্যার দ্রুত, নিরাপদে ও বারবার একইভাবে ডেলিভার করা। কিন্তু অ্যাপ্লিকেশন কোড যতই ভালো হোক, তা চলার জায়গাটি — সার্ভার, নেটওয়ার্ক, ডেটাবেস, লোড ব্যালান্সার, ফায়ারওয়াল — যদি হাতে বানানো হয়, তাহলে পুরো ডেলিভারি শৃঙ্খলটিই ভঙ্গুর হয়ে যায়। একজন প্রকৌশলী মনে রাখা নির্দেশনা দিয়ে একটি সার্ভার বানালেন, আরেকজন সামান্য ভিন্নভাবে বানালেন — ফলাফল: **Development**-এ চলে, **Production**-এ চলে না। এই সমস্যার নাম **Environment Drift** বা পরিবেশের বিচ্যুতি। IaC এই সমস্যার মূলে আঘাত করে: অবকাঠামোকে আর মানুষের স্মৃতি বা টিকিটের নির্দেশনায় না রেখে, তাকে কোডে লেখা হয় — যে কোড **Version Control**-এ থাকে, **Code Review** হয়, পরীক্ষা করা যায় এবং বারবার একই ফল দেয়।

> **CONCEPT:** Infrastructure as Code মানে শুধু স্ক্রিপ্ট লেখা নয়। স্ক্রিপ্ট বলে “কী করতে হবে, কোন ক্রমে” — IaC বলে “শেষ অবস্থাটি কেমন হবে”। এই পার্থক্যটিই তৃতীয় অধ্যায়ে বিস্তারিত আসবে, তবে এখন মনে রাখুন: IaC-তে অবকাঠামোর কাঙ্ক্ষিত অবস্থা একটি ফাইলে ঘোষণা করা হয়, আর একটি ইঞ্জিন সেই অবস্থা বাস্তবায়নের দায়িত্ব নেয়।

> **REAL WORLD:** TechNova-তে একবার একজন সিনিয়র প্রকৌশলী ছুটিতে গেলেন, আর তাঁর হাতে বানানো প্রোডাকশন সার্ভারটি ক্র্যাশ করল। কেউ জানত না সার্ভারে কোন প্যাকেজের কোন সংস্করণ, কোন ফায়ারওয়াল নিয়ম, কোন পরিবেশ চলক ছিল। পুনর্নির্মাণে লেগেছিল তিন দিন। IaC থাকলে পুনর্নির্মাণ হতো একটি কমান্ডে, পনেরো মিনিটে — কারণ সবকিছুই কোডে লেখা থাকত।

১.৩ পূর্বশর্ত জ্ঞান

এই অধ্যায় পড়তে আপনার লিনাক্স কমান্ড লাইনের মৌলিক ধারণা, **Git**-এর `commit`, `branch` ও `pull request`-এর ধারণা, এবং সার্ভার, নেটওয়ার্ক ও ডেটাবেস কী — এই প্রাথমিক ধারণাটুকু থাকলেই চলবে। Terraform বা ক্লাউডের পূর্ব অভিজ্ঞতা প্রয়োজন নেই; সেগুলো এই ভলিউমেই ধাপে ধাপে শেখানো হবে।

১.৪ Infrastructure as Code আসলে কী

Infrastructure as Code (IaC): কম্পিউটিং অবকাঠামো — সার্ভার, ভার্চুয়াল মেশিন, নেটওয়ার্ক, স্টোরেজ, ডেটাবেস, লোড ব্যালান্সার, DNS রেকর্ড, নিরাপত্তা নিয়ম — ম্যানুয়াল প্রক্রিয়ায় না বানিয়ে, মেশিন-পাঠযোগ্য সংজ্ঞা ফাইলে লিখে, সংস্করণ নিয়ন্ত্রণে রেখে, স্বয়ংক্রিয়ভাবে তৈরি, পরিবর্তন ও ধ্বংস করার চর্চা।

এই সংজ্ঞার প্রতিটি শব্দ গুরুত্বপূর্ণ। “মেশিন-পাঠযোগ্য সংজ্ঞা ফাইল” মানে মানুষের লেখা নথি নয়, এমন ফাইল যা একটি টুল পড়ে বাস্তব অবকাঠামো বানাতে পারে — Terraform-এ সেই ফাইল লেখা হয় **HCL (HashiCorp Configuration Language)**-এ। “সংস্করণ নিয়ন্ত্রণে রাখা” মানে Git-এ রাখা: কে, কখন, কেন একটি সার্ভারের আকার বদলালেন, তার ইতিহাস থাকে। “স্বয়ংক্রিয়ভাবে তৈরি, পরিবর্তন ও ধ্বংস” মানে একই কোড দিয়ে পরিবেশ বানানো যায়, বদলানো যায়, আবার মুছে ফেলা যায় — হাতে ক্লিক করে নয়।

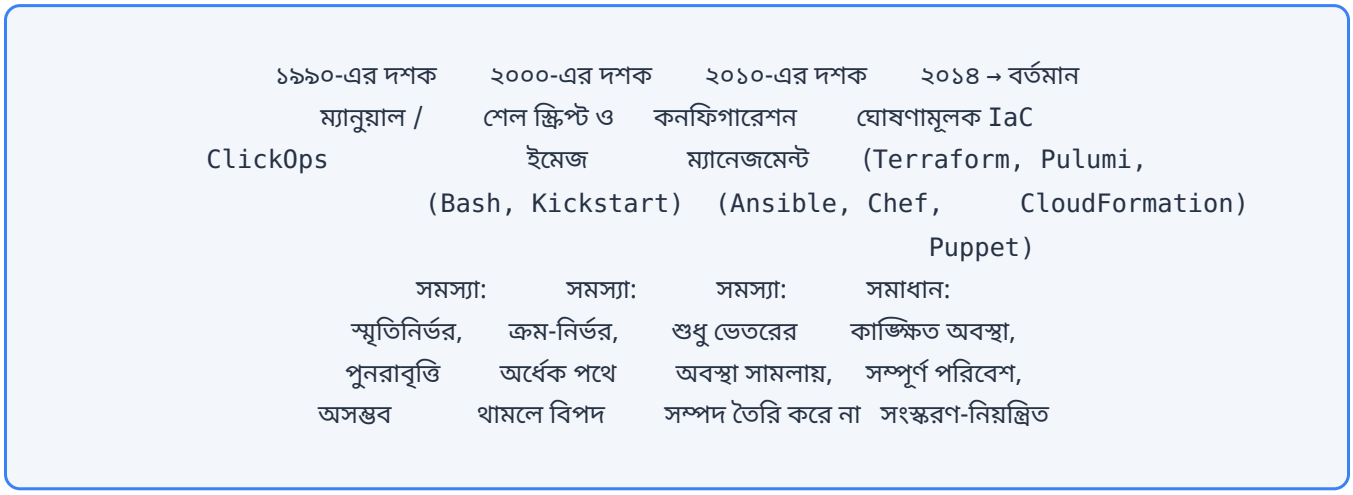
১.৪.১ IaC-এর পরিধি: কী IaC, কী নয়

বিষয়	IaC-এর আওতায় পড়ে কি?	ব্যাখ্যা
সার্ভার, VPC, সাবনেট, নিরাপত্তা গোষ্ঠী তৈরি	হ্যাঁ	এগুলোই মূল অবকাঠামো সম্পদ (Resource)।
অ্যাপ্লিকেশন কোড ডেপ্লয় করা	আংশিক	IaC পরিবেশ বানায়; অ্যাপ ডেপ্লয় CI/CD-র কাজ, তবে দুটো একসাথে চলে।
অপারেটিং সিস্টেমের ভেতরে প্যাকেজ ইনস্টল	আংশিক	এটি কনফিগারেশন ম্যানেজমেন্টের এলাকা; Terraform প্রায়শই এর সাথে যুক্ত হয়।
একবার চালানো শেল স্ক্রিপ্ট	না	স্ক্রিপ্টে কাঙ্ক্ষিত অবস্থা থাকে না, থাকে শুধু ধাপ; এটি IaC নয়।
ক্লাউড কনসোলে হাতে ক্লিক করে বানানো সম্পদ	না	এটি ClickOps — IaC-র ঠিক বিপরীত।

Provisioning: সম্পদ তৈরি ও বরাদ্দ করার প্রক্রিয়া। **Configuration:** সম্পদের ভেতরের অবস্থা ঠিক করা — যেমন কোন প্যাকেজ ইনস্টল থাকবে। Terraform মূলত Provisioning-এর টুল; তবে **Provisioner** দিয়ে সীমিত কনফিগারেশনও করা যায়, যদিও প্রোডাকশনে সেটি এড়িয়ে চলাই ভালো অভ্যাস।

১.৫ ইতিহাস: হাতে বানানো সার্ভার থেকে কোডে লেখা অবকাঠামো

অবকাঠামো পরিচালনা একদিনে IaC হয়নি। প্রতিটি ধাপ আগের ধাপের একটি নির্দিষ্ট যন্ত্রণার উত্তর দিয়েছে।



১.৫.১ ধাপ ১ — ম্যানুয়াল অপারেশন

প্রথম যুগে সিস্টেম অ্যাডমিনিস্ট্রেটর ডেটা সেন্টারে গিয়ে সার্ভার র‍্যাকে বসাতেন, অপারেটিং সিস্টেম ইনস্টল করতেন, হাতে কনফিগার করতেন। জ্ঞান থাকত মানুষের মাথায় আর খাতায়। একজন চলে গেলে জ্ঞানও চলে যেত। প্রতিটি সার্ভার হতো অনন্য — ইংরেজিতে এর নাম **Snowflake Server**, তুষারকণার মতো, যার কোনো দুটি এক নয়।

১.৫.২ ধাপ ২ — স্ক্রিপ্ট ও গোল্ডেন ইমেজ

এরপর এলো শেল স্ক্রিপ্ট ও ডিস্ক ইমেজ। একবার বানানো সার্ভারের ছবি তুলে রেখে তা কপি করা হতো। গতি বাড়ল, কিন্তু স্ক্রিপ্ট দ্বিতীয়বার চালালে কী হবে, তা নিশ্চিত ছিল না; ইমেজ পুরোনো হয়ে গেলে তার ভেতরে কী আছে, কেউ জানত না।

১.৫.৩ ধাপ ৩ — কনফিগারেশন ম্যানেজমেন্ট

Chef, Puppet, Ansible এলো সার্ভারের ভেতরের অবস্থা — কোন প্যাকেজ, কোন ফাইল, কোন সার্ভিস চলবে — ঘোষণা করার জন্য। এটি বড় অগ্রগতি, তবে এগুলো মূলত বিদ্যমান সার্ভারের কনফিগারেশন সামলায়; সার্ভারটি নিজে, নেটওয়ার্কটি, ডেটাবেসটি কোথা থেকে এলো, তা এদের আওতার বাইরে ছিল।

১.৫.৪ ধাপ ৪ — ঘোষণামূলক IaC ও Terraform

২০১৪ সালে HashiCorp Terraform প্রকাশ করে। Terraform সম্পূর্ণ অবকাঠামো — নেটওয়ার্ক থেকে ডেটাবেস পর্যন্ত — একটি ঘোষণামূলক ভাষায় লেখার সুযোগ দিল, সাথে **State** নামে বাস্তব অবস্থার হিসাব রাখার ব্যবস্থা। ক্লাউডের উত্থানের সাথে IaC মূলধারায় পরিণত হলো, কারণ ক্লাউডে সম্পদ API দিয়ে তৈরি হয় — আর API দিয়ে তৈরি করা যায় মানেই কোড দিয়ে তৈরি করা যায়।

১.৬ IaC-এর পাঁচটি প্রকৌশল নীতি

১.৬.১ Idempotence — একই ফল, যতবারই চালান

Idempotence: একটি অপারেশন যতবারই চালানো হোক, প্রথমবারের পর আর কোনো পরিবর্তন হয় না। Terraform-এর `apply` প্রথমবার একটি সার্ভার বানায়; দ্বিতীয়বার চালালে দেখে সার্ভারটি ইতিমধ্যে কাঙ্ক্ষিত অবস্থায় আছে, তাই কিছু করে না। শেল স্ক্রিপ্টে `mkdir /data` দ্বিতীয়বার চালালে ত্রুটি দেয়; `mkdir -p /data` Idempotent। প্রোডাকশনে Idempotence মানে নিরাপত্তা: ভুল করে দুবার চালালেও বিপর্যয় হয় না।

১.৬.২ Version Control — ইতিহাসই সত্যের উৎস

অবকাঠামোর কোড Git-এ থাকলে প্রতিটি পরিবর্তনের লেখক, সময়, কারণ ও পর্যালোচক থাকে। ভুল হলে `git revert` করে আগের অবস্থায় ফেরা যায়। এটি **GitOps**-এর ভিত্তি: Git-ই অবকাঠামোর সত্যের একমাত্র উৎস (**Source of Truth**)।

১.৬.৩ Immutability — বদলানো নয়, বদলে ফেলা

Immutability: চলমান সম্পদকে জায়গায় দাঁড়িয়ে বদলানোর (**Mutate**) বদলে, নতুন সম্পদ বানিয়ে পুরোনোটি সরিয়ে দেওয়া। সার্ভারে SSH করে প্যাকেজ আপডেট করার বদলে নতুন ইমেজ দিয়ে নতুন সার্ভার বানানো হয়। এতে পরিবেশ সবসময় কোডের সাথে মেলে; কেউ "একটু ঠিক করে দিয়েছি" বলে অদৃশ্য পরিবর্তন করতে পারে না।

১.৬.৪ Reproducibility — যেকোনো জায়গায়, একই ফল

একই কোড, একই ইনপুট চলক (**Variable**) দিয়ে Development, Staging ও Production — তিন জায়গায় একই ধরনের পরিবেশ বানানো যায়, শুধু আকার বা সংখ্যা বদলে। TechNova-তে ডেভেলপার তাঁর ল্যাপটপে যে নেটওয়ার্ক কাঠামো পরীক্ষা করেন, প্রোডাকশনে সেই একই কাঠামো — শুধু বড় আকারে — চলে।

১.৬.৫ Reviewability ও Testability — কোডের মতোই পরীক্ষা

অবকাঠামো কোড হওয়ায় তা Pull Request-এ পর্যালোচনা করা যায়, `terraform plan` দিয়ে পরিবর্তনের পূর্বরূপ দেখা যায়, `terraform validate` ও নীতি-পরীক্ষা (**Policy as Code**) দিয়ে ভুল ধরা যায়। কোনো পরিবর্তন প্রোডাকশনে যাওয়ার আগেই তার প্রভাব দেখা যায় — এটাই IaC-র সবচেয়ে বড় নিরাপত্তা।

> **WARNING:** IaC মানেই স্বয়ংক্রিয় নিরাপত্তা নয়। কোডে ভুলভাবে খোলা ফায়ারওয়াল নিয়ম লিখলে সেই ভুলও স্বয়ংক্রিয়ভাবে, দ্রুত, সব পরিবেশে ছড়িয়ে পড়বে। তাই Plan পর্যালোচনা ও নীতি-পরীক্ষা ছাড়া Apply করা প্রোডাকশনে নিষিদ্ধ অভ্যাস।

১.৭ TechNova দৃশ্যকল্প: সমস্যা থেকে সমাধান

TechNova একটি কাল্পনিক সফটওয়্যার কোম্পানি — ডোমেইন `technova.example`, ওয়েব অ্যাপ্লিকেশন, API ও PostgreSQL ডেটাবেস চালায়। আজ তাদের অবস্থা: প্রোডাকশনে তিনটি সার্ভার, প্রতিটি হাতে বানানো; Staging-এ দুটি, তাও ভিন্ন কনফিগারেশনে; ডেভেলপাররা লোকাল পরিবেশে ভিন্নভাবে চালান। নতুন ফিচার প্রোডাকশনে গিয়ে ভাঙে, কারণ পরিবেশ মেলে না। এই ভলিউম জুড়ে আমরা TechNova-র অবকাঠামোকে ধাপে ধাপে কোডে রূপান্তর করব: প্রথমে লোকাল ফাইল দিয়ে Terraform শেখা, তারপর নেটওয়ার্ক, সার্ভার, ডেটাবেস — প্রতিটি অধ্যায়ে একটি করে স্তর।

```
# TechNova-র লক্ষ্য — একই কোড, তিনটি পরিবেশ
# environments/
#   dev/        variables: small size, 1 server
#   staging/    variables: medium size, 2 servers
#   prod/       variables: large size, 3 servers + backup
# মূল কোড একটাই; বদলায় শুধু ইনপুট চলক।
```

১.৮ সাধারণ ভুল

- **IaC = শুধু Terraform ইনস্টল করা:** টুল ইনস্টল করলেই IaC হয় না; কোড Git-এ না থাকলে, Review না হলে, তা শুধু দ্রুততর ClickOps।

- **সবকিছু এক ফাইলে লেখা:** শুরুতে সহজ মনে হলেও বড় হলে রক্ষণাবেক্ষণ অসম্ভব হয়; অধ্যায় ৫-এর কাঠামো অনুসরণ করুন।
- **গোপন তথ্য কোডে লেখা:** পাসওয়ার্ড, API কী কখনোই `.tf` ফাইলে লিখবেন না — প্লেসহোল্ডার `CHANGE-ME-...` ব্যবহার করুন।
- **State ফাইলকে গুরুত্ব না দেওয়া:** State হলো Terraform-এর স্মৃতি; এটি হারালে বা হাতে সম্পাদনা করলে বাস্তবতা ও কোডের মিল ভেঙে যায়।

১.৯ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	করণীয়
“এটা আমার মেশিনে চলে”	পরিবেশের Drift	পরিবেশগুলো একই কোড থেকে বানান; পার্থক্য শুধু চলকে রাখুন।
কে কী বদলেছে জানা যায় না	কোড Git-এ নেই	সব অবকাঠামো কোড Git-এ তুলুন; সরাসরি প্রোডাকশনে পরিবর্তন বন্ধ করুন।
পুনর্নির্মাণে দিন লেগে যায়	অনথিভুক্ত ম্যানুয়াল ধাপ	প্রতিটি ধাপ কোডে লিখুন; পুনর্নির্মাণ পরীক্ষা (Disaster Recovery Drill) করুন।

১.১০ নিরাপত্তা বিবেচনা

IaC কোডেই নিরাপত্তা নীতি লেখা যায়: কোন পোর্ট খোলা থাকবে, কোন ডেটা এনক্রিপ্টেড থাকবে, কার কী অনুমতি থাকবে। কিন্তু কোড রিপোজিটরিতে গোপন তথ্য রাখা যাবে না; **Secret** রাখতে হয় ভল্ট বা পরিবেশ চলকে। কোড পর্যালোচনার সময় নিরাপত্তা-সংবেদনশীল পরিবর্তন — যেমন `0.0.0.0/0` দিয়ে সবাইকে প্রবেশাধিকার — আলাদাভাবে চিহ্নিত করতে হয়।

১.১১ প্রোডাকশন প্রকৌশল চর্চা

- অবকাঠামো কোডের জন্য আলাদা রিপোজিটরি বা মনোরেপোতে আলাদা ডিরেক্টরি রাখুন।
- প্রতিটি পরিবর্তন Pull Request দিয়ে যাক; `terraform plan`-এর আউটপুট PR-এ সংযুক্ত করুন।
- পরিবেশভেদে শুধু চলক বদলান, মূল কোড নয়।
- নিয়মিত পুনর্নির্মাণ পরীক্ষা করুন — কোড থেকে পরিবেশ আবার বানানো যায় কি না।

১.১২ অধ্যায়ের সারাংশ

IaC হলো অবকাঠামোকে কোড হিসেবে লেখা, সংস্করণ নিয়ন্ত্রণে রাখা ও স্বয়ংক্রিয়ভাবে পরিচালনা করা। এর ইতিহাস ম্যানুয়াল অপারেশন থেকে স্ক্রিপ্ট, কনফিগারেশন ম্যানেজমেন্ট হয়ে ঘোষণামূলক IaC-তে এসেছে। এর পাঁচটি নীতি — Idempotence, Version Control, Immutability, Reproducibility ও Reviewability — একসাথে পরিবেশের Drift দূর করে, গতি বাড়ায় এবং ভুল কমায়। TechNova-র মতো যেকোনো দলের জন্য এটাই DevOps-এর ভিত্তি।

১.১৩ জ্ঞান যাচাই

1. IaC-র সংজ্ঞা দিন এবং ClickOps থেকে এর পার্থক্য ব্যাখ্যা করুন।
2. Snowflake Server কী? কেন এটি সমস্যা?
3. Idempotence-এর একটি বাস্তব উদাহরণ দিন — Terraform ছাড়াও।

4. Immutability কেন Mutate করার চেয়ে নিরাপদ?
5. কোন পরিস্থিতিতে IaC ব্যবহার না করাই ভালো হতে পারে? যুক্তি দিন।

১.১৪ ব্যবহারিক অ্যাসাইনমেন্ট

TechNova-র বর্তমান অবকাঠামোর একটি তালিকা বানান: কয়টি সার্ভার, কী কী সার্ভিস, কোন নেটওয়ার্ক নিয়ম — যা আপনি জানেন, একটি Markdown ফাইলে লিখুন। তারপর প্রতিটি আইটেমের পাশে লিখুন: এটি কোডে লেখা যায় কি না, এবং কোডে লিখলে কী সুবিধা হবে। পরের অধ্যায়ে এই তালিকাটিই আপনার মাইগ্রেশন পরিকল্পনার ভিত্তি হবে।

অধ্যায় ২ • PART I — Infrastructure as Code Foundations

Chapter 2 — Traditional Infrastructure vs Automated Infrastructure

২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — ঐতিহ্যবাহী ম্যানুয়াল অবকাঠামো ও স্বয়ংক্রিয় অবকাঠামোর পার্থক্য খরচ, গতি, ভুল ও স্কেলের দিক থেকে তুলনা করতে; **Snowflake Server**, **Drift** ও **Ticket-Driven** কাজের ঝুঁকি ব্যাখ্যা করতে; এবং TechNova-র জন্য একটি বাস্তবসম্মত মাইগ্রেশন পথ আঁকতে।

- ম্যানুয়াল প্রক্রিয়ার প্রতিটি ধাপ চিহ্নিত করা।
- স্বয়ংক্রিয় প্রক্রিয়ার সুবিধা সংখ্যায় ও উদাহরণে দেখানো।
- কখন ম্যানুয়াল হস্তক্ষেপ এখনও প্রয়োজন, তা বলা।

২.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

অনেক দল অ্যাপ্লিকেশন ডেপ্লয় স্বয়ংক্রিয় করে, কিন্তু অবকাঠামো হাতে বানায় — ফলে ডেপ্লয় দ্রুত হলেও পরিবেশ তৈরি ধীর থাকে। নতুন পরিবেশ চাইলে টিকিট কাটতে হয়, অপেক্ষা করতে হয়, তারপর কেউ হাতে বানিয়ে দেয়। এই বোতল-ঘাড় (Bottleneck) দূর না করলে DevOps-এর গতি কখনোই পুরো পাইপলাইনে পৌঁছায় না।

> **CONCEPT:** Traditional Infrastructure মানে খারাপ প্রকৌশলী নয় — মানে এমন প্রক্রিয়া যেখানে জ্ঞান মানুষের মাথায়, পরিবর্তন অপরীক্ষিত, এবং পুনরাবৃত্তি মানুষের স্মৃতির উপর নির্ভরশীল। Automated Infrastructure মানে সেই একই জ্ঞানকে কোডে, পরীক্ষায় ও পর্যালোচনায় বাঁধা।

> **REAL WORLD:** TechNova-তে নতুন ডেভেলপার যোগ দিলে তাঁর লোকাল পরিবেশ তৈরি করতে দুই দিন লাগত — কেউ Wiki পড়ে, কেউ Slack-এ জিজ্ঞেস করে। স্বয়ংক্রিয় সেটআপের পর একই কাজ হয় এক কমান্ডে, আধ ঘণ্টায়, এবং সবার পরিবেশ একরকম হয়।

২.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ১-এর IaC ধারণা, টিকিটভিত্তিক কাজ ও লিনাক্স সার্ভারের প্রাথমিক ধারণা থাকলেই চলবে।

২.৪ ঐতিহ্যবাহী অবকাঠামো কেমন দেখায়

Traditional Infrastructure: মানুষের হাতে, টিকিটের নির্দেশনায়, কনসোল বা SSH দিয়ে ধাপে ধাপে তৈরি অবকাঠামো। এর বৈশিষ্ট্য: **Ticket-Driven** — প্রতিটি পরিবর্তন একটি টিকিট দিয়ে শুরু হয়; **Undocumented** — কী করা হলো, তার পূর্ণ রেকর্ড থাকে না; **Snowflake** — প্রতিটি সার্ভার অনন্য; **Slow** — দিন থেকে সপ্তাহ লাগে; **Fragile** — একই কাজ দ্বিতীয়বার একইভাবে হয় না।

ঐতিহ্যবাহী প্রবাহ:
ডেভেলপার → টিকিট → অপেক্ষা → অ্যাডমিন SSH-তে ঢোকেন
→ হাতে কমান্ড চালান → মনে রাখা কনফিগারেশন
→ "হয়ে গেছে" (কোনো রেকর্ড নেই) → পরের বার আবার শুরু

স্বয়ংক্রিয় প্রবাহ:
ডেভেলপার → `Pull Request` (কোড পরিবর্তন) → `Review + Plan`
→ `Apply` → `Git`-এ রেকর্ড → একই কোডে পুনর্নির্মাণ সম্ভব

২.৪.১ Drift: নীরবে সরে যাওয়া

Drift: বাস্তব অবকাঠামো ধীরে ধীরে নথি বা কোডে লেখা অবস্থা থেকে সরে যাওয়া। কেউ জরুরি প্রয়োজনে ফায়ারওয়ালে একটি পোর্ট খুললেন, কেউ ডিস্ক বাড়ালেন — কোনোটাই নথিতে এলো না। মাসখানেক পর কেউ জানে না প্রোডাকশন আসলে কেমন। Terraform-এ Drift ধরা যায় `terraform plan` দিয়ে: কোডের বাইরে কেউ কিছু বদলালে Plan তা দেখিয়ে দেয়।

২.৫ স্বয়ংক্রিয় অবকাঠামো কেমন দেখায়

Automated Infrastructure: অবকাঠামোর সংজ্ঞা কোডে, পরিবর্তন Pull Request-এ, প্রয়োগ স্বয়ংক্রিয় পাইপলাইনে। বৈশিষ্ট্য: **Repeatable** — একই কোডে একই ফল; **Auditable** — Git ইতিহাসে সবকিছু; **Fast** — মিনিটে পরিবেশ; **Testable** — প্রয়োগের আগে Plan দেখা যায়; **Recoverable** — ধ্বংস হলে আবার বানানো যায়।

দিক	ঐতিহ্যবাহী	স্বয়ংক্রিয় (IaC)
নতুন পরিবেশ	দিন–সপ্তাহ, টিকিটের অপেক্ষা	মিনিট–ঘণ্টা, এক কমান্ড
পুনরাবৃত্তি	মানুষভেদে ভিন্ন ফল	একই কোডে একই ফল
পরিবর্তনের রেকর্ড	টিকিট, চ্যাট, স্মৃতি	Git ইতিহাস, Review, Plan আউটপুট
ভুলের হার	টাইপো, ভুলে যাওয়া ধাপ	কোড Review ও পরীক্ষায় ধরা পড়ে
স্কেল	১০ সার্ভারেই হিমশিম	১০০ সার্ভারেও একই প্রক্রিয়া
পুনরুদ্ধার	জ্ঞান হারালে অসম্ভব	কোড থাকলেই পুনর্নির্মাণ
খরচ	মানুষের সময় বেশি, অদৃশ্য খরচ	শুরুতে বিনিয়োগ, পরে কম খরচ

২.৬ খরচ, ভুল ও স্কেল: সংখ্যায় ভাবা

ধরুন একটি সার্ভার হাতে বানাতে ৪ ঘণ্টা লাগে, ভুলের হার ২০%। ২০টি সার্ভারে তা ৮০ ঘণ্টা ও প্রায় ৪টি ভুল। IaC-তে প্রথমবার কোড লিখতে হয়তো ৮ ঘণ্টা লাগবে, কিন্তু পরের ১৯টি সার্ভার বানাতে লাগবে মিনিটখানেক করে, ভুল প্রায় শূন্য — কারণ ভুল থাকলে প্রথম সার্ভারেই ধরা পড়ে, আর ঠিক করলে সব জায়গায় ঠিক হয়। এটাই স্বয়ংক্রিয়তার অর্থনীতি: খরচ একবার, সুবিধা বারবার।

> **WARNING:** স্বয়ংক্রিয়তা ভুলকেও দ্রুত ছড়ায়। ভুল কোডে ১টির বদলে ১০০টি সার্ভার বানালে ক্ষতি ১০০ গুণ। তাই Apply-এর আগে Plan দেখা, ছোট পরিসরে পরীক্ষা করা, এবং ধ্বংসাত্মক পরিবর্তনে আলাদা অনুমোদন রাখা বাধ্যতামূলক।

২.৭ কখন ম্যানুয়াল ধাপ এখনও থাকে

সবকিছু স্বয়ংক্রিয় করা যায় না, আর করা উচিতও নয়। প্রথমবার ক্লাউড অ্যাকাউন্ট খোলা, রুট অ্যাকাউন্টের নিরাপত্তা, হার্ডওয়্যার স্থাপন, জরুরি বিভ্রাটে (Incident) তাৎক্ষণিক হস্তক্ষেপ — এগুলোতে মানুষের সিদ্ধান্ত লাগে। নিয়মটি হলো: ম্যানুয়াল ধাপটি ব্যতিক্রম হিসেবে নথিভুক্ত করুন, তারপর যত দ্রুত সম্ভব তাকে কোডে তুলুন। “একবারের জরুরি ঠিক” যদি কোডে না ওঠে, তবে সেটিই পরের Drift-এর বীজ।

২.৮ মাইগ্রেশন পথ: TechNova কীভাবে এগোবে

```
# ধাপ ১ — তালিকা: বর্তমান সম্পদগুলো লিখুন (অধ্যায় ১-এর অ্যাসাইনমেন্ট)
# ধাপ ২ — নতুন পরিবেশ প্রথমে কোডে বানান; পুরোনোটি হাতে রাখুন
# ধাপ ৩ — একটি ছোট সার্ভিস (যেমন স্টেজিং) কোডে সরান, পরীক্ষা করুন
# ধাপ ৪ — প্রোডাকশন সরান; পুরোনো হাতে-বানানো অংশ ধীরে ধীরে বন্ধ করুন
# ধাপ ৫ — সব পরিবর্তন শুধু কোড দিয়ে; হাতে পরিবর্তন নিষিদ্ধ ঘোষণা করুন
```

একসাথে সবকিছু সরানোর চেষ্টা করবেন না। ছোট, বিপরীতমুখী (Reversible) ধাপে এগোন — প্রতিটি ধাপে শিখুন, পরের ধাপে প্রয়োগ করুন।

২.৯ সাধারণ ভুল

- **সবকিছু একসাথে স্বয়ংক্রিয় করা:** বড় ব্যাং (Big Bang) মাইগ্রেশন ব্যর্থতার ঝুঁকি বাড়ায়।
- **পুরোনো প্রক্রিয়া রেখে টুল যোগ করা:** টিকিট ও কোড দুটোই চললে কোনটি সত্য, তা নিয়ে বিভ্রান্তি হয়।
- **ডকুমেন্টেশন না লেখা:** কোড নিজেই নথি, তবে কেন এমন সিদ্ধান্ত নেওয়া হলো, তা README-তে লিখতে হয়।

২.১০ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	করণীয়
কোড ও বাস্তবতা মেলে না	হাতে পরিবর্তন হয়েছে	`terraform plan` দিয়ে Drift দেখুন; হাতে পরিবর্তন বন্ধ করুন।
মাইগ্রেশন থেমে আছে	পরিধি খুব বড়	সবচেয়ে ছোট সার্ভিসটি বেছে আবার শুরু করুন।
দল কোড ব্যবহার করে না	প্রশিক্ষণ বা মালিকানা নেই	একজন মালিক (Owner) ঠিক করুন; Pair করে কাজ করুন।

২.১১ নিরাপত্তা বিবেচনা

ম্যানুয়াল প্রক্রিয়ায় কে সার্ভারে ঢুকল, কী করল — তার রেকর্ড প্রায় থাকে না। স্বয়ংক্রিয় প্রক্রিয়ায় Git ইতিহাস ও CI লগই অডিট ট্রেইল। তবে কোড রিপোজিটরিতে প্রবেশাধিকারই এখন অবকাঠামোর প্রবেশাধিকার — তাই রিপোজিটরিতে **Branch Protection**, অনুমোদন ও গোপন তথ্য স্ক্যানিং বাধ্যতামূলক।

২.১২ প্রোডাকশন প্রকৌশল চর্চা

- “কোডই সত্য” — প্রোডাকশনে হাতে পরিবর্তন নিষিদ্ধ করুন, ব্যতিক্রম নথিভুক্ত করুন।
- নিয়মিত Drift পরীক্ষা চালান (`terraform plan` নিয়মিতভাবে)।
- মাইগ্রেশনের প্রতিটি ধাপে Rollback পরিকল্পনা রাখুন।

২.১৩ অধ্যায়ের সারাংশ

ঐতিহ্যবাহী অবকাঠামো ধীর, অনথিভুক্ত ও Drift-প্রবণ; স্বয়ংক্রিয় অবকাঠামো পুনরাবৃত্তিযোগ্য, নিরীক্ষাযোগ্য ও দ্রুত। পার্থক্যটি টুলের নয়, প্রক্রিয়ার: জ্ঞান মানুষের মাথা থেকে কোডে সরানো। মাইগ্রেশন ছোট ধাপে, নতুন পরিবেশ দিয়ে শুরু করে, ধীরে ধীরে পুরোনোকে প্রতিস্থাপন করে করতে হয়।

২.১৪ জ্ঞান যাচাই

1. Ticket-Driven কাজের তিনটি সমস্যা লিখুন।
2. Drift কীভাবে তৈরি হয়? একটি উদাহরণ দিন।
3. স্বয়ংক্রিয়তা কীভাবে খরচ কমায়, সংখ্যার উদাহরণ দিয়ে বোঝান।
4. কোন তিনটি কাজে ম্যানুয়াল হস্তক্ষেপ এখনও যুক্তিসঙ্গত?

২.১৫ ব্যবহারিক অ্যাসাইনমেন্ট

অধ্যায় ১-এর তালিকাটি নিয়ে মাইগ্রেশন ক্রম ঠিক করুন: কোন সম্পদটি প্রথমে কোডে নেবেন, কোনটি শেষে — এবং কেন। প্রতিটির পাশে ঝুঁকি (উচ্চ/মাঝারি/নিম্ন) ও Rollback উপায় লিখুন।

অধ্যায় ৩ • PART I — Infrastructure as Code Foundations

Chapter 3 — Declarative vs Imperative Infrastructure Management

৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — **Declarative** ও **Imperative** পদ্ধতির পার্থক্য নিজের ভাষায় ব্যাখ্যা করতে; একই কাজ Terraform HCL-এ ও শেল স্ক্রিপ্টে লিখে তুলনা করতে; **Convergence** ও **Drift Detection** কীভাবে কাজ করে, তা বলতে; এবং কেন Terraform ঘোষণামূলক পদ্ধতি বেছেছে, তার যুক্তি দিতে।

৩.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

IaC-র সবচেয়ে গভীর সিদ্ধান্তটি হলো — আপনি কী লিখবেন: শেষ অবস্থা, নাকি সেখানে পৌঁছানোর ধাপগুলো? এই সিদ্ধান্তই নির্ধারণ করে ভুল হলে কী হবে, দ্বিতীয়বার চালালে কী হবে, আর কেউ হাতে কিছু বদলালে সিস্টেম তা ধরতে পারবে কি না। Terraform-এর প্রায় সব সুবিধা — Plan দেখা, Drift ধরা, নিরাপদ পরিবর্তন — আসে ঘোষণামূলক মডেল থেকে।

> **CONCEPT:** Declarative মানে “আমি কী চাই, তা বলা”; Imperative মানে “কীভাবে করতে হবে, ধাপে ধাপে বলা”। রেস্তোরাঁয় “এক প্লেট ভাত চাই” বলা Declarative; রাঁধুনিকে “চাল ধোও, পানি দাও, চুলা জ্বালাও...” বলা Imperative। প্রথমটিতে রাঁধুনি (ইঞ্জিন) পদ্ধতি ঠিক করে; দ্বিতীয়টিতে ভুল ধাপ মানেই ভুল ফল।

> **REAL WORLD:** TechNova-র পুরোনো সেটআপ স্ক্রিপ্টে লেখা ছিল: “সার্ভার বানাও, তারপর প্যাকেজ ইনস্টল করো, তারপর ফাইল কপি করো।” মাঝপথে নেটওয়ার্ক ছিঁড়ে গেলে স্ক্রিপ্ট থামত, আর আবার চালালে প্রথম ধাপেই “সার্ভার ইতিমধ্যে আছে” ত্রুটি দিত। Terraform-এ একই কাজ ঘোষণা করলে ইঞ্জিন নিজেই বুঝত কোন অংশ হয়েছে, কোনটি বাকি।

৩.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ১–২, শেল স্ক্রিপ্টের প্রাথমিক ধারণা (`apt install`, `mkdir`) ও HCL-এর নামটুকু জানলেই চলবে।

৩.৪ Imperative: ধাপের তালিকা

Imperative Infrastructure Management: কাঙ্ক্ষিত অবস্থায় পৌঁছানোর প্রতিটি ধাপ ক্রমানুসারে লেখা। চালককে (Executor) কিছু ভাবতে হয় না — শুধু ধাপ অনুসরণ করতে হয়। সুবিধা: নিয়ন্ত্রণ সম্পূর্ণ, যেকোনো জটিল ক্রম লেখা যায়। অসুবিধা: ধাপগুলো বর্তমান অবস্থা জানে না; মাঝপথে থামলে আবার শুরু করা কঠিন; একই স্ক্রিপ্ট দ্বিতীয়বার চালালে প্রায়ই ত্রুটি হয়।

```
#!/bin/bash
# Imperative: TechNova ওয়েব সার্ভার সেটআপ (সরলীকৃত)
apt-get update
apt-get install -y nginx
mkdir /var/www/technova
echo "Hello TechNova" > /var/www/technova/index.html
```

```
systemctl start nginx
# সমস্যা: দ্বিতীয়বার চালালে? mkdir ত্রুটি দেবে না (-p নেই),
# কিন্তু apt আবার চলবে, start আবার চলবে — অবস্থা যাচাই করা হয় না।
```

৩.৫ Declarative: কাঙ্ক্ষিত শেষ অবস্থা

Declarative Infrastructure Management: শেষ অবস্থাটি ঘোষণা করা; কীভাবে সেখানে পৌঁছাতে হবে, তা ইঞ্জিন ঠিক করে। ইঞ্জিন বর্তমান অবস্থা পড়ে, কাঙ্ক্ষিত অবস্থার সাথে তুলনা করে, শুধু পার্থক্যটুকু (Diff) বাস্তবায়ন করে। একই ঘোষণা যতবারই চালান, ফল একই — এটাই **Convergence**: সিস্টেম বারবার কাঙ্ক্ষিত অবস্থার দিকে ফিরে আসে।

```
# Declarative: Terraform HCL-এ একই উদ্দেশ্য (ধারণাগত)
resource "local_file" "index" {
  filename = "/tmp/technova/index.html" # কোথায় ফাইলটি থাকবে
  content  = "Hello TechNova"           # ফাইলের ভেতরে কী থাকবে
}
# এখানে কোনো ধাপ নেই — শুধু বলা আছে ফাইলটি কোথায়, কী থাকবে।
# Terraform নিজেই দেখবে ফাইলটি আছে কি না, মিলছে কি না।
```

উপরের ব্লকে `resource` মানে পরিচালনাযোগ্য সম্পদ; `"local_file"` হলো সম্পদের ধরন; `"index"` হলো এই কনফিগারেশনে তার নাম; `filename` ও `content` হলো তার **Argument** — অর্থাৎ বৈশিষ্ট্য। বিস্তারিত ব্যাখ্যা অধ্যায় ৪–৫-এ আসবে।

দিক	Imperative	Declarative
লেখা হয়	ধাপের ক্রম	কাঙ্ক্ষিত শেষ অবস্থা
দ্বিতীয়বার চালালে	প্রায়ই ত্রুটি বা অপ্রয়োজনীয় কাজ	কোনো পরিবর্তন নেই (Idempotent)
মাঝপথে ব্যর্থতা	অর্ধেক অবস্থায় আটকে যায়	পরেরবার পার্থক্যটুকু শেষ করে
পরিবর্তনের পূর্বরূপ	দেখা যায় না	Plan-এ দেখা যায়
Drift ধরা	নিজে থেকে পারে না	বর্তমান বনাম কাঙ্ক্ষিত তুলনায় ধরা পড়ে
উদাহরণ	Bash, Python স্ক্রিপ্ট	Terraform HCL, Kubernetes YAML

৩.৬ Idempotence, Convergence ও Drift Detection — ভেতরের যন্ত্র

Declarative ইঞ্জিন তিনটি কাজ করে: (১) **Read** — বর্তমান অবস্থা পড়ে (Terraform-এ State ও Provider API থেকে); (২) **Compare** — কাঙ্ক্ষিত অবস্থার সাথে মেলায়; (৩) **Apply Diff** — শুধু পার্থক্য বাস্তবায়ন করে। কেউ হাতে ফাইলটি বদলে ফেললে পরের `plan`-এ তা Drift হিসেবে দেখা যাবে, আর `apply` তা আবার কাঙ্ক্ষিত অবস্থায় ফিরিয়ে আনবে — এটাই Convergence। Imperative স্ক্রিপ্টে এই তুলনার ধাপটিই নেই, তাই Drift অদৃশ্য থেকে যায়।

```
Declarative চক্র:
কাঙ্ক্ষিত অবস্থা (HCL)
        ↓
বর্তমান অবস্থা পড়া (State + API)
        ↓
তুলনা → পার্থক্য (Diff / Plan)
        ↓
শুধু পার্থক্য প্রয়োগ (Apply)
        ↓
আবার কাঙ্ক্ষিত অবস্থায় ফেরা (Converged)
```

৩.৭ ট্রেড-অফ: কখন কোনটি

Declarative সবসময় সেরা নয়। একবারের জটিল মাইগ্রেশন, শর্তসাপেক্ষ ধাপ, বা ইঞ্জিনের আওতার বাইরের কাজে Imperative স্ক্রিপ্ট প্রয়োজন হয়। Terraform-ও তাই **Provisioner** ও `null_resource`-এর মতো পালানোর পথ (Escape Hatch) রাখে — তবে সেগুলো ব্যতিক্রম, নিয়ম নয়। নিয়ম: যা ঘোষণা করা যায়, তা ঘোষণা করুন; যা যায় না, তা ছোট স্ক্রিপ্টে লিখে ঘোষণামূলক কাঠামোর ভেতরে বাঁধুন।

> **WARNING:** Declarative কোডে ধাপের ক্রম লুকিয়ে থাকে বলে অনেকে ভাবেন ক্রমের কোনো গুরুত্ব নেই। আসলে নির্ভরতা (Dependency) থাকে — ডেটাবেসের আগে নেটওয়ার্ক লাগে। Terraform নির্ভরতা নিজে বের করে (অধ্যায় ৪), তবে ভুল নির্ভরতা লিখলে Plan ভুল ক্রমে চলতে পারে।

৩.৮ কেন Terraform Declarative বেছেছে

Terraform-এর লক্ষ্য ছিল বহু ক্লাউড ও সার্ভিসে একই ভাষায় অবকাঠামো লেখা। Imperative হলে প্রতিটি ক্লাউডের API ধাপ আলাদা করে লিখতে হতো। Declarative মডেলে ব্যবহারকারী শুধু "কী চাই" বলেন; **Provider** জানে "কীভাবে" সেই ক্লাউডে তা বানাতে হয়। ফলে একই মানসিক মডেলে AWS, Azure, লোকাল ফাইল — সব পরিচালনা করা যায়।

৩.৯ সাধারণ ভুল

- **HCL-কে স্ক্রিপ্ট ভাবা:** HCL-এ লুপ বা শর্ত থাকলেও তা ধাপ নয় — তা কাঙ্ক্ষিত অবস্থার বর্ণনা।
- **সবকিছু Provisioner দিয়ে করা:** Provisioner Imperative পালানোর পথ; বেশি ব্যবহার করলে Declarative সুবিধা হারিয়ে যায়।
- **Plan না দেখে Apply:** Declarative-র সবচেয়ে বড় সুবিধা — পূর্বরূপ — না দেখে Apply করা মানে সুবিধাটিই ফেলে দেওয়া।

৩.১০ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	করণীয়

Apply করেও অবস্থা বদলায় না	কোডে সেই বৈশিষ্ট্য ঘোষণা করা হয়নি	কাঙ্ক্ষিত বৈশিষ্ট্যটি HCL-এ যোগ করুন।
প্রতিবার Plan-এ একই পরিবর্তন দেখায়	বাইরে কেউ বদলাচ্ছে, বা Provider মান স্বাভাবিক করছে	Drift-এর উৎস খুঁজুন; প্রয়োজনে `ignore_changes` বিবেচনা করুন (সাবধানে)।
স্ক্রিপ্টের মতো ক্রম নিয়ন্ত্রণ করতে চান	নির্ভরতা অনুপস্থিত	`depends_on` বা রেফারেন্স দিয়ে নির্ভরতা প্রকাশ করুন।

৩.১১ নিরাপত্তা বিবেচনা

Declarative কোডে নিরাপত্তা নীতি — যেমন কোন পোর্ট বন্ধ থাকবে — ঘোষণা হিসেবে থাকে, তাই পর্যালোচনায় ধরা পড়ে। Imperative স্ক্রিপ্টে একই নিয়ম ধাপের ভিড়ে হারিয়ে যেতে পারে। তবে ঘোষণায় ভুল থাকলে তা প্রতিটি পরিবেশে একইভাবে ছড়ায় — তাই নীতি-পরীক্ষা অপরিহার্য।

৩.১২ প্রোডাকশন প্রকৌশল চর্চা

- কাঙ্ক্ষিত অবস্থা ছোট, স্পষ্ট ব্লকে লিখুন; একটি ব্লক একটি সম্পদ।
- Provisioner এড়িয়ে চলুন; প্রয়োজনে কনফিগারেশন ম্যানেজমেন্ট টুল ব্যবহার করুন।
- প্রতিটি Apply-এর আগে Plan সংরক্ষণ করুন — সেটিই পরিবর্তনের প্রমাণ।

৩.১৩ অধ্যায়ের সারাংশ

Imperative বলে কীভাবে করতে হবে; Declarative বলে কী চাই। Declarative ইঞ্জিন বর্তমান ও কাঙ্ক্ষিত অবস্থা মিলিয়ে শুধু পার্থক্য প্রয়োগ করে — তাই তা Idempotent, Convergence-প্রবণ ও Drift ধরতে সক্ষম। Terraform এই মডেল বেছেছে বলেই Plan, Drift Detection ও বহু-ক্লাউড পরিচালনা সম্ভব হয়েছে।

৩.১৪ জ্ঞান যাচাই

1. একটি ফাইল তৈরি করার কাজ Imperative ও Declarative — দুইভাবে লিখুন।
2. Convergence বলতে কী বোঝায়? উদাহরণ দিন।
3. কেন Imperative স্ক্রিপ্ট Drift ধরতে পারে না?
4. কখন Provisioner ব্যবহার যুক্তিসঙ্গত?

৩.১৫ ব্যবহারিক অ্যাসাইনমেন্ট

আপনার লোকাল মেশিনে একটি শেল স্ক্রিপ্ট লিখুন যা একটি ডিরেক্টরি ও একটি ফাইল বানায়। সেটি দুবার চালিয়ে কী হয়, লক্ষ করুন। তারপর একই কাজের Declarative বর্ণনা (কাগজে HCL-এর মতো করে) লিখুন — ধাপ নয়, শুধু শেষ অবস্থা। দুটির পার্থক্য তিন লাইনে লিখুন।

অধ্যায় ৪ • PART I — Infrastructure as Code Foundations

Chapter 4 — Terraform Architecture — Providers, Resources, Data Sources, and Plugins

৪.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — **Terraform Core**, **Provider**, **Resource**, **Data Source**, **State** ও **Module**-এর ভূমিকা আলাদা করতে; `init` কীভাবে Provider ডাউনলোড করে, তা বলতে; এবং Plan→Apply পাইপলাইনের প্রতিটি ধাপ ব্যাখ্যা করতে।

৪.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

Terraform ব্যবহার করা যায় না-বুঝে — কিন্তু ভুল হলে বুঝতে হয় ভেতরটা। Plan কেন একটি পরিবর্তন দেখাল, State কেন গুরুত্বপূর্ণ, Provider কেন আলাদা প্রোগ্রাম — এগুলো না জানলে সমস্যায় অন্ধভাবে চেষ্টা করতে হয়। স্থাপত্য (Architecture) জানলে সমস্যা সমাধান হয় যুক্তি দিয়ে।

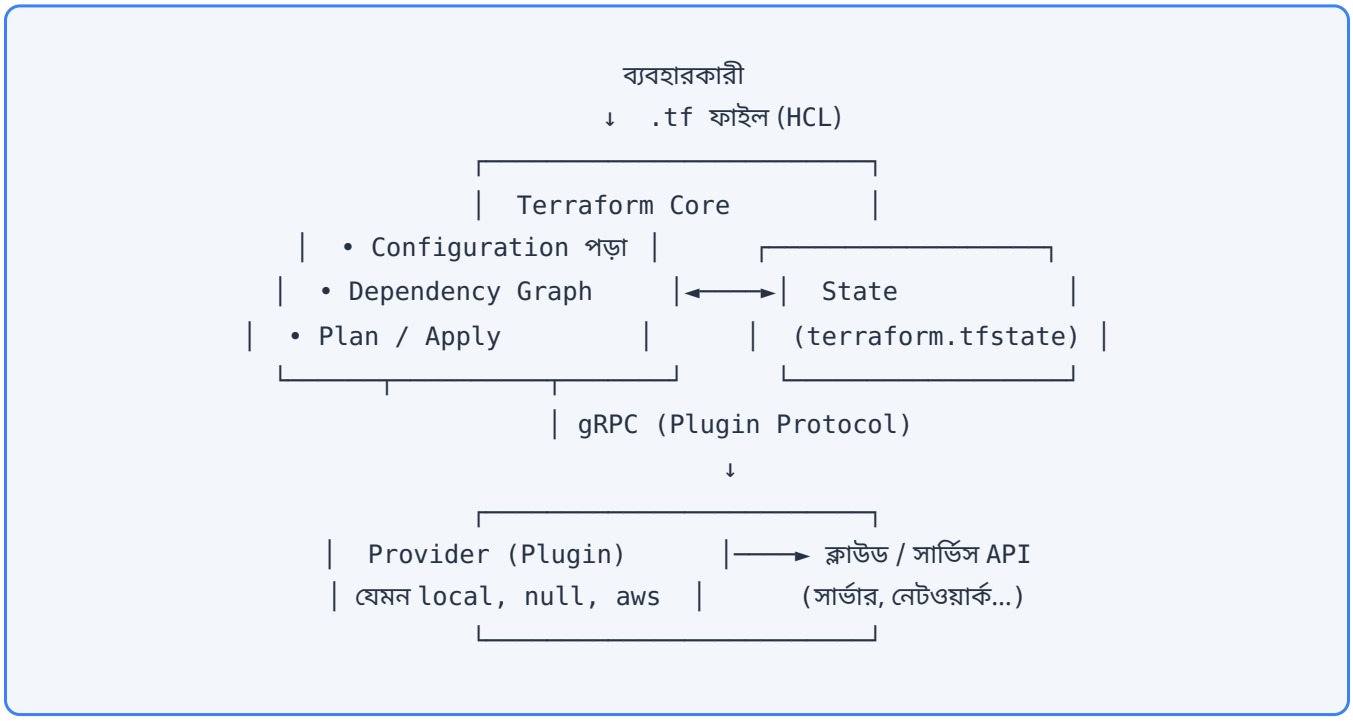
CONCEPT: Terraform Core নিজে কোনো ক্লাউড চেনে না। Core শুধু জানে — কনফিগারেশন পড়া, নির্ভরতার গ্রাফ বানানো, State-এর সাথে তুলনা করা, আর পরিকল্পনা করা। ক্লাউডের সাথে কথা বলে Provider — একটি আলাদা প্লাগইন প্রোগ্রাম। এই বিচ্ছেদই Terraform-কে বহু ক্লাউডে চলতে দেয়।

REAL WORLD: TechNova-তে একবার Plan-এ একটি ডেটাবেস মুছে যাওয়ার কথা দেখা গেল। কারণ ছিল ভুল Resource নাম — Core শুধু গ্রাফ অনুযায়ী কাজ করছিল। স্থাপত্য বুঝে দলটি বুঝল: সমস্যা Provider-এ নয়, কনফিগারেশনে — আর Plan দেখার অভ্যাসই বিপর্যয় ঠেকিয়েছিল।

৪.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ৩-এর Declarative ধারণা, HCL ব্লকের প্রাথমিক ধারণা ও API বলতে কী বোঝায়, তা জানলেই চলবে।

৪.৪ স্থাপত্যের মানচিত্র



Terraform Core: মূল বাইনারি — কনফিগারেশন পার্স করা, গ্রাফ বানানো, Plan ও Apply চালানো। **Provider:** একটি প্লাগইন — নির্দিষ্ট প্ল্যাটফর্মের API-র সাথে কথা বলার অনুবাদক। **Plugin Protocol:** Core ও Provider-এর মধ্যে gRPC-ভিত্তিক যোগাযোগের নিয়ম। Core Provider-কে বলে “এই Resource বানাও”, Provider API কল করে ফল জানায়।

৪.৫ Resource বনাম Data Source

Resource: Terraform যা তৈরি, পরিচালনা ও মুছে ফেলে — যেমন `local_file`, সার্ভার, VPC। **Data Source:** যা Terraform তৈরি করে না, শুধু পড়ে — যেমন বিদ্যমান নেটওয়ার্কের তথ্য, একটি ফাইলের বিষয়বস্তু। Data Source দিয়ে বাইরের তথ্য কনফিগারেশনে আনা যায়, কিন্তু তার জীবনচক্র Terraform নিয়ন্ত্রণ করে না।

```
resource "local_file" "app" {   # Terraform বানাবে ও মুছবে
  filename = "/tmp/technova/app.txt"
  content  = "TechNova App"
}
data "local_file" "existing" {  # শুধু পড়বে, বানাবে না
  filename = "/tmp/technova/app.txt"
}
# data.local_file.existing.content দিয়ে পড়া মান ব্যবহার করা যায়।
```

৪.৬ Configuration বনাম State

বিষয়	Configuration (.tf)	State (.tfstate)
কী	কাঙ্ক্ষিত অবস্থা	বাস্তব অবস্থার রেকর্ড
লেখে কে	মানুষ	Terraform (Apply-এর সময়)
Git-এ যায় কি	হ্যাঁ	না (লোকালে; দূরবর্তী Backend-এ থাকে)

হাতে সম্পাদনা	স্বাভাবিক	নিষিদ্ধ — স্বাভাবিক কাজ নয়

State: প্রতিটি Resource-এর বাস্তব পরিচয় (ID), বৈশিষ্ট্য ও নির্ভরতার হিসাব। Terraform Plan করার সময় State পড়ে জানতে পারে বাস্তবে কী আছে। State হারালে Terraform মনে করবে কিছুই নেই — আর সবকিছু আবার বানাতে চাইবে। তাই State-এর ব্যাকআপ ও **Backend** (State রাখার জায়গা) অত্যন্ত গুরুত্বপূর্ণ; বিস্তারিত পরের ভলিউম অংশে আসবে।

> **WARNING:** State ফাইল কখনোই হাতে সম্পাদনা করবেন না, Git-এ তুলবেন না, বা অন্যের সাথে শেয়ার করবেন না — তাতে গোপন তথ্য (Secret) থাকতে পারে। সমস্যা হলে `terraform state` কমান্ডের নিরাপদ উপ-কমান্ড ব্যবহার করুন, তাও সাবধানে।

৪.৭ Dependency Graph: ক্রম কীভাবে ঠিক হয়

Terraform কনফিগারেশনের সব Resource ও তাদের রেফারেন্স পড়ে একটি **Dependency Graph** (নির্ভরতার গ্রাফ) বানায় — কোনটির আগে কোনটি লাগবে, তার মানচিত্র। যেমন ফাইলের নাম যদি আরেকটি Resource-এর আউটপুট থেকে আসে, Terraform বুঝবে প্রথমটি আগে বানাতে হবে। স্বাধীন Resource-গুলো সমান্তরালে (Parallel) বানানো যায়, তাই Apply দ্রুত হয়। স্পষ্ট ক্রম দরকার হলে `depends_on` ব্যবহার করা যায়, তবে রেফারেন্স দিয়েই নির্ভরতা প্রকাশ করা ভালো অভ্যাস।

৪.৮ terraform init: Provider কীভাবে আসে

`terraform init` চালালে Core: (১) `required_providers` ব্লক পড়ে কোন Provider, কোন সংস্করণ লাগবে, তা দেখে; (২) রেজিস্ট্রি থেকে Provider বাইনারি ডাউনলোড করে `.terraform/` ডিরেক্টরিতে রাখে; (৩) `.terraform.lock.hcl`-এ সংস্করণ ও হ্যাশ লিখে রাখে, যাতে সবাই একই সংস্করণ পায়; (৪) Backend ও Module প্রস্তুত করে। Init ছাড়া Plan/Apply চলবে না — কারণ Provider ছাড়া Core ক্লাউডের ভাষা জানে না।

৪.৯ Plan → Apply পাইপলাইন

```
write (.tf) → init (Provider ডাউনলোড)
      → validate (সিনট্যাক্স ও ধরন যাচাই)
→ plan (State পড়া → Diff → পরিকল্পনা দেখানো)
→ apply (পরিকল্পনা অনুমোদন → Provider API কল
                  → State আপডেট)
        → show / output (ফল দেখা)
```

Plan পর্যায়ে Terraform বর্তমান State পড়ে, কনফিগারেশনের সাথে মেলায়, এবং কী তৈরি (+), কী বদলাবে (~), কী মুছবে (-) — তা দেখায়, কিন্তু কিছু বদলায় না। Apply পর্যায়ে সেই পরিকল্পনাই বাস্তবায়িত হয়। এই দুই ধাপের বিচ্ছেদই নিরাপত্তা: দেখে, বুঝে, তারপর বদলানো।

৪.১০ Module ও Workspace: সংক্ষেপে

Module: পুনঃব্যবহারযোগ্য কনফিগারেশনের প্যাকেজ — একগুচ্ছ `.tf` ফাইল একসাথে। **Workspace:** একই কনফিগারেশন দিয়ে আলাদা State রাখার ব্যবস্থা (যেমন dev/prod)। দুটোর বিস্তারিত এই ভলিউমের পরের অংশে আসবে; এখন শুধু নাম ও ভূমিকাটুকু মনে রাখুন।

৪.১১ সাধারণ ভুল

- **Provider সংস্করণ না বাঁধা:** `required_providers`-এ সংস্করণ না দিলে ভবিষ্যতে ভিন্ন সংস্করণ এসে আচরণ বদলে দিতে পারে।
- **State Git-এ তোলা:** গোপন তথ্য ফাঁস ও দ্বন্দ্বের কারণ।
- **Resource ও Data Source গুলিয়ে ফেলা:** Data Source দিয়ে কিছু বানানো যায় না — তা শুধু পড়ে।

৪.১২ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	করণীয়
`init`-এ Provider পাওয়া যায় না	নেটওয়ার্ক বা ভুল Provider নাম/সংস্করণ	নাম ও সংস্করণ যাচাই করুন; `terraform init` আবার চালান।
Plan-এ অপ্রত্যাশিত মুছে ফেলা দেখায়	কনফিগারেশন থেকে Resource সরানো হয়েছে	Apply-এর আগে Plan মনোযোগ দিয়ে পড়ুন; ভুল হলে কোড ফিরিয়ে আনুন।
State লক হয়ে আছে	আগের Apply থেমে গেছে	চলমান প্রক্রিয়া নেই নিশ্চিত হয়ে `terraform force-unlock` বিবেচনা করুন (সাবধানে)।

৪.১৩ নিরাপত্তা বিবেচনা

Provider-কে ক্লাউডের শংসাপত্র (Credential) দিতে হয় — তা কখনোই `.tf` ফাইলে লিখবেন না; পরিবেশ চলক বা শংসাপত্র ফাইল ব্যবহার করুন। State-এ গোপন তথ্য থাকতে পারে, তাই Backend-এ এনক্রিপশন ও প্রবেশাধিকার নিয়ন্ত্রণ প্রয়োজন।

৪.১৪ প্রোডাকশন প্রকৌশল চর্চা

- সব Provider-এর সংস্করণ বাঁধুন; `.terraform.lock.hcl` Git-এ রাখুন।
- Plan আউটপুট সংরক্ষণ করুন; Apply শুধু অনুমোদিত Plan দিয়ে করুন।
- State-এর দূরবর্তী Backend ও লকিং ব্যবহার করুন (পরের অংশে বিস্তারিত)।

৪.১৫ অধ্যায়ের সারাংশ

Terraform Core কনফিগারেশন পড়ে গ্রাফ বানায় ও পরিকল্পনা করে; Provider প্লাগইন হিসেবে API-র সাথে কথা বলে; Resource Terraform পরিচালনা করে, Data Source শুধু পড়ে; State বাস্তবতার রেকর্ড; আর Plan → Apply-এর বিচ্ছেদই নিরাপদ পরিবর্তনের ভিত্তি।

৪.১৬ জ্ঞান যাচাই

1. Core ও Provider-এর দায়িত্ব আলাদা করে লিখুন।
2. Resource ও Data Source-এর পার্থক্য উদাহরণসহ দিন।
3. State হারালে কী হবে? কেন?
4. `init` কী কী করে?

৪.১৭ ব্যবহারিক অ্যাসাইনমেন্ট

একটি খালি ডিরেক্টরিতে অধ্যায় ৫-এর প্রথম প্রকল্পটি বানানোর আগে, কাগজে এর স্থাপত্য আঁকুন: Core, Provider, State ও আপনার ফাইলগুলো কোথায় থাকবে। পরের অধ্যায়ে বাস্তবে মিলিয়ে দেখুন।

অধ্যায় ৫ • PART I — Infrastructure as Code Foundations

Chapter 5 — Terraform Installation, CLI, Project Structure, and Workflow

৫.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — Ubuntu Server 24.04 LTS-এ Terraform 1.9+ ইনস্টল ও সংস্করণ বাঁধতে; মূল CLI কমান্ডগুলো ব্যবহার করতে; একটি সুশৃঙ্খল প্রকল্প কাঠামো বানাতে; এবং `local` Provider দিয়ে ক্লাউড শংসাপত্র ছাড়াই প্রথম সম্পূর্ণ প্রকল্পটি চালাতে — Plan দেখে, Apply করে, Output পড়ে।

৫.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

এতক্ষণ তত্ত্ব শিখেছেন; এবার হাতেকলমে। সঠিক ইনস্টলেশন, সংস্করণ বাঁধা ও প্রকল্প কাঠামো না থাকলে শেখা Terraform দ্রুতই অগোছালো হয়ে যায় — "আমার মেশিনে চলে, তোমারটায় চলে না" সমস্যাটি ফিরে আসে। এই অধ্যায়ের অভ্যাসগুলোই পরের সব অধ্যায়ের ভিত্তি।

CONCEPT: Terraform CLI একটি একক বাইনারি — `terraform`। এর প্রতিটি উপ-কমান্ড (Subcommand) পাইপলাইনের একটি ধাপ: `init` প্রস্তুত করে, `fmt` সাজায়, `validate` যাচাই করে, `plan` পূর্বরূপ দেখায়, `apply` বাস্তবায়ন করে, `destroy` মুছে ফেলে। ক্রমটি মনে রাখুন: লিখুন → প্রস্তুত → সাজান → যাচাই → পরিকল্পনা → প্রয়োগ।

REAL WORLD: TechNova-তে সবাই একই Terraform সংস্করণ ব্যবহার করে না বলে একদিন একজনের Plan আরেকজনের সাথে মিলছিল না। সমাধান ছিল সংস্করণ বাঁধা: `required_version` ও লক ফাইল — এরপর থেকে সবার ফল একরকম।

৫.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ৪-এর স্থাপত্য, লিনাক্সে প্যাকেজ ইনস্টল করা ও টার্মিনালে কাজ করার অভিজ্ঞতা থাকলেই চলবে। ক্লাউড অ্যাকাউন্ট বা শংসাপত্র এই অধ্যায়ে লাগবে না — সবকিছু লোকাল।

৫.৪ ইনস্টলেশন: Ubuntu Server 24.04 LTS

HashiCorp-এর অফিসিয়াল APT রিপোজিটরি থেকে ইনস্টল করাই নির্ভরযোগ্য পথ — এতে আপডেটও সহজে আসে। নিচের কমান্ডগুলো ক্রমানুসারে চালান।

```
sudo apt-get update && sudo apt-get install -y gnupg software-properties-common curl
curl -fsSL https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/
hashicorp-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://
apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/
hashicorp.list
```

```
sudo apt-get update && sudo apt-get install -y terraform
terraform version
```

প্রত্যাশিত আউটপুট (সংস্করণ সংখ্যা ভিন্ন হতে পারে, তবে 1.9.x হতে হবে):

```
Terraform v1.9.8
on linux_amd64
```

সংস্করণ যাচাই করা কেন জরুরি? Terraform-এর নতুন প্রধান সংস্করণে আচরণ বদলাতে পারে। তাই প্রকল্পে সংস্করণ বাঁধুন — অধ্যায় ৪-এর মতো: `required_version = ">= 1.9, < 2.0"`। এর মানে: 1.9 বা তার নতুন ছোট সংস্করণ চলবে, কিন্তু 2.0 চলবে না। এভাবে দলের সবাই একই আচরণ পায়।

> **WARNING:** অজানা স্ক্রিপ্ট বা তৃতীয় পক্ষের ইনস্টলার দিয়ে Terraform ইনস্টল করবেন না। শুধু HashiCorp-এর অফিসিয়াল রিপোজিটরি বা অফিসিয়াল বাইনারি ব্যবহার করুন, আর ডাউনলোডের চেকসাম যাচাই করুন।

৫.৫ CLI পরিচিতি

কমান্ড	কাজ	কখন ব্যবহার করবেন
`terraform init`	Provider ডাউনলোড, Backend/Module প্রস্তুত	নতুন প্রকল্পে, Provider বদলালে
`terraform fmt`	HCL সাজানো (ইনডেন্ট, স্পেস)	প্রতিটি পরিবর্তনের পর
`terraform validate`	সিনট্যাক্স ও ধরন যাচাই	Apply-এর আগে, CI-তে
`terraform plan`	পরিবর্তনের পূর্বরূপ	Apply-এর আগে, সবসময়
`terraform apply`	পরিকল্পনা বাস্তবায়ন	Plan দেখে অনুমোদনের পর
`terraform destroy`	সব Resource মুছে ফেলা	পরিবেশ বন্ধ করার সময়
`terraform show`	State বা Plan পড়া	বাস্তব অবস্থা দেখতে
`terraform state`	State তালিকা/পরীক্ষা	সমস্যা নির্ণয়ে (সাবধানে)
`terraform output`	Output মান দেখা	Apply-এর পর ফল জানতে
`terraform import`	বিদ্যমান সম্পদ State-এ আনা	হাতে-বানানো সম্পদ কোডে আনতে

সাহায্যের জন্য যেকোনো কমান্ডে `-help` যোগ করুন, যেমন `terraform plan -help`।

৫.৬ প্রকল্প কাঠামো

```
technova-first/
├── terraform.tf      # Terraform ও Provider সংস্করণ বাঁধা
├── providers.tf      # Provider কনফিগারেশন
├── variables.tf      # ইনপুট চলক (Variable)
├── main.tf           # মূল Resource
```

```
├── outputs.tf        # আউটপুট মান (Output)
└── README.md         # প্রকল্পের উদ্দেশ্য ও চালানোর নিয়ম
```

প্রতিটি ফাইলের ভূমিকা: `terraform.tf`-এ `terraform { required_version ... required_providers ... }` ব্লক থাকে; `providers.tf`-এ Provider-এর সেটিং; `variables.tf`-এ `variable` ব্লক — নাম, ধরন (`type`), বর্ণনা (`description`) ও ডিফল্ট মান (`default`); `main.tf`-এ `resource` ব্লক; `outputs.tf`-এ `output` ব্লক — Apply-এর পর কী মান দেখতে চান। ফাইল আলাদা রাখলে খুঁজে পাওয়া ও পর্যালোচনা সহজ হয়। `.terraform/` (ডাউনলোড করা Provider) ও `terraform.tfstate` Git-এ যাবে না — `.gitignore`-এ রাখুন। `.terraform.lock.hcl` Git-এ যাবে — এতে Provider সংস্করণ ও হ্যাশ থাকে।

৫.৭ প্রথম প্রকল্প: TechNova লোকাল ফাইল (ধাপে ধাপে)

এই প্রকল্পে কোনো ক্লাউড খরচ নেই — শুধু লোকাল ফাইল তৈরি হবে। ডোমেইন `technova.example` ও IP `203.0.113.10` শুধু উদাহরণ হিসেবে ফাইলের ভেতরে লেখা থাকবে।

৫.৭.১ ধাপ ১ — ডিরেক্টরি ও ফাইল তৈরি

```
mkdir -p ~/technova-first && cd ~/technova-first
```

৫.৭.২ ধাপ ২ — terraform.tf: সংস্করণ বাঁধা

```
terraform {
  required_version = ">= 1.9, < 2.0"
  required_providers {
    local = {
      source  = "hashicorp/local" # Provider-এর উৎস (Registry ঠিকানা)
      version = ">= 2.5, < 3.0"  # সংস্করণের সীমা — কেন? ভবিষ্যতের
                                  # বড় পরিবর্তন যেন নীরবে না আসে
    }
    null = {
      source  = "hashicorp/null"
      version = ">= 3.2, < 4.0"
    }
  }
}
```

৫.৭.৩ ধাপ ৩ — providers.tf, variables.tf, main.tf, outputs.tf

```
# providers.tf — local Provider-এর কোনো বিশেষ সেটিং লাগে না
provider "local" {}

# variables.tf
variable "app_name" {
  description = "অ্যাপ্লিকেশনের নাম — ফাইলের বিষয়বস্তুতে ব্যবহৃত হবে"
  type        = string
  default     = "technova-web"
}
variable "server_ip" {
  description = "উদাহরণ IP (কাল্পনিক TEST-NET পরিসর)"
  type        = string
  default     = "203.0.113.10"
```

```
}

# main.tf
resource "local_file" "welcome" {
  filename = "${path.module}/welcome.txt"
  # path.module মানে এই কনফিগারেশন যে ডিরেক্টরিতে আছে, তার পথ।
  content = "Welcome to ${var.app_name} at technova.example (${var.server_ip})\n"
  # ${...} হলো Interpolation — চলকের মান এখানে বসে।
}
resource "null_resource" "announce" {
  # null_resource কিছু বানায় না; শুধু একটি ধাপ/চিহ্ন হিসেবে কাজ করে।
  # triggers বদলালে এটি আবার চলবে — এখানে app_name-এর উপর নির্ভরশীল।
  triggers = { app = var.app_name }
}

# outputs.tf
output "welcome_file" {
  description = "তৈরি হওয়া ফাইলের পথ"
  value       = local_file.welcome.filename
}
output "welcome_content" {
  description = "ফাইলের বিষয়বস্তু"
  value       = local_file.welcome.content
}
```

৫.৭.৪ ধাপ ৪ — প্রমিত কর্মপ্রবাহ চালানো

```
terraform init
terraform fmt
terraform validate
terraform plan
terraform apply
```

`terraform init`-এর প্রত্যাশিত আউটপুট (সংক্ষেপিত):

```
Initializing the backend...
Initializing provider plugins...
- Installing hashicorp/local v2.5.2...
- Installed hashicorp/local v2.5.2
- Installing hashicorp/null v3.2.2...
Terraform has been successfully initialized!
```

`terraform validate`-এর প্রত্যাশিত আউটপুট:

```
Success! The configuration is valid.
```

`terraform plan`-এর প্রত্যাশিত আউটপুট (সংক্ষেপিত):

```
Terraform will perform the following actions:
  # local_file.welcome will be created
  + resource "local_file" "welcome" {
      + content  = "Welcome to technova-web at technova.example (203.0.113.10)\n"
      + filename = "./welcome.txt"
    }
  # null_resource.announce will be created
```

```
  + resource "null_resource" "announce" {
      + id       = (known after apply)
      + triggers = { "app" = "technova-web" }
    }
Plan: 2 to add, 0 to change, 0 to destroy.
```

ব্যাখ্যা: `+` মানে তৈরি হবে; `(known after apply)` মানে মানটি Apply-এর পর জানা যাবে; শেষ লাইনটি সংখ্যায় সারাংশ দেয়। `apply` চালালে অনুমোদন চাওয়া হবে — `yes` লিখুন। তারপর `terraform output` ও `cat welcome.txt` দিয়ে ফল মিলিয়ে দেখুন। দ্বিতীয়বার `terraform plan` চালালে দেখবেন: `No changes. Your infrastructure matches the configuration.` — এটাই Idempotence।

৫.৭.৫ ধাপ ৫ — পরিষ্কার করা

```
terraform destroy
# অনুমোদন দিলে welcome.txt মুছে যাবে; Plan-এ "2 to destroy" দেখাবে।
```

TROUBLESHOOTING: `init`-এ নেটওয়ার্ক ত্রুটি হলে সংযোগ ও প্রক্সি যাচাই করুন। `validate`-এ ত্রুটি হলে প্রথমে বন্ধনী `{ }` ও উদ্ধৃতি চিহ্ন মেলান। Plan-এ অপ্রত্যাশিত পরিবর্তন দেখালে `terraform show` দিয়ে State দেখুন — তবে State হাতে সম্পাদনা করবেন না।

৫.৮ সাধারণ ভুল

- **`init` না করে `plan` চালানো:** Provider না থাকলে Plan চলবে না।
- **সংস্করণ না বাঁধা:** দলের একেকজনের ফল একেকরকম হবে।
- **State Git-এ তোলা:** `.gitignore`-এ `.terraform/` ও `*.tfstate*` রাখুন।
- **Plan না দেখে Apply:** বিশেষ করে `destroy` দেখায় এমন Plan-এ।
- **গোপন তথ্য চলকে লেখা:** এই অধ্যায়ে কোনো গোপন তথ্য নেই; ভবিষ্যতেও প্লেসহোল্ডার `CHANGE-ME-...` ব্যবহার করুন।

৫.৯ সমস্যা সমাধান নির্দেশিকা

লক্ষণ	সম্ভাব্য কারণ	করণীয়
`terraform: command not found`	ইনস্টল হয়নি বা PATH-এ নেই	ইনস্টলেশন ধাপ আবার করুন; `terraform version` যাচাই করুন।
`validate`-এ সিনট্যাক্স ত্রুটি	বন্ধনী/উদ্ধৃতি ভুল	`terraform fmt` চালান; ত্রুটির লাইন নম্বর দেখুন।
Plan-এ "No changes" আসে না দ্বিতীয়বার	বাইরে ফাইল বদলানো হয়েছে	ফাইলটি হাতে বদলাবেন না; কোডে বদলান, তারপর Apply করুন।

৫.১০ নিরাপত্তা বিবেচনা

এই অধ্যায়ে কোনো ক্লাউড শংসাপত্র নেই, তাই খরচ বা ফাঁসের ঝুঁকি নেই। তবে অভ্যাসটি এখনই গড়ুন: গোপন তথ্য কখনোই `.tf` ফাইলে নয়; `.tfstate` ফাইলে কী আছে, তা না জেনে শেয়ার করবেন না।

৫.১১ প্রোডাকশন প্রকৌশল চর্চা

- প্রতিটি প্রকল্পে `terraform.tf`-এ সংস্করণ বাঁধুন; লক ফাইল Git-এ রাখুন।
- কর্মপ্রবাহটি CI-তে বাঁধুন: `fmt -check`, `validate`, `plan` — প্রতিটি Pull Request-এ।
- README-তে চালানোর ধাপ ও প্রয়োজনীয় সংস্করণ লিখুন।

৫.১২ অধ্যায়ের সারাংশ

Terraform ইনস্টল করে সংস্করণ যাচাই করা, CLI-র মূল কমান্ডগুলো জানা, ফাইলভিত্তিক সুশৃঙ্খল কাঠামো ব্যবহার করা, এবং লিখুন → `init` → `fmt` → `validate` → `plan` → `apply` কর্মপ্রবাহ অনুসরণ করা — এই হলো Terraform কাজের ভিত্তি। Local Provider দিয়ে প্রথম প্রকল্পটি প্রমাণ করে: ক্লাউড ছাড়াই, খরচ ছাড়াই, Terraform-এর পুরো চক্রটি শেখা যায়।

৫.১৩ জ্ঞান যাচাই

1. `init`, `validate`, `plan`, `apply`-এর কাজ আলাদা করে লিখুন।
2. কেন `.terraform.lock.hcl` Git-এ রাখা হয়, কিন্তু `.terraform/` রাখা হয় না?
3. `required_version`-এর সীমা `>= 1.9, < 2.0`-এর অর্থ কী?
4. দ্বিতীয়বার Plan-এ “No changes” কেন আসে?

৫.১৪ ব্যবহারিক অ্যাসাইনমেন্ট

এই অধ্যায়ের প্রকল্পটি নিজে বানান, Apply করুন, Output মিলিয়ে দেখুন, তারপর একটি চলক বদলে (`app_name`-এর মান) আবার Plan ও Apply করুন — Plan-এ কী বদল দেখায়, তা লিখুন। শেষে `terraform destroy` দিয়ে পরিষ্কার করুন। আপনার পর্যবেক্ষণগুলো README-তে যোগ করুন।

PART II • DEVOPS ENGINEERING MASTERY

Terraform Core Engineering

HCL, Variable, Output, Local, Resource, Data Source, Dependency Graph, Terraform State, Remote Backend, Locking, Plan, Apply, Destroy ↻ Lifecycle Rules

অধ্যায় ৬ • PART II — Terraform Core Engineering

Chapter 6 — HCL Fundamentals — Blocks, Arguments, Expressions, and Comments

৬.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — HCL (HashiCorp Configuration Language) এর গঠন প্রথম নীতি থেকে ব্যাখ্যা করতে; Block, Argument, Expression, Label — এই চারটি মৌলিক উপাদান চিনতে ও লিখতে; String, Number, Bool, List, Map, Object, Tuple-এর মতো Type ব্যবহার করতে; Heredoc, Comment, Operator ও Function-এর প্রাথমিক ব্যবহার করতে; `terraform fmt` ও `terraform validate` দিয়ে কোড পরিষ্কার ও যাচাই করতে; এবং HCL-এর সাধারণ Syntax Error পড়ে নিজে ঠিক করতে।

৬.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

Terraform দিয়ে Infrastructure as Code লেখা মানে আসলে HCL লেখা। আপনি যত বড় Architecture-ই বানান না কেন — VPC, Kubernetes Cluster, Database, Load Balancer — সবকিছুর ভিত্তি এই ভাষা। HCL না বুঝে Terraform শেখা মানে ব্যাকরণ না শিখে প্রবন্ধ লেখার চেষ্টা করা। TechNova-তে একজন Engineer যখন `main.tf` ফাইলে একটি Server-এর সংজ্ঞা লেখেন, তখন তিনি আসলে একটি ঘোষণা (Declaration) দিচ্ছেন — "আমি এই জিনিসটি চাই, এই অবস্থায় চাই।" Terraform সেই ঘোষণা পড়ে, বুঝে, তারপর বাস্তব Infrastructure-এর সঙ্গে মিলিয়ে কাজ করে। এই অধ্যায় সেই ভাষার ব্যাকরণ।

> **REAL WORLD:** TechNova-র Staging Environment-এ একবার একজন নতুন Engineer `instance_type = t3.micro` লিখেছিলেন — উদ্ধৃতি চিহ্ন (Quotation Mark) ছাড়া। Terraform সঙ্গে সঙ্গে Error দিল। কারণ HCL-এ উদ্ধৃতি ছাড়া `t3.micro` একটি String নয়, এটি একটি Reference হিসেবে ধরা হয়। ছোট একটি চিহ্নের ভুল, কিন্তু পুরো Apply থেমে যায়। ভাষা ঠিকমতো জানা থাকলে এই ভুল প্রথম দিনেই ধরা পড়ে।

৬.৩ পূর্বশর্ত জ্ঞান

এই অধ্যায়ের জন্য Part I-এর ধারণা — Infrastructure as Code কী, Declarative বনাম Imperative পদ্ধতি, এবং Terraform-এর মৌলিক Workflow (`init`, `plan`, `apply`) — জানা থাকা ভালো। কোনো প্রোগ্রামিং ভাষার অভিজ্ঞতা থাকলে সুবিধা হবে, তবে আবশ্যক নয়। Linux Terminal-এ ফাইল তৈরি ও সম্পাদনা করতে পারলেই চলবে।

৬.৪ HCL কী এবং কেন এটি তৈরি হলো

HCL অর্থ **HashiCorp Configuration Language** — HashiCorp কোম্পানির তৈরি একটি Configuration ভাষা। এটি কোনো সাধারণ প্রোগ্রামিং ভাষা নয়; এটি একটি **Declarative** ভাষা। অর্থাৎ আপনি এতে "কীভাবে করব" লেখেন না, লেখেন "কী চাই"। যেমন — "আমার একটি Server চাই, যার Type `t3.micro`, Region `ap-south-1`।" কীভাবে Server তৈরি হবে, কোন API Call যাবে, কোন ক্রমে যাবে — সেসব Terraform নিজে ঠিক করে।

HCL কেন JSON বা YAML নয়? JSON-এ Comment লেখা যায় না, মানুষের পড়তে কষ্ট হয়, আর Trailing Comma-তে Error হয়। YAML মানুষের জন্য সহজ, কিন্তু Indentation-এর ছোট ভুলে অর্থ বদলে যায়। HCL এই দুইয়ের মাঝখানে —

মানুষের পড়ার মতো সহজ, আবার মেশিনের পার্স করার মতো কঠোর ও নিখুঁত। Terraform 0.12 সংস্করণ থেকে HCL2 ব্যবহৃত হয়, যেখানে Expression অনেক শক্তিশালী — Function, Operator, For Expression, Conditional — সবকিছু লেখা যায়।

> **CONCEPT:** HCL একটি **Domain-Specific Language (DSL)** — অর্থাৎ এটি শুধু Infrastructure বর্ণনার জন্য তৈরি বিশেষ ভাষা। সাধারণ ভাষায় আপনি যেকোনো প্রোগ্রাম লিখতে পারেন; HCL-এ আপনি শুধু Infrastructure ঘোষণা করতে পারেন। এই সীমাবদ্ধতাই এর শক্তি — এটি ভুল করার সুযোগ কমায়।

৬.৫ HCL-এর মৌলিক গঠন: Block, Label, Argument

প্রতিটি HCL ফাইল কয়েকটি **Block** দিয়ে তৈরি। একটি Block হলো একটি নামযুক্ত অংশ, যার ভেতরে সেটিংস থাকে। গঠনটি এরকম:

```
block_type "label1" "label2" {
  argument_name = expression
  argument_name = expression
}
```

এখানে `block_type` হলো Block-এর ধরন — যেমন `resource`, `variable`, `output`, `terraform`। `label1`, `label2` হলো **Label** — Block-টিকে চিহ্নিত করার নাম। যেমন `resource "aws_instance" "web"`-এ প্রথম Label হলো Resource-এর ধরন (`aws_instance`), দ্বিতীয় Label হলো আপনার দেওয়া নাম (`web`)। আর `argument_name = expression` হলো **Argument** — Block-এর ভেতরে একটি সেটিং। সমান চিহ্নের (`=`) বামে নাম, ডানে মান — এই মানটিই **Expression**।

৬.৫.১ একটি সম্পূর্ণ উদাহরণ

```
terraform {
  required_version = ">= 1.9, < 2.0"

  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.5"
    }
  }
}

resource "local_file" "welcome" {
  filename = "${path.module}/welcome.txt"
  content  = "Hello from TechNova - technova.example"
}
```

এই উদাহরণে দুটি Block আছে — `terraform` Block ও `resource` Block। `terraform` Block-এর ভেতরে আবার একটি Nested Block আছে — `required_providers`। Nested Block মানে Block-এর ভেতরে Block। `filename` ও `content` হলো Argument। `"${path.module}/welcome.txt"` একটি Expression — এখানে `path.module` একটি Built-in Reference, আর `${ ... }` হলো Interpolation — String-এর ভেতরে মান বসানো।

৬.৬ Expression ও ডেটা টাইপ

Expression হলো এমন যেকোনো লেখা, যা মূল্যায়ন (Evaluate) করলে একটি মান দেয়। HCL-এ মৌলিক Type কয়েকটি:

Type	উদাহরণ	ব্যাখ্যা
String	`"technova.example"`	উদ্ধৃতি চিহ্নের ভেতরে লেখা
Number	`8080`, `3.14`	পূর্ণসংখ্যা বা দশমিক; উদ্ধৃতি লাগে না
Bool	`true`, `false`	সত্য বা মিথ্যা; উদ্ধৃতি লাগে না
List	`["a", "b"]`	ক্রমযুক্ত মানের তালিকা
Map	`{ name = "web", port = 80 }`	Key-Value জোড়া
Object	`{ name = string, port = number }`	নির্দিষ্ট Type-সহ Map-এর মতো গঠন
Tuple	`["web", 80, true]`	ভিন্ন Type-এর মানের ক্রম
Null	`null`	কোনো মান নেই — Argument বাদ দেওয়ার মতো

৬.৬.১ String ও Heredoc

সাধারণ String লেখা হয় ডাবল কোটে: `"Hello TechNova"`। বড়, বহু লাইনের লেখার জন্য **Heredoc** ব্যবহার করুন — `<<-EOT ... EOT`। যেমন User Data বা Policy লেখার সময়:

```
locals {
  welcome_message = <<-EOT
    Welcome to TechNova!
    Domain: technova.example
    Environment: staging
  EOT
}
```

`<<-`-এর ড্যাশ (-) মানে শেষের `EOT`-এর আগের Indentation উপেক্ষা করা হবে। Heredoc-এর ভেতরেও `${ ... }` Interpolation কাজ করে।

৬.৬.২ List, Map, Object — কখন কোনটি

```
locals {
  ports       = [80, 443, 8080]
  tags        = { Team = "platform", Env = "staging" }
  server_spec = { name = "web-01", cpu = 2, memory_gb = 4 }
}
```

List ব্যবহার করুন যখন ক্রম গুরুত্বপূর্ণ (Port-এর তালিকা)। Map ব্যবহার করুন যখন নাম দিয়ে খুঁজতে হবে (Tag)। Object ব্যবহার করুন যখন গঠন নির্দিষ্ট (একটি Server-এর Spec)। Index দিয়ে List-এর মান পাওয়া যায় — `local.ports[0]` দেবে `80`। Map-এর মান পাওয়া যায় Key দিয়ে — `local.tags["Team"]`।

৬.৭ Operator ও Function-এর প্রাথমিক ধারণা

HCL-এ গাণিতিক ও যৌক্তিক Operator আছে: `+ - * / %`, তুলনা `== != < > <= >=`, যুক্তি `&& || !`। Conditional Expression লেখা যায়: `condition ? value_if_true : value_if_false`। যেমন:

```
locals {
  instance_count = var.environment == "production" ? 3 : 1
  total_ports    = length(local.ports)
  upper_domain   = upper("technova.example")
}
```

এখানে `length()` ও `upper()` হলো Built-in **Function**। Terraform-এ শতাধিক Function আছে — String, Number, Collection, Encoding, Filesystem, Date-Time, Crypto ইত্যাদি। Function-এর পুরো তালিকা পরের অধ্যায়গুলোতে প্রয়োজনে ব্যবহার করা হবে। মনে রাখুন: Function Call-এর ভেতরে Argument Comma দিয়ে আলাদা করা হয়, আর Function সবসময় একটি মান ফেরত দেয়।

৬.৮ Comment ও Formatting

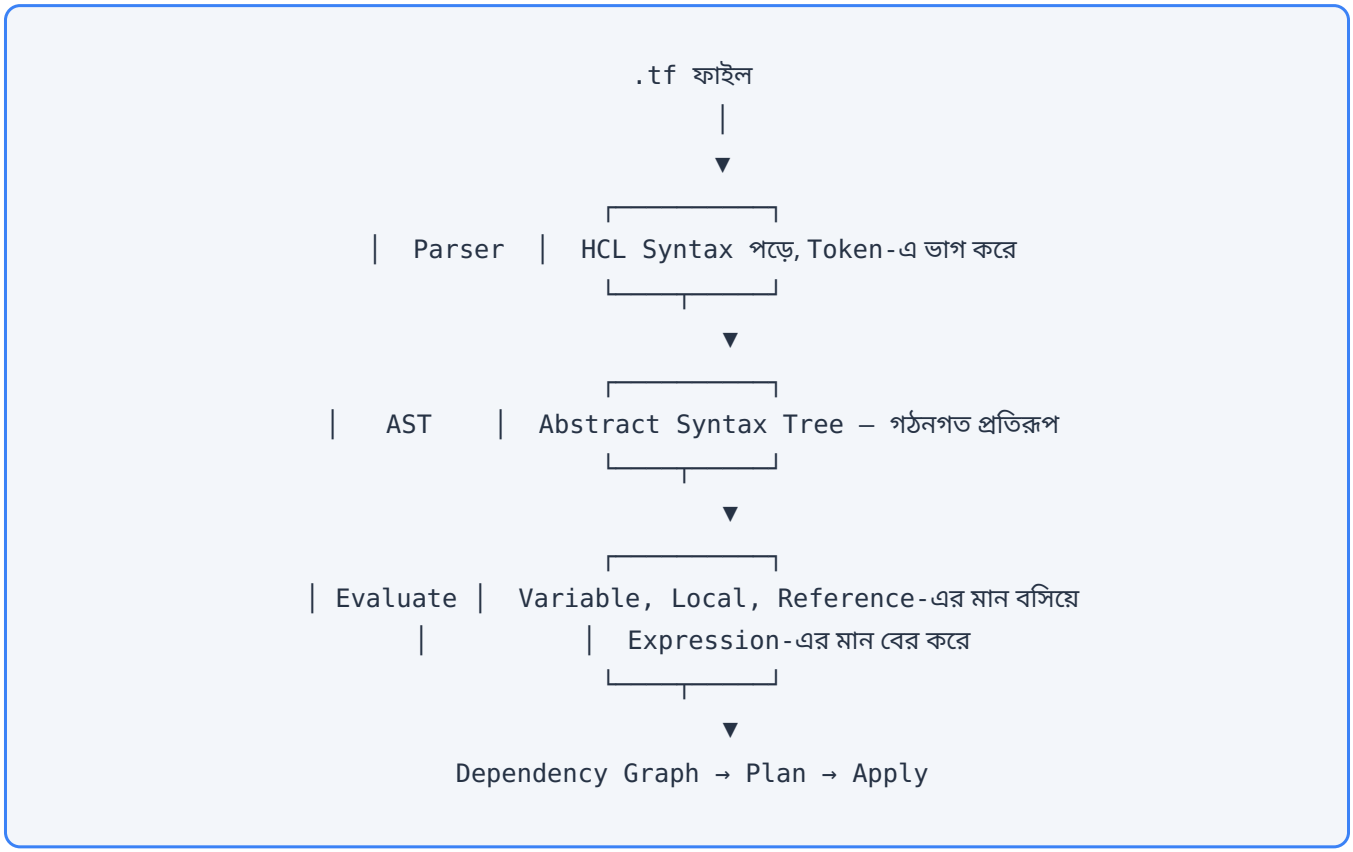
Comment তিনভাবে লেখা যায়: `# এক লাইনের মন্তব্য` `// এক লাইনের মন্তব্য` এবং `/* বহু লাইনের মন্তব্য */`। Terraform সম্প্রদায়ে `#` সবচেয়ে প্রচলিত। ভালো Comment লেখে "কেন" — "কী" নয়। যেমন `# Production-এ 3টি Replica, কারণ Traffic বেশি` ভালো; `# instance_count = 3` অর্থহীন।

Formatting-এর জন্য `terraform fmt` চালান — এটি Indentation, Spacing, সমান চিহ্নের Alignment সব ঠিক করে দেয়। `terraform fmt -check -diff` দিয়ে দেখা যায় কী বদলাবে, না বদলে। `terraform validate` Syntax ও Type যাচাই করে — কিন্তু Provider-এর সঙ্গে যোগাযোগ করে না। দুটি Command-ই Commit-এর আগে চালানো Production অভ্যাস।

```
$ terraform fmt
main.tf
$ terraform validate
Success! The configuration is valid.
```

৬.৯ HCL কীভাবে Parse ও Evaluate হয়

ভেতরে কী ঘটে, তা বুঝলে Error পড়া সহজ হয়। ধাপগুলো:



Parser প্রথমে Syntax পরীক্ষা করে — বন্ধনী (Brace) মিলেছে কি না, সমান চিহ্ন আছে কি না। তারপর Evaluator Variable ও Reference-এর মান বসায়। এজন্য Syntax Error হলে Terraform শুরুতেই থামে, আর Reference Error (যেমন ভুল Resource-এর নাম) হলে Evaluate ধাপে থামে।

৬.১০ সাধারণ ভুল ও Troubleshooting

ভুল	লক্ষণ / Error	সমাধান
String-এ কোট ভুলে যাওয়া	`Reference to undeclared resource`	String সবসময় `"..."` -এ লিখুন
বন্ধনী না মেলানো	`Unclosed configuration block`	Editor-এর Bracket Matching ব্যবহার করুন
সমান চিহ্নের বদলে কোলন (:)	`Argument definition must end with a newline`	HCL-এ `=` ব্যবহার করুন, JSON-এর `:` নয়
List-এ Trailing Comma	সাধারণত বৈধ, তবে পুরোনো সংস্করণে সমস্যা	Terraform 1.9+-এ বৈধ; তবু পরিষ্কার রাখুন
Interpolation-এ ভুল Syntax	`Invalid template interpolation value`	`${ ... }` -এর ভেতরে বৈধ Expression লিখুন

TROUBLESHOOTING: Error Message পড়ার নিয়ম — প্রথমে ফাইলের নাম ও লাইন নম্বর দেখুন, তারপর Error-এর ধরন। Terraform সাধারণত ভুলের ঠিক জায়গাটি দেখিয়ে দেয়। যেমন `Error: Missing argument separator` মানে কোনো লাইনে `=` বা Comma বাদ পড়েছে। পুরো Message না পড়ে শুধু প্রথম লাইন দেখে আন্দাজ করবেন না।

WARNING: কখনোই `.tf` ফাইলে আসল Password, API Key বা Secret লিখবেন না। এগুলো Variable-এর মাধ্যমে Environment বা Secret Manager থেকে আসবে — বিস্তারিত অধ্যায় ৭ ও ১০-এ।

৬.১১ Production Engineering Practices

প্রতিটি `.tf` ফাইলে উদ্দেশ্য অনুযায়ী ভাগ করুন — `main.tf`, `variables.tf`, `outputs.tf`, `providers.tf`। Version Pin করুন (`required_version` ও Provider Version)। সব ফাইলে `terraform fmt` চালিয়ে Commit করুন — অনেক দল CI-তে `fmt -check` বাধ্যতামূলক করে। অর্থবহ নাম দিন — `web` ভালো, `x1` খারাপ।

৬.১২ অধ্যায়ের সারসংক্ষেপ

HCL একটি Declarative, মানুষের পড়ার মতো Configuration ভাষা। এর মূল উপাদান Block, Label, Argument ও Expression। String, Number, Bool, List, Map, Object — এই Type-গুলো দিয়ে যেকোনো Infrastructure বর্ণনা করা যায়। Heredoc বড় লেখার জন্য, Comment ব্যাখ্যার জন্য, Function ও Operator হিসাবের জন্য। Parser প্রথমে Syntax, তারপর Evaluator মান যাচাই করে — তাই Error Message-এর ধাপ বুঝলে সমস্যা দ্রুত ধরা যায়।

৬.১৩ Knowledge Check

১. Block, Label ও Argument-এর পার্থক্য কী? ২. `resource "local_file" "welcome"`-এ Label দুটি কী কী? ৩. Heredoc কখন ব্যবহার করবেন? ৪. `terraform fmt` ও `terraform validate`-এর পার্থক্য কী? ৫. String-এ কোট না দিলে Terraform কী ধরে নেয়?

৬.১৪ Practical Assignment

TechNova-র জন্য একটি `main.tf` লিখুন: `local_file` Resource দিয়ে `/tmp/technova-welcome.txt`-এ একটি স্বাগত বার্তা তৈরি করুন, যাতে Domain, Environment ও Port-এর তালিকা থাকে। একটি `locals` Block-এ Heredoc ব্যবহার করুন। `terraform fmt` ও `validate` চালিয়ে সফল Output-এর স্ক্রিনশট/লগ সংরক্ষণ করুন। ইচ্ছাকৃতভাবে একটি কোট বাদ দিয়ে Error Message-টি পড়ে তার অর্থ লিখুন, তারপর ঠিক করুন।

অধ্যায় ৭ • PART II — Terraform Core Engineering

Chapter 7 — Variables, Outputs, Locals, and Type Constraints

৭.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — Input Variable ঘোষণা করতে (Type, Default, Description, Validation, Sensitive-সহ); Output Value লিখতে; Local Value দিয়ে হিসাব পুনর্ব্যবহার করতে; Type Constraint — String, Number, Bool, List, Map, Set, Object, Tuple, Any — সঠিকভাবে ব্যবহার করতে; Variable Precedence-এর ক্রম ব্যাখ্যা করতে (`tfvars`, `-var`, `TF_VAR_`, Default); এবং কখন Variable, কখন Local, কখন Output ব্যবহার করবেন তা সিদ্ধান্ত নিতে।

৭.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

একই Configuration দিয়ে Staging ও Production — দুটি Environment চালাতে হলে কোডে Hardcode করা চলবে না। Environment-এর নাম, Server-এর সংখ্যা, Port — এসব বদলায়। Variable সেই বদলানোর জায়গা। Variable ছাড়া প্রতিটি Environment-এর জন্য আলাদা কোড লিখতে হতো — ভুল আর Drift-এর জন্ম হতো সেখানেই। TechNova-তে একটিই Module, আর Variable-এর মান বদলে Staging (১টি Server) আর Production (৩টি Server) — এটাই Infrastructure as Code-এর পুনর্ব্যবহারযোগ্যতা।

> **REAL WORLD:** TechNova-র Production-এ একবার ভুল করে Staging-এর `terraform.tfvars` দিয়ে Apply দেওয়া হয়েছিল — Instance Count ১ হয়ে গিয়েছিল। কারণ দুটি Environment-এর Variable ফাইল আলাদা ছিল, কিন্তু Apply-এর সময় কোন ফাইল ব্যবহার হচ্ছে তা যাচাই করা হয়নি। শিক্ষা: Variable ফাইলের নাম ও Environment স্পষ্ট করুন, আর Apply-এর আগে `terraform plan` -এ মানগুলো দেখে নিন।

৭.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ৬-এর HCL — Block, Argument, Expression, Type — জানা থাকতে হবে। Variable-এর ধারণা যেকোনো প্রোগ্রামিং ভাষা থেকে পরিচিত হলে ভালো, তবে এখানে Terraform-এর নিজস্ব নিয়মগুলোই মূল।

৭.৪ Input Variable: প্রথম নীতি

Input Variable হলো Configuration-এর বাইরে থেকে মান দেওয়ার দরজা। এটি ঘোষণা করা হয় `variable` Block-এ, সাধারণত `variables.tf` ফাইলে। ঘোষণার সময় আপনি বলেন — নাম কী, Type কী, Default কী, বর্ণনা কী। মান দেওয়া হয় Apply-এর সময়।

```
variable "environment" {
  type        = string
  description = "Deployment environment: staging or production"
  default     = "staging"

  validation {
    condition     = contains(["staging", "production"], var.environment)
```

```
    error_message = "Environment must be staging or production."
  }
}

variable "instance_count" {
  type        = number
  description = "Number of application servers"
  default     = 1
}

variable "db_password" {
  type        = string
  description = "Database password - provide via TF_VAR_db_password, never commit"
  sensitive   = true
}
```

এখানে প্রতিটি Argument-এর কাজ: `type` — মানের ধরন; `default` — মান না দিলে যা হবে (না থাকলে Variable আবশ্যক — মান দিতেই হবে); `description` — ডকুমেন্টেশন; `validation` — ভুল মান আটকানো; `sensitive` — মান Log ও Output-এ লুকানো। Variable ব্যবহার করা হয় `var.environment` লিখে — `var.` Prefix-টি Terraform-কে বলে, এটি একটি Variable Reference।

> **CONCEPT:** `validation` Block হলো প্রতিরক্ষার প্রথম স্তর। ভুল Environment নাম বা ঋণাত্মক Instance Count Apply-এর আগেই আটকে যায়। Production-এ ভুল মান যাওয়া ঠেকাতে Validation-এর কোনো বিকল্প নেই — এটি "Fail Fast" নীতির প্রয়োগ।

৭.৫ Type Constraint — বিস্তারিত

Type	উদাহরণ	কখন ব্যবহার করবেন
`string`	`"staging"`	নাম, Region, Domain
`number`	`3`	Count, Port, Size
`bool`	`true`	Feature চালু/বন্ধ
`list(string)`	`["80", "443"]`	ক্রমযুক্ত তালিকা
`map(string)`	`{ Env = "prod" }`	Tag, Key-Value
`set(string)`	`["a", "b"]`	অনন্য মানের সংগ্রহ, ক্রম অপ্রাসঙ্গিক
`object({...})`	`object({ name = string, cpu = number })`	নির্দিষ্ট গঠনের জটিল মান
`tuple([...])`	`tuple([string, number])`	ভিন্ন Type-এর নির্দিষ্ট ক্রম
`any`	যেকোনো মান	শুধু যখন Type আগে থেকে জানা যায় না — এড়িয়ে চলুন

List বনাম Set: List-এ একই মান দুবার থাকতে পারে, ক্রম থাকে; Set-এ প্রতিটি মান একবারই থাকে, ক্রমের নিশ্চয়তা নেই। Terraform-এ `for_each` (অধ্যায় ১২-এ বিস্তারিত) সাধারণত Set বা Map চায়। Object-এর উদাহরণ:

```
variable "server" {
  type = object({
    name      = string
    cpu       = number
    memory_gb = optional(number, 4)
  })
  default = { name = "web-01", cpu = 2 }
}
```

`optional()` মানে Field-টি না দিলেও চলবে, Default মান বসবে। Type ভুল হলে Terraform Plan-এর আগেই Error দেয় — এটাই Type Constraint-এর মূল্য: ভুল মান Infrastructure-এ পৌঁছায় না।

৭.৬ Output Value

Output হলো Configuration-এর ভেতর থেকে বাইরে মান বের করার দরজা। Apply-এর পর আপনি জানতে চান — Server-এর IP কী, File-এর Path কী? Output তা দেখায়। এটি লেখা হয় `outputs.tf`-এ:

```
output "welcome_file_path" {
  description = "Path of the generated welcome file"
  value       = local_file.welcome.filename
}

output "server_names" {
  description = "Names of all application servers"
  value       = [for s in local_file.app : s.filename]
  sensitive   = false
}
```

`terraform output` Command দিয়ে মান দেখা যায়, `terraform output -json` দিয়ে Script-এ ব্যবহার করা যায়। `sensitive = true` দিলে মান Terminal-এ দেখাবে না — তবে মনে রাখুন, State ফাইলে মানটি Plain Text-এই থাকে (বিস্তারিত অধ্যায় ৯)।

৭.৭ Local Value

Local হলো Configuration-এর ভেতরে হিসাব করে রাখা নামযুক্ত মান — একবার হিসাব, বারবার ব্যবহার। এটি `locals` Block-এ লেখা হয়, ব্যবহার করা হয় `local.` Prefix দিয়ে:

```
locals {
  common_tags = {
    Team        = "platform"
    Environment = var.environment
    Domain      = "technova.example"
  }
  app_name = "technova-${var.environment}"
  is_prod  = var.environment == "production"
}
```

Local-এর মান অন্য Local, Variable বা Resource-এর Attribute থেকে হিসাব করা যায়। কিন্তু Local-এর মান বাইরে থেকে বদলানো যায় না — এটি Configuration-এর অভ্যন্তরীণ বিষয়।

৭.৮ Variable বনাম Local বনাম Output

বৈশিষ্ট্য	Variable	Local	Output
মান আসে	বাইরে থেকে	ভেতরে হিসাব করে	ভেতর থেকে বাইরে যায়
বদলানো যায়	প্রতি Run-এ	কোড বদলে	—
উদাহরণ	Environment, Count	Tag-এর সেট, নামের গঠন	IP, File Path
Prefix	`var.`	`local.`	—

নিয়ম: যা প্রতি Environment-এ বদলায় তা Variable; যা একই হিসাব বারবার লাগে তা Local; যা Apply-এর পর জানা দরকার তা Output।

৭.৯ Variable Precedence — কোন মান জিতবে?

একই Variable-এর মান একাধিক জায়গা থেকে এলে Terraform একটি নির্দিষ্ট ক্রম মানে — নিচের তালিকায় উপরেরটি জেতে:

সর্বোচ্চ অগ্রাধিকার
1. -var ও -var-file (Command Line-এ দেওয়া)
2. *.auto.tfvars ও terraform.tfvars (স্বয়ংক্রিয়ভাবে লোড)
3. TF_VAR_ Environment Variable
4. terraform.tfvars.json
5. Default মান (variable Block-এ)
সর্বনিম্ন অগ্রাধিকার

```
# terraform.tfvars
environment    = "staging"
instance_count = 1

# Command line - সবচেয়ে শক্তিশালী
$ TF_VAR_instance_count=2 terraform plan -var="environment=production"
```

এই উদাহরণে `environment` হবে `production` (`-var` জিতেছে), আর `instance_count` হবে `2` (`TF_VAR_` জিতেছে `tfvars`-এর ১-এর উপর)। Precedence না জানলে "আমি তো মান বদলেছি, তবু বদলাল না কেন?" — এই বিভ্রান্তি হয়।

WARNING: `terraform.tfvars` ফাইলে Secret লিখবেন না, আর `*.tfvars`-এ Secret থাকলে তা Git-এ Commit করবেন না। Password-এর মতো Sensitive মান দিন `TF_VAR_db_password` Environment Variable দিয়ে, বা Secret Manager থেকে। `sensitive = true` শুধু Terminal Output লুকায় — State ফাইলে মানটি এনক্রিপ্ট ছাড়া থাকলে যে কেউ পড়তে পারে।

৭.১০ TechNova সম্পূর্ণ উদাহরণ

```
# variables.tf
variable "environment" {
  type        = string
  description = "Environment name"
  default     = "staging"
  validation {
    condition     = contains(["staging", "production"], var.environment)
    error_message = "Must be staging or production."
  }
}
variable "app_ports" {
  type    = list(number)
  default = [80, 443]
}
variable "tags" {
  type    = map(string)
  default = { Team = "platform" }
}

# main.tf
locals {
  app_name = "technova-${var.environment}"
  all_tags = merge(var.tags, { Environment = var.environment })
}

resource "local_file" "app_config" {
  filename = "${path.module}/${local.app_name}.txt"
  content  = <<-EOT
    App: ${local.app_name}
    Ports: ${join(", ", [for p in var.app_ports : tostring(p)])}
    Tags: ${jsonencode(local.all_tags)}
  EOT
}

# outputs.tf
output "app_name" { value = local.app_name }
output "config_file" { value = local_file.app_config.filename }
```

```
$ terraform init && terraform plan -var="environment=production"
Plan: 1 to add, 0 to change, 0 to destroy.
$ terraform apply -auto-approve
Outputs:
app_name = "technova-production"
config_file = "./technova-production.txt"
```

৭.১১ সাধারণ ভুল ও Troubleshooting

ভুল	লক্ষণ	সমাধান
Variable ঘোষণা না করে `var.x` ব্যবহার	`Reference to undeclared variable`	`variable "x"` Block যোগ করুন
ভুল Type-এর মান দেওয়া	`Invalid value for input variable`	Type মিলিয়ে মান দিন, প্রয়োজনে `tostring()` ব্যবহার করুন
Default নেই, মানও দেওয়া হয়নি	Terraform মান চেয়ে Prompt দেখায়	Default দিন বা `-var` দিয়ে মান দিন; CI-তে Prompt আটকে যায়
Secret Output-এ দেখানো	Password Terminal-এ ছাপা হয়	Output-এ `sensitive = true` দিন; তবু State সুরক্ষিত করুন

TROUBLESHOOTING: কোন মান আসলে ব্যবহার হচ্ছে তা দেখতে `terraform plan`-এর শুরুতে Variable-এর মানগুলো দেখুন, বা `terraform console`-এ গিয়ে `var.environment` লিখে Enter চাপুন — তাৎক্ষণিক মান দেখাবে।

৭.১২ Production Engineering Practices

প্রতিটি Variable-এ `description` লিখুন — এটি আপনার Module-এর ডকুমেন্টেশন। Type যত নির্দিষ্ট, তত ভালো — `any` এড়ান। Sensitive Variable-এর মান কখনো Log-এ বা Git-এ নয়। Environment-ভিত্তিক `tfvars` ফাইল আলাদা রাখুন — `staging.tfvars`, `production.tfvars` — আর Apply-এর সময় `-var-file` দিয়ে স্পষ্ট করে দিন।

৭.১৩ অধ্যায়ের সারসংক্ষেপ

Variable বাইরে থেকে মান আনে, Local ভেতরে হিসাব করে, Output বাইরে মান দেয়। Type Constraint ভুল মান আগেই আটকায়। Precedence-এর ক্রম জানলে মান নিয়ে বিভ্রান্তি হয় না। Sensitive শুধু দেখা লুকায়, রক্ষা করে না — রক্ষা করে Backend-এর Encryption ও Access Control।

৭.১৪ Knowledge Check

১. Variable, Local ও Output-এর পার্থক্য কী? ২. `-var`, `tfvars` ও `TF_VAR_`-এর মধ্যে কোনটি জেতে? ৩. `list` ও `set`-এর পার্থক্য কী? ৪. `validation` Block কেন লিখবেন? ৫. Sensitive Variable-এর মান কি State ফাইলে নিরাপদ?

৭.১৫ Practical Assignment

TechNova-র জন্য একটি Module লিখুন যেখানে `environment` (Validation-সহ), `instance_count`, `enable_monitoring` (Bool), `tags` (Map) Variable আছে; Local-এ App-এর নাম ও Tag তৈরি হয়; Output-এ App-এর নাম দেখায়। Staging ও Production — দুটি আলাদা `tfvars` ফাইল দিয়ে দুবার Plan করুন, পার্থক্য লিখে রাখুন। একটি Sensitive Variable যোগ করে দেখুন Plan Output-এ তা কীভাবে লুকানো হয়।

অধ্যায় ৮ • PART II — Terraform Core Engineering

Chapter 8 — Resources, Data Sources, and Resource Dependencies

৮.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — Managed Resource ঘোষণা করতে ও তার Argument ও Attribute-এর পার্থক্য বলতে; Data Source দিয়ে বিদ্যমান Infrastructure পড়তে; Implicit ও Explicit Dependency-এর পার্থক্য করতে ও `depends_on` ব্যবহার করতে; Dependency Graph বুঝতে ও `terraform graph` পড়তে; Resource Addressing (`aws_instance.web`) ও Reference/Interpolation সঠিকভাবে লিখতে; এবং `local` Provider দিয়ে সম্পূর্ণ উদাহরণ চালাতে।

৮.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

Terraform-এর কাজের কেন্দ্রে আছে Resource। আপনি যা তৈরি করতে চান — Server, Database, File, Network — সবই Resource। কিন্তু Infrastructure কখনো একা দাঁড়ায় না: Database তৈরি হওয়ার আগে Network থাকতে হবে, App Server-এর আগে Database-এর Address জানতে হবে। এই ক্রমটি ঠিক করাই Dependency। Dependency ভুল হলে Apply মাঝপথে ব্যর্থ হয়, অর্ধেক Infrastructure তৈরি হয়ে পড়ে থাকে। TechNova-তে App Server চালু হওয়ার আগে তার Configuration File তৈরি হওয়া, আর সেই File-এর আগে Directory থাকা — এই ছোট ক্রমটিও Dependency-রই উদাহরণ।

> **REAL WORLD:** TechNova-র এক Deploy-এ App Server তৈরি হয়েছিল, কিন্তু তার Configuration File-এর Path ভুল ছিল — কারণ File-টি তৈরি হয়েছিল Server-এর পরে, আর Server চালু হওয়ার সময় File-টি তখনো তৈরি হয়নি। সমস্যা ছিল Dependency ঘোষণা না করা। Reference দিয়ে Dependency বোঝালে Terraform নিজেই ক্রম ঠিক করে দিত।

৮.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ৬-এর Block ও Expression, অধ্যায় ৭-এর Variable ও Local — এই দুটি জানা থাকতে হবে। Resource-এর ধারণা Part I থেকে পরিচিত হওয়ার কথা।

৮.৪ Resource: প্রথম নীতি

Resource হলো Terraform-এর পরিচালিত (Managed) Infrastructure-এর একক। প্রতিটি Resource একটি Provider-এর একটি Type-এর অন্তর্গত। ঘোষণার গঠন:

```
resource "TYPE" "NAME" {
  argument = value
}
```

যেমন `resource "local_file" "welcome"` — Type `local_file`, নাম `welcome`। এই নামটি Configuration-এর ভেতরে Resource-টিকে চিহ্নিত করে; বাস্তব জগতের নাম নয়। Resource-এর দুই ধরনের ক্ষেত্র আছে: **Argument** —

আপনি যা দেন (যেমন `filename`, `content`); **Attribute** — Provider যা ফেরত দেয় (যেমন তৈরি হওয়ার পর File-এর `id`, `content_sha256`)। Argument লেখার সময় জানা থাকে, Attribute জানা যায় তৈরি হওয়ার পর — তাই Attribute-কে **Computed** বলা হয়।

> **CONCEPT:** Resource-এর Address হলো তার পূর্ণ পরিচয় — `local_file.welcome` বা `aws_instance.web`। Type ও নাম মিলে Address হয়। Terraform State-এ Resource এই Address দিয়েই সংরক্ষিত থাকে, আর অন্য Resource এই Address দিয়েই তাকে ডাকে।

৮.৫ Meta-Argument-এর পূর্বাভাস

সব Resource-এ কিছু বিশেষ Argument ব্যবহার করা যায়, যাদের বলে **Meta-Argument** — এগুলো কোনো নির্দিষ্ট Provider-এর নয়, Terraform-এর নিজের। এগুলোর বিস্তারিত আলোচনা পরের Volume-এ আসবে, তবে নামগুলো এখন জেনে রাখুন:

Meta-Argument	কাজ
`count`	একই Resource-এর একাধিক Copy তৈরি করা
`for_each`	Map বা Set-এর প্রতিটি মানের জন্য একটি Resource
`provider`	কোন Provider Configuration ব্যবহার হবে
`depends_on`	স্পষ্ট Dependency ঘোষণা
`lifecycle`	তৈরি/ধ্বংসের আচরণ নিয়ন্ত্রণ (অধ্যায় ১১)

```
resource "local_file" "app" {
  count    = 3
  filename = "${path.module}/app-${count.index}.txt"
  content  = "TechNova app server ${count.index}"
}
```

এখানে `count.index` হলো ০, ১, ২ — প্রতিটি Copy-এর নম্বর। Address হয় `local_file.app[0]`, `local_file.app[1]` ইত্যাদি।

৮.৬ Data Source: পড়া, তৈরি করা নয়

Data Source হলো Read-Only Resource — এটি কিছু তৈরি করে না, বিদ্যমান জিনিসের তথ্য পড়ে আনে। যেমন — ইতিমধ্যে থাকা একটি File-এর Content পড়া, বা AWS-এ থাকা একটি AMI-এর ID জানা। ঘোষণা:

```
data "local_file" "existing" {
  filename = "${path.module}/welcome.txt"
}

output "existing_content" {
  value = data.local_file.existing.content
}
```

Data Source ব্যবহার করা হয় `data.` Prefix দিয়ে — `data.local_file.existing.content`। কখন ব্যবহার করবেন? যখন তথ্যটি Terraform তৈরি করেনি, কিন্তু আপনার Configuration-এর দরকার। যেমন — অন্য দলের তৈরি Network-এর ID, বা বাইরের File-এর বিষয়বস্তু। মনে রাখুন: Data Source Plan-এর সময় পড়া হয় (সাধারণত), তাই এর মানের উপর নির্ভরশীল Resource সঠিক ক্রমে তৈরি হয়।

৮.৭ Dependency: Implicit বনাম Explicit

Dependency দুই ধরনের। **Implicit Dependency** তৈরি হয় Reference-এর মাধ্যমে — আপনি যখন একটি Resource-এর Attribute অন্য Resource-এ ব্যবহার করেন, Terraform বুঝে নেয় প্রথমটি আগে তৈরি হতে হবে:

```
resource "local_file" "config" {
  filename = "${path.module}/config.txt"
  content  = "App: technova.example"
}

resource "local_file" "summary" {
  filename = "${path.module}/summary.txt"
  content  = "Config hash: ${local_file.config.content_sha256}"
}
```

এখানে `summary` Resource `config`-এর `content_sha256` ব্যবহার করেছে — তাই Terraform প্রথমে `config` তৈরি করবে, তারপর `summary`। কোনো ঘোষণা লিখতে হয়নি — Reference-ই Dependency।

Explicit Dependency দরকার হয় যখন কোনো Reference নেই, তবু ক্রম জরুরি। তখন `depends_on` ব্যবহার করুন:

```
resource "local_file" "app" {
  filename   = "${path.module}/app.txt"
  content    = "TechNova app"
  depends_on = [local_file.config]
}
```

নিয়ম: Reference দিয়ে পারলে `depends_on` ব্যবহার করবেন না। `depends_on` শুধু তখনই, যখন Dependency লুকানো — যেমন একটি Resource অন্য Resource-এর Side Effect-এর উপর নির্ভর করে। অতিরিক্ত `depends_on` Graph-কে জটিল করে, Parallelism কমায়, আর ভুল Dependency লুকিয়ে রাখে।

৮.৮ Dependency Graph ও terraform graph

Terraform সব Resource ও তাদের Dependency নিয়ে একটি **Dependency Graph** তৈরি করে — একটি Directed Acyclic Graph (DAG)। Graph-এ চক্র (Cycle) থাকলে Error হয় — A নির্ভর করে B-এর উপর, B নির্ভর করে A-এর উপর — এমন অবস্থা অসম্ভব, Terraform তা ধরে ফেলে। Graph দেখতে:

```
$ terraform graph | dot -Tpng > graph.png
# অথবা শুধু Text হিসেবে দেখুন:
$ terraform graph
```

```
local_file.config
        |
| content_sha256 (reference)
        ▼
local_file.summary

Graph-এর নিয়ম:
- তীর মানে "আগে তৈরি হতে হবে"
- স্বাধীন Resource সমান্তরালে (Parallel) তৈরি হয়
- চক্র (Cycle) থাকলে Error: "Cycle: ..."
```

Apply-এর সময় Terraform Graph-এর Topological Order মেনে চলে, আর স্বাধীন Resource একসঙ্গে তৈরি করে — এজন্যই বড় Infrastructure দ্রুত তৈরি হয়।

৮.৯ Reference ও Interpolation

Reference লেখার নিয়ম: `<type>.<name>.<attribute>` — যেমন `local_file.config.filename`। String-এর ভেতরে Reference বসাতে Interpolation ব্যবহার করুন: `"Config: ${local_file.config.filename}"`। Terraform 0.12+ থেকে শুধু Reference হলে কোটের প্রয়োজন নেই — `content = local_file.config.content` সরাসরি লেখা যায়। তবে String-এর মাঝখানে মান বসাতে `${ ... }` লাগবেই।

TROUBLESHOOTING: `Reference to undeclared resource` মানে নাম বা Type-এ বানান ভুল। `Cycle` Error মানে দুটি Resource পরস্পরের উপর নির্ভরশীল — ডিজাইন বদলাতে হবে, কোনো একদিকে Dependency ভাঙতে হবে। `Invalid reference` মানে Attribute-এর নাম ভুল — Provider-এর ডকুমেন্টেশনে সঠিক Attribute দেখুন।

৮.১০ সম্পূর্ণ উদাহরণ — Local Provider

```
# main.tf — TechNova local example, কোনো Cloud Credential লাগে না
terraform {
  required_version = ">= 1.9, < 2.0"
  required_providers {
    local = { source = "hashicorp/local", version = "~> 2.5" }
  }
}

variable "environment" { type = string, default = "staging" }

locals { app_name = "technova-${var.environment}" }

data "local_file" "template" {
  filename = "${path.module}/template.txt" # আগে থেকে থাকা ফাইল
}

resource "local_file" "config" {
```

```
  filename = "${path.module}/${local.app_name}-config.txt"
  content  = "App: ${local.app_name}\nTemplate: ${data.local_file.template.content}"
}

resource "local_file" "summary" {
  filename = "${path.module}/${local.app_name}-summary.txt"
  content  = "Config SHA: ${local_file.config.content_sha256}"
  depends_on = [local_file.config]
}

output "config_path" { value = local_file.config.filename }
```

```
$ terraform init && terraform apply -auto-approve
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
```

৮.১১ AWS-ধরনের উদাহরণ (Credential প্রয়োজন — শুধু ধারণার জন্য)

> **WARNING:** নিচের উদাহরণটি চালাতে আসল AWS Credential ও খরচের প্রয়োজন — এটি শুধু ধারণা বোঝার জন্য। Credential কখনোই কোডে লিখবেন না; Environment Variable বা AWS CLI Profile ব্যবহার করুন।

```
# AWS-style example — DO NOT run without credentials and cost awareness
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"] # Canonical (Ubuntu)
  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
  }
}

resource "aws_instance" "web" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t3.micro"
  tags          = { Name = "technova-web", Env = "staging" }
}

output "web_public_ip" { value = aws_instance.web.public_ip }
```

এখানে Data Source (`aws_ami`) Ubuntu Image-এর ID পড়ে আনে, Resource (`aws_instance`) সেই ID ব্যবহার করে Server তৈরি করে — Implicit Dependency। `public_ip` একটি Computed Attribute — Apply-এর পরেই জানা যায়।

৮.১২ সাধারণ ভুল ও Production Practices

ভুল	পরিণতি	সঠিক অভ্যাস
অপ্রয়োজনে `depends_on`	ধীর Apply, জটিল Graph	Reference দিয়ে Dependency প্রকাশ করুন
Attribute-এর নাম ভুল	Plan ব্যর্থ	Provider ডকুমেন্টেশন দেখুন

Data Source দিয়ে তৈরি করা জিনিস পড়া	অপ্রয়োজনীয় জটিলতা	নিজের Resource-এর Attribute সরাসরি ব্যবহার করুন
Cycle তৈরি করা	Apply অসম্ভব	একদিকের Dependency সরান

৮.১৩ অধ্যায়ের সারসংক্ষেপ

Resource তৈরি করে, Data Source পড়ে। Argument আপনি দেন, Attribute Provider ফেরত দেয়। Reference-ই সবচেয়ে ভালো Dependency — এটি Graph তৈরি করে, Graph-ই Apply-এর ক্রম ঠিক করে। `depends_on` শুধু লুকানো Dependency-র জন্য। Resource Address (`type.name`) হলো Terraform-এর ভেতরে প্রতিটি জিনিসের পরিচয়।

৮.১৪ Knowledge Check

১. Argument ও Attribute-এর পার্থক্য কী? ২. Data Source কখন ব্যবহার করবেন? ৩. Implicit ও Explicit Dependency-এর পার্থক্য কী? ৪. `terraform graph` কী দেখায়? ৫. Cycle Error-এর অর্থ কী?

৮.১৫ Practical Assignment

Local Provider দিয়ে একটি Chain তৈরি করুন: একটি Template File (হাতে তৈরি) → Data Source দিয়ে পড়ুন → Config File তৈরি করুন → Summary File তৈরি করুন যাতে Config-এর Hash থাকে। `terraform graph` চালিয়ে Graph-টি পড়ুন ও আঁকুন। তারপর ইচ্ছাকৃতভাবে `depends_on` সরিয়ে দেখুন Reference-ই ক্রম ঠিক রাখে কি না। শেষে একটি Cycle তৈরি করে Error Message-টি লিখে রাখুন।

অধ্যায় ৯ • PART II — Terraform Core Engineering

Chapter 9 — Terraform State — Purpose, Lifecycle, and Internal Structure

৯.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — State কী এবং Terraform-এর কেন State দরকার তা প্রথম নীতি থেকে ব্যাখ্যা করতে; `terraform.tfstate` ফাইলের ভেতরের JSON গঠন ক্ষেত্র ধরে ধরে পড়তে; State-এ কী কী সংরক্ষিত হয় (Resource, Attribute, Dependency) তা বলতে; Plan ও Apply-এর সময় State-এর Lifecycle ব্যাখ্যা করতে; State হারিয়ে গেলে বা বাস্তবতার সঙ্গে না মিললে কী হয় তা বলতে; এবং Drift-এর ধারণা ব্যাখ্যা করতে।

৯.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

Terraform Declarative — আপনি বলেন কী চাই। কিন্তু দ্বিতীয়বার Apply দেওয়ার সময় Terraform কীভাবে জানবে কী আগে থেকে আছে, কী বদলাতে হবে, কী মুছতে হবে? উত্তর: State। State হলো Terraform-এর স্মৃতি — গতবার কী তৈরি হয়েছিল, তার পরিচয় কী, তার বৈশিষ্ট্য কী ছিল। State ছাড়া Terraform প্রতিবারই মনে করত কিছুই নেই, আর সবকিছু আবার তৈরি করতে চাইত। TechNova-তে State-ই সেই খাতা, যেখানে লেখা আছে কোন Server-টি Terraform-এর তৈরি, কোনটি হাতে তৈরি।

> **REAL WORLD:** TechNova-র এক Engineer ভুল করে `terraform.tfstate` ফাইলটি মুছে ফেলেছিলেন। পরের `plan`-এ Terraform বলল — সবকিছু নতুন করে তৈরি করতে হবে, যদিও Server-গুলো তখনো চলছিল। কারণ Terraform-এর স্মৃতি মুছে গিয়েছিল। ভাগ্য ভালো, Backup ছিল। এই অধ্যায় পড়লে বুঝবেন কেন State মুছে ফেলা একটি বিপর্যয়, আর কেন এর Backup ও সুরক্ষা অপরিহার্য।

৯.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ৮-এর Resource, Attribute ও Dependency জানা থাকতে হবে। JSON-এর মৌলিক ধারণা (Key-Value, Array, Object) জানা থাকলে State ফাইল পড়তে সুবিধা হবে।

৯.৪ State কী — প্রথম নীতি

Terraform State হলো একটি JSON ফাইল — সাধারণত `terraform.tfstate` — যেখানে Terraform তার পরিচালিত প্রতিটি Resource-এর বর্তমান অবস্থা লিখে রাখে। এতে থাকে: Resource-এর Address, Provider-এর দেওয়া আসল ID (যেমন AWS Instance ID `i-0abc123`), প্রতিটি Attribute-এর মান, Dependency-র তথ্য, এবং Metadata (Terraform Version, Serial Number, Lineage)।

কেন দরকার? তিনটি কারণে। প্রথম, **Mapping** — Configuration-এর `aws_instance.web` বাস্তবে কোন Instance (`i-0abc123`)-এর সঙ্গে মিলে, তা State-ই জানে। দ্বিতীয়, **Change Detection** — Plan-এর সময় Terraform Configuration, State ও বাস্তব Infrastructure — এই তিনটি তুলনা করে সিদ্ধান্ত নেয় কী বদলাতে হবে। তৃতীয়, **Performance** — প্রতিবার সব Resource-এর সব Attribute API দিয়ে পড়া ধীর; State-এ সংরক্ষিত মান দিয়ে অনেক সিদ্ধান্ত দ্রুত নেওয়া যায় (যদিও Refresh-এর সময় বাস্তব অবস্থা আবার পড়া হয়)।

> **CONCEPT:** একটি গুরুত্বপূর্ণ সত্য — **Configuration হলো Source of Truth, State নয়।** Configuration বলে কী হওয়া উচিত; State বলে Terraform গতবার কী দেখেছিল। State ভুল হতে পারে, পুরোনো হতে পারে, হারিয়ে যেতে পারে — কিন্তু Configuration-ই চূড়ান্ত ইচ্ছা। State-কে হাতে সম্পাদনা করে "সত্য" বদলানোর চেষ্টা করবেন না; Configuration বদলে Apply দিন।

৯.৫ State ফাইলের ভেতরের গঠন — ক্ষেত্র ধরে ধরে

নিচে একটি বাস্তবধর্মী `terraform.tfstate`-এর অংশ — Local Provider-এর উদাহরণ দিয়ে। প্রতিটি ক্ষেত্রের অর্থ দেওয়া হলো:

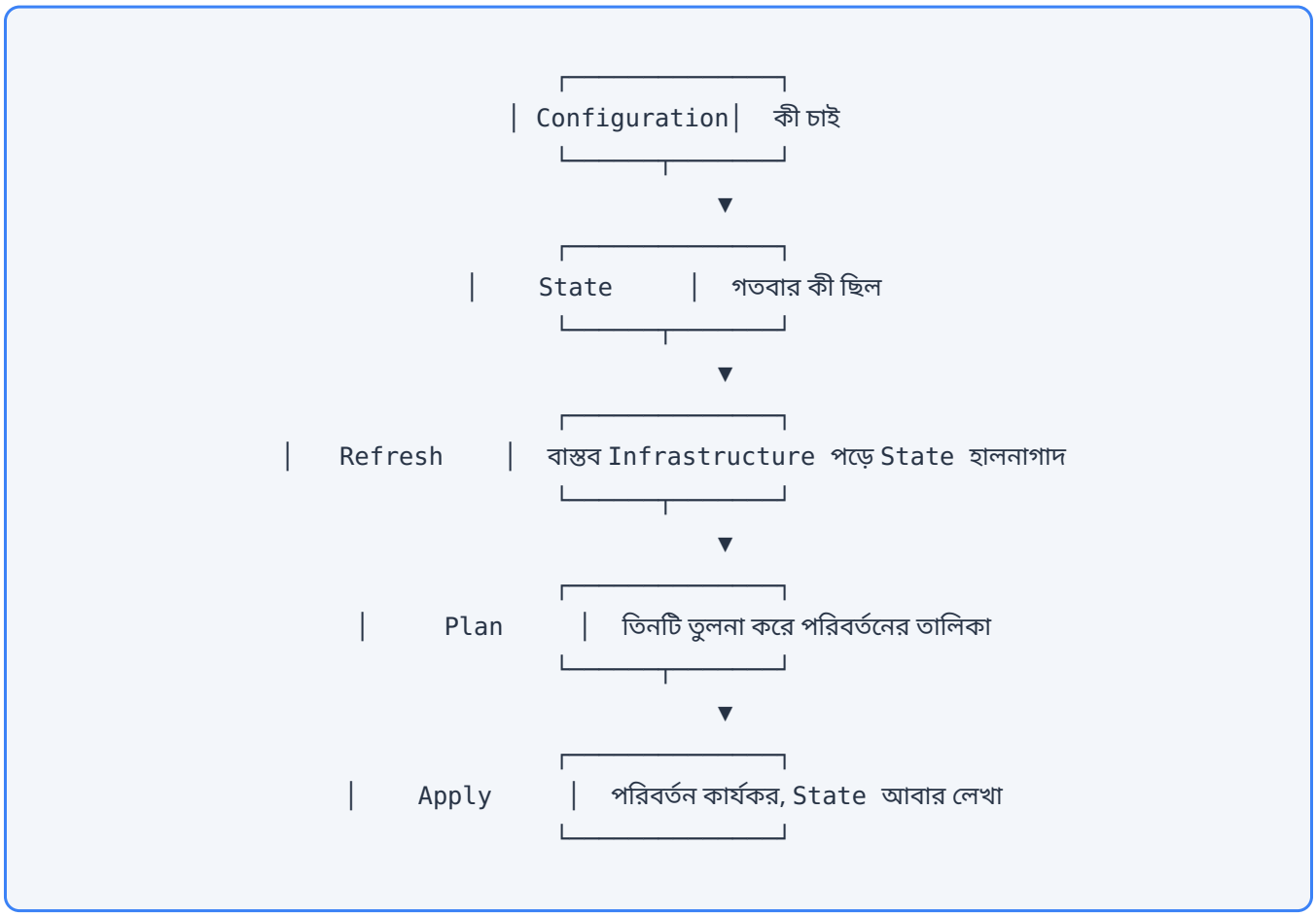
```
{
  "version": 4,
  "terraform_version": "1.9.8",
  "serial": 3,
  "lineage": "a1b2c3d4-1111-2222-3333-444455556666",
  "outputs": {
    "config_path": {
      "value": "./technova-staging-config.txt",
      "type": "string"
    }
  },
  "resources": [
    {
      "mode": "managed",
      "type": "local_file",
      "name": "config",
      "provider": "provider[\"registry.terraform.io/hashicorp/local\"]",
      "instances": [
        {
          "schema_version": 0,
          "attributes": {
            "id": "9f86d081884c7d659a2feaa0c55ad015",
            "filename": "./technova-staging-config.txt",
            "content": "App: technova-staging\n",
            "content_sha256": "9f86d081884c7d659a2feaa0c55ad015",
            "file_permission": "0777"
          },
          "dependencies": ["data.local_file.template"]
        }
      ]
    }
  ],
  "check_results": null
}
```

ক্ষেত্র	অর্থ
`version`	State Format-এর সংস্করণ (বর্তমানে ৪)
`terraform_version`	যে Terraform সংস্করণ State লিখেছিল
`serial`	প্রতি লেখায় ১ করে বাড়ে — State-এর "সংস্করণ নম্বর"

`lineage`	State-এর স্থায়ী পরিচয় — নতুন State তৈরি হলে বদলায়
`outputs`	Output Value-গুলো ও তাদের Type
`resources`	প্রতিটি Resource-এর তালিকা
`mode`	`managed` (Resource) বা `data` (Data Source)
`instances`	Resource-এর প্রতিটি Copy (count/for_each-এ একাধিক)
`attributes`	Provider-এর ফেরত দেওয়া সব মান, `id` -সহ
`dependencies`	এই Resource যাদের উপর নির্ভরশীল

লক্ষ করুন: `attributes.id` -ই বাস্তব জগতের পরিচয়। Configuration-এর নাম (`config`) আর বাস্তব ID আলাদা — State-ই তাদের সংযোগ। আর `content_sha256` -এর মতো Computed Attribute-ও State-এ থাকে, যাতে পরের Plan-এ তুলনা করা যায়।

৯.৬ State Lifecycle: Plan ও Apply-এর সময় কী ঘটে



Plan-এর সময় Terraform প্রথমে State পড়ে, তারপর (সাধারণত) বাস্তব Infrastructure Refresh করে State হালনাগাদ করে, তারপর Configuration-এর সঙ্গে তুলনা করে। Apply-এর সময় প্রতিটি পরিবর্তনের পর State আবার লেখা হয় — `serial` বাড়ে। Apply মাঝপথে ব্যর্থ হলে State আংশিক হালনাগাদ থাকে — যা তৈরি হয়েছে তা State-এ আছে, যা হয়নি তা নেই। এজন্যই ব্যর্থ Apply-এর পর আবার Plan দিলে Terraform বুঝতে পারে কোথায় থেমেছিল।

৯.৭ State হারিয়ে গেলে বা না মিললে কী হয়

পরিস্থিতি	যা ঘটে	করণীয়
State ফাইল মুছে গেছে	Terraform মনে করে কিছুই নেই; সব নতুন করে তৈরি করতে চায়	Backup থেকে ফেরান; না থাকলে `terraform import` দিয়ে Resource State-এ আনুন
বাস্তবে Resource মুছে গেছে, State-এ আছে	Plan দেখায় Resource আবার তৈরি হবে	সাধারণত ঠিক আচরণ — Configuration অনুযায়ী আবার তৈরি হবে
বাস্তবে কেউ হাতে বদলেছে (Drift)	Plan দেখায় Configuration-এ ফেরত নেওয়া হবে	Drift ইচ্ছাকৃত হলে Configuration হালনাগাদ করুন; না হলে Apply দিন
দুজন একসঙ্গে Apply দিচ্ছে	State নষ্ট (Corrupt) হতে পারে	Remote Backend + Locking ব্যবহার করুন (অধ্যায় ১০)

WARNING: State ফাইলে Sensitive মান থাকতে পারে — Password, Key, Token — এমনকি Output-এ `sensitive = true` দিলেও। কারণ State-এ Resource-এর সব Attribute Plain Text-এ থাকে। তাই State ফাইল কখনোই Git-এ Commit করবেন না, Email-এ পাঠাবেন না, বা Public জায়গায় রাখবেন না।

৯.৮ Drift — বাস্তবতা যখন Configuration থেকে সরে যায়

Drift মানে বাস্তব Infrastructure আর Configuration-এর মধ্যে পার্থক্য — কেউ হাতে Console থেকে Server-এর Type বদলে দিয়েছে, বা কেউ সরাসরি File সম্পাদনা করেছে। Terraform Refresh-এর সময় Drift ধরে ফেলে, আর Plan-এ দেখায় Configuration-এ ফেরত নেওয়ার পরিকল্পনা। Drift খারাপ নয় — এটি একটি সংকেত। প্রশ্ন হলো: Drift-টি ইচ্ছাকৃত ছিল কি না। ইচ্ছাকৃত হলে Configuration হালনাগাদ করুন; না হলে Apply দিয়ে Configuration-এর অবস্থায় ফেরত আনুন। নিয়মিত `terraform plan` চালানোই Drift ধরার সহজ উপায়।

৯.৯ State দেখার নিরাপদ উপায়

```
$ terraform state list
data.local_file.template
local_file.config
local_file.summary

$ terraform state show local_file.config
# local_file.config:
resource "local_file" "config" {
    content          = "App: technova-staging\n"
    content_sha256   = "9f86d081884c7d659a2feaa0c55ad015"
    filename         = "./technova-staging-config.txt"
    id               = "9f86d081884c7d659a2feaa0c55ad015"
}
```

`terraform state list` ও `show` হলো State পড়ার নিরাপদ উপায় — সরাসরি JSON ফাইল সম্পাদনা করবেন না। `terraform show` পুরো State মানুষের পড়ার মতো করে দেখায়।

৯.১০ সাধারণ ভুল ও Troubleshooting

TROUBLESHOOTING: `Error acquiring the state lock` মানে অন্য কেউ Apply দিচ্ছে বা আগের Apply আটকে গেছে — অধ্যায় ১০ দেখুন। `State data in S3 does not have the lineage` মানে ভুল Backend বা ভুল State ফাইল — Backend Configuration যাচাই করুন। Plan-এ হঠাৎ সব Resource "নতুন" দেখালে প্রথমে State ফাইল আছে কি না, Backend ঠিক আছে কি না দেখুন — Configuration-এ হাত দেওয়ার আগে।

৯.১১ Production Engineering Practices

State-এর Backup রাখুন — Remote Backend-এ Versioning চালু করুন। Local State ব্যবহার করলে `.gitignore`-এ `*.tfstate*` যোগ করুন। একাধিক Engineer একসঙ্গে কাজ করলে Remote Backend ও Locking বাধ্যতামূলক। State-এর Access সীমিত রাখুন — যার State পড়ার অধিকার আছে, সে Sensitive মানও পড়তে পারে।

৯.১২ অধ্যায়ের সারসংক্ষেপ

State হলো Terraform-এর স্মৃতি — Configuration-এর নামের সঙ্গে বাস্তব ID-র সংযোগ। Plan তিনটি জিনিস তুলনা করে: Configuration, State ও বাস্তবতা। State হারালে Terraform সবকিছু নতুন মনে করে; Drift হলে Plan তা ফেরত নিতে চায়। Configuration-ই Source of Truth — State শুধু তার প্রতিফলন। আর State-এ Sensitive মান থাকে — তাই এর সুরক্ষা Infrastructure-এর সুরক্ষারই অংশ।

৯.১৩ Knowledge Check

১. State কেন দরকার? ২. `serial` ও `lineage`-এর কাজ কী? ৩. Configuration বনাম State — কোনটি Source of Truth? ৪. Drift কী? ৫. State মুছে গেলে কী হয়?

৯.১৪ Practical Assignment

অধ্যায় ৮-এর Local উদাহরণটি চালান, তারপর `terraform.tfstate` ফাইলটি খুলে প্রতিটি ক্ষেত্র চিহ্নিত করুন। `terraform state list` ও `state show` চালান। তারপর হাতে তৈরি করা File-টির Content বদলে (Drift তৈরি করে) `terraform plan` চালান — Terraform কী দেখায় লিখুন। State ফাইলের একটি Copy Backup হিসেবে সংরক্ষণ করুন এবং `.gitignore`-এ `*.tfstate*` যোগ করুন।

অধ্যায় ১০ • PART II — Terraform Core Engineering

Chapter 10 — State Management — Remote Backend, Locking, and State Operations

১০.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — Remote Backend কেন দরকার তা ব্যাখ্যা করতে; S3 + DynamoDB Locking-এর ক্লাসিক Pattern-সহ Backend Configuration লিখতে; `azurerm`, `gcs`, `http` ও Terraform Cloud-এর ধারণা বলতে; State Locking কীভাবে কাজ করে ও Lock Error-এর সময় কী করবেন তা বলতে; `terraform state` Subcommand-গুলো নিরাপদে ব্যবহার করতে; Local থেকে Remote-এ State নিরাপদে Migrate করতে; এবং Backend-এর Encryption ও Access Control ব্যাখ্যা করতে।

১০.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

একজন Engineer একা কাজ করলে Local State ফাইল চলতে পারে। কিন্তু TechNova-তে যখন পাঁচজন Engineer একই Infrastructure-এ কাজ করেন, তখন Local State একটি বিপদ: কারো ল্যাপটপে থাকা State-ই "সত্য" হয়ে যায়, অন্যরা তা দেখতে পায় না, দুজন একসঙ্গে Apply দিলে State নষ্ট হয়। Remote Backend এই সমস্যার সমাধান — State একটি ভাগ করা, সুরক্ষিত জায়গায় থাকে, সবাই একই State দেখে, আর Locking নিশ্চিত করে এক সময়ে একজনই State বদলাতে পারে। এটি দলগত Terraform-এর ভিত্তি।

> **REAL WORLD:** TechNova-র Staging-এ একবার দুজন Engineer একই সময়ে Apply দিয়েছিলেন — একজন Database-এর Size বদলাচ্ছিলেন, অন্যজন Server-এর সংখ্যা। দুজনের Local State আলাদা ছিল। ফল: State ফাইল দুটি ভিন্ন সংস্করণে ভাগ হয়ে গেল, আর পরের Plan-এ Terraform বিভ্রান্তিকর পরিবর্তন দেখাতে লাগল। Remote Backend + Locking থাকলে দ্বিতীয় Apply-টি Lock Error দিয়ে থেমে যেত — এটাই কাঙ্ক্ষিত আচরণ।

১০.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ৯-এর State গঠন ও Lifecycle জানা থাকতে হবে। Cloud Storage (S3-এর মতো Object Storage) ও IAM-এর মৌলিক ধারণা থাকলে ভালো।

১০.৪ Backend কী

Backend হলো State সংরক্ষণের জায়গা ও পদ্ধতি — Terraform State কোথায় থাকবে, কীভাবে Lock হবে। Default Backend হলো `local` — State আপনার কম্পিউটারে `terraform.tfstate` ফাইলে থাকে। Remote Backend-এ State Cloud-এ থাকে। Backend ঘোষণা করা হয় `terraform` Block-এর ভেতরে:

```
terraform {
  backend "s3" {
    bucket         = "technova-terraform-state-CHANGE-ME"
    key            = "staging/terraform.tfstate"
    region         = "ap-south-1"
    encrypt        = true
    dynamodb_table = "technova-terraform-locks"
```

```
  }
}
```

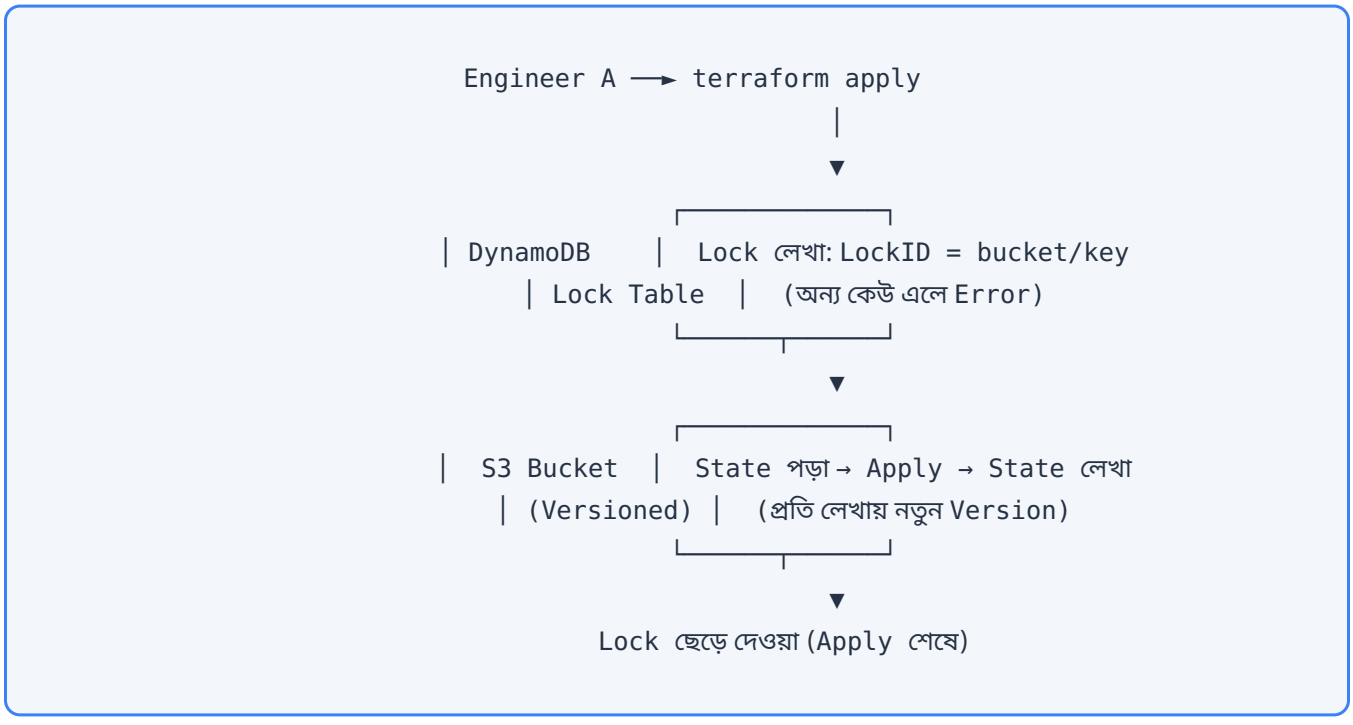
মনে রাখুন: Backend Configuration-এ Variable ব্যবহার করা যায় না — Backend Init-এর আগেই ঠিক হতে হয়। মান দিতে হলে `-backend-config` Flag বা আলাদা Configuration ফাইল ব্যবহার করুন। Backend বদলালে `terraform init -migrate-state` চালাতে হয়।

> **CONCEPT:** Backend শুধু Storage নয় — এটি দলের সমন্বয়ের যন্ত্র। State কোথায় আছে, কে বদলাতে পারে, একসঙ্গে দুজন বদলাতে পারবে কি না — এই তিনটি প্রশ্নের উত্তর Backend-ই দেয়।

১০.৫ S3 + DynamoDB — ক্লাসিক Pattern

সবচেয়ে প্রচলিত Remote Backend Pattern হলো AWS S3 + DynamoDB। S3-তে State ফাইল থাকে, DynamoDB-তে Lock-এর তথ্য থাকে। কেন এই জুটি? S3-তে Versioning চালু করলে State-এর প্রতিটি সংস্করণ সংরক্ষিত থাকে — ভুল হলে আগের সংস্করণে ফেরা যায়। Encryption চালু করলে State বিশ্রামে (At Rest) এনক্রিপ্ট থাকে। DynamoDB-তে Lock Table-এ একটি Item লেখা হয় Apply-এর সময় — অন্য কেউ Apply দিতে গেলে দেখে Lock আছে, তাই থেমে যায়।

```
# Backend-এর জন্য প্রয়োজনীয় AWS Resource (একবারই তৈরি করতে হয়)
# S3 Bucket: technova-terraform-state-CHANGE-ME
#   - Versioning: Enabled
#   - Encryption: AES-256 / KMS
#   - Public Access: Blocked
# DynamoDB Table: technova-terraform-locks
#   - Partition Key: LockID (String)
#   - Billing: Pay per request
```



১০.৬ অন্যান্য Backend

Backend	কোথায় State থাকে	Locking	কখন ব্যবহার করবেন
`s3`	AWS S3	DynamoDB	AWS-এ কাজ করলে — সবচেয়ে প্রচলিত
`azurerm`	Azure Blob Storage	Blob Lease	Azure-এ কাজ করলে
`gcs`	Google Cloud Storage	অন্তর্নির্মিত	GCP-এ কাজ করলে
`http`	যেকোনো HTTP Endpoint	Endpoint-এর উপর নির্ভরশীল	নিজের State Server থাকলে
`remote` / Terraform Cloud	HashiCorp Cloud	অন্তর্নির্মিত	Managed State, Run History, Team Workflow চাইলে
`local`	স্থানীয় ফাইল	নেই (ফাইল Lock ছাড়া)	শেখা, একা কাজ, পরীক্ষা

Terraform Cloud (HCP Terraform) শুধু State রাখে না — Run History, Policy Check, Team Permission, VCS Integration-ও দেয়। ছোট দলের জন্য এটি একটি ভালো Managed বিকল্প।

১০.৭ State Locking — ভেতরে কী ঘটে

Apply বা Plan (যা State লেখে) শুরু হলে Terraform প্রথমে Lock নেয়। S3 Backend-এ Lock মানে DynamoDB Table-এ একটি Item লেখা — যাতে থাকে কে Lock করেছে, কখন, কোন Operation। Apply শেষ হলে Lock মুছে যায়। কেউ মাঝখানে Apply দিলে সে এই Error পায়:

```
Error: Error acquiring the state lock
Error message: ConditionalCheckFailedException: The conditional request failed
Lock Info:
  ID:        a1b2c3d4-xxxx-xxxx-xxxx-xxxxxxxxxxxx
  Path:      technova-terraform-state-CHANGE-ME/staging/terraform.tfstate
  Operation: OperationTypeApply
  Who:       nahid@technova-laptop
  Created:   2026-11-09 00:30:00 +06
```

এই Error দেখলে প্রথমে অপেক্ষা করুন — অন্য কারো Apply শেষ হলে Lock নিজে থেকেই ছেড়ে যাবে। Lock Info-তে "Who" দেখে সেই ব্যক্তিকে জিজ্ঞেস করুন। `terraform force-unlock` শুধু তখনই ব্যবহার করবেন যখন নিশ্চিত যে কোনো Apply চলছে না — যেমন আগের Apply Crash করেছে, Process মারা গেছে। ভুল সময়ে Force Unlock করলে দুটি Apply একসঙ্গে State লিখবে — State নষ্ট হওয়ার সবচেয়ে সহজ পথ।

> **WARNING:** `terraform force-unlock` একটি বিপজ্জনক Command। এটি Lock ভাঙে, কিন্তু চলমান Apply থামায় না। নিশ্চিত না হয়ে কখনোই চালাবেন না। Lock ID-টি Error Message থেকে নিয়ে ব্যবহার করুন — আন্দাজে নয়।

১০.৮ State Operation — নিরাপদ Subcommand

```
$ terraform state list                    # State-এ কী কী Resource আছে
$ terraform state show local_file.config  # একটি Resource-এর বিবরণ
$ terraform state pull > backup.tfstate  # State ডাউনলোড (Backup)
$ terraform state mv local_file.old local_file.new   # Resource-এর নাম বদল
$ terraform state rm local_file.unused    # State থেকে Resource সরান
                                          # (বাস্তব Resource মুছবে না!)
```

প্রতিটি Command-এর অর্থ বুঝুন: `state mv` মানে State-এ Resource-এর নাম বদলানো — Configuration-এ নাম বদলালে Terraform পুরোনোটি মুছে নতুনটি তৈরি করতে চায়; `mv` দিলে সে বোঝে এটি একই জিনিস, শুধু নাম বদলেছে। `state rm` মানে Terraform আর ওই Resource পরিচালনা করবে না — বাস্তব Resource থেকে যাবে, কিন্তু Terraform-এর নজরের বাইরে। `state push` দিয়ে State আপলোড করা যায় — শুধু Backup থেকে ফেরানোর মতো জরুরি কাজে।

> **CONCEPT:** State Operation হলো Surgery — দরকার হলে করতে হয়, কিন্তু নিয়মিত Workflow নয়। সাধারণ পরিবর্তন Configuration বদলে Apply দিয়ে করুন। State Command শুধু তখনই, যখন Configuration দিয়ে সমস্যার সমাধান হয় না — যেমন নাম বদল, Import, বা ভাঙা State মেরামত।

১০.৯ Local থেকে Remote-এ Migration — নিরাপদ ধাপ

```
# ধাপ ১: Local State-এর Backup নিন
$ cp terraform.tfstate terraform.tfstate.backup

# ধাপ ২: Backend Block যোগ করুন (main.tf / backend.tf-এ)
# ধাপ ৩: Migrate করুন — Terraform জিজ্ঞেস করবে, "yes" দিন
$ terraform init -migrate-state
Do you want to copy existing state to the new backend?
  Enter a value: yes

# ধাপ ৪: যাচাই করুন
$ terraform state list
$ terraform plan   # কোনো পরিবর্তন দেখানো উচিত নয় (No changes)
```

Migration-এর পর Plan-এ "No changes" দেখা মানে Migration সফল — State ঠিকমতো গেছে, Configuration-এর সঙ্গে মিলছে। পরিবর্তন দেখালে থামুন, State-এর Backup যাচাই করুন, তারপর এগোন।

১০.১০ Refresh-Only Mode

`terraform plan -refresh-only` ও `terraform apply -refresh-only` শুধু State হালনাগাদ করে — কোনো Infrastructure বদলায় না। এটি Drift দেখার নিরাপদ উপায়: বাস্তবে কী বদলেছে, State-এ তা লিখে নিন, কিন্তু কিছু তৈরি বা মুছবেন না। নিয়মিত Refresh-Only চালিয়ে Drift ধরুন, তারপর সিদ্ধান্ত নিন Configuration বদলাবেন না Apply দেবেন।

১০.১১ Backend Security

সুরক্ষা	কী করবেন

Encryption at Rest	S3-এ Encryption চালু করুন (KMS ভালো); GCS/Azure-এ সমতুল্য সেটিং
Encryption in Transit	HTTPS-ই ব্যবহার হয়; HTTP Backend-এ TLS নিশ্চিত করুন
Access Control	Bucket-এ শুধু Terraform চালানো ব্যক্তি/Role-এর Access; Public Access Block করুন
Versioning	চালু রাখুন — ভুল State লেখা হলে আগের Version-এ ফেরা যায়
Git	State ফাইল কখনোই Git-এ নয় — `.gitignore`-এ `*.tfstate*`

WARNING: State-এ Sensitive মান Plain Text-এ থাকে — Password, Key, Token। Backend-এর Encryption ও Access Control-ই সেই মানের একমাত্র সুরক্ষা। যার S3 Bucket পড়ার অধিকার আছে, সে State-এর Sensitive মানও পড়তে পারে। তাই Backend Bucket-এর IAM Policy সবচেয়ে কঠোর রাখুন।

১০.১২ সাধারণ ভুল ও Troubleshooting

TROUBLESHOOTING: `Backend configuration changed` Error হলে `terraform init -reconfigure` বা `-migrate-state` চালান — পরিস্থিতি বুঝে। `Failed to get existing workspaces` মানে Bucket বা Credential-এ সমস্যা — Bucket-এর নাম, Region ও IAM Permission যাচাই করুন। Lock আটকে থাকলে প্রথমে Lock Info-র ব্যক্তিকে খুঁজুন; Force Unlock শেষ উপায়। হাতে State JSON সম্পাদনা করবেন না — ভুল Format-এ পুরো State নষ্ট হতে পারে।

১০.১৩ Production Engineering Practices

প্রতিটি Environment-এর জন্য আলাদা State — `staging/terraform.tfstate`, `production/terraform.tfstate` — আলাদা Key বা Workspace ব্যবহার করুন। Backend Bucket-এর নামে কোম্পানির নাম ও উদ্দেশ্য স্পষ্ট করুন। State-এর নিয়মিত Backup যাচাই করুন — Backup আছে কি না, তা Restore করে দেখা ছাড়া জানা যায় না। CI/CD থেকে Apply দিলে Locking আরও জরুরি — Pipeline-এ একসঙ্গে দুটি Run যেন না চলে।

১০.১৪ অধ্যায়ের সারসংক্ষেপ

Remote Backend দলগত Terraform-এর ভিত্তি — State ভাগ করা জায়গায় থাকে, Locking একসঙ্গে লেখা ঠেকায়। S3 + DynamoDB ক্লাসিক Pattern; azurerm, gcs, Terraform Cloud বিকল্প। State Operation Surgery-র মতো — দরকারে, সাবধানে। Migration-এর আগে Backup, পরে "No changes" যাচাই। আর State-এর সুরক্ষাই Sensitive মানের সুরক্ষা।

১০.১৫ Knowledge Check

১. Remote Backend কেন দরকার? ২. S3 + DynamoDB-তে Lock কীভাবে কাজ করে? ৩. `force-unlock` কখন বিপজ্জনক? ৪. `state mv` ও `state rm`-এর পার্থক্য কী? ৫. Migration-এর পর সাফল্য কীভাবে যাচাই করবেন?

১০.১৬ Practical Assignment

একটি Local Project-এর State Backup নিন। একটি Backend Configuration লিখুন (S3 + DynamoDB, কাল্পনিক Bucket নাম দিয়ে — আসল AWS ছাড়াই)। `terraform init -migrate-state`-এর ধাপগুলো লিখে রাখুন। `state list`, `state show`, `state pull` চালিয়ে Output সংরক্ষণ করুন। একটি Resource-এর নাম `state mv` দিয়ে বদলে দেখুন Plan-এ কী হয়। Lock Error-এর একটি নমুনা Message লিখে তার প্রতিটি ক্ষেত্রের অর্থ ব্যাখ্যা করুন।

অধ্যায় ১১ • PART II — Terraform Core Engineering

Chapter 11 — Terraform Plan, Apply, Destroy, Refresh, and Lifecycle Rules

১১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন — Plan কী গণনা করে তা ব্যাখ্যা করতে ও Plan File ব্যবহার করতে; Apply-এর Execution Order ও Parallelism বুঝতে; Destroy-এর আগে Plan করে নিরাপদে ধ্বংস করতে; Refresh ও Refresh-Only Mode-এর পার্থক্য বলতে; Lifecycle Rule — `create_before_destroy`, `prevent_destroy`, `ignore_changes`, `replace_triggered_by`, `precondition` / `postcondition` — ব্যবহার করতে; এবং Resource Rename/Move/Remove-এর সময় `moved` ও `removed` Block ব্যবহার করতে।

১১.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

Plan, Apply, Destroy — এই তিনটি Command-ই Terraform-এর দৈনন্দিন কাজ। কিন্তু এগুলো না বুঝে চালালে Production মুছে যেতে পারে এক Command-এ। TechNova-তে একজন Engineer একবার `terraform destroy` চালিয়েছিলেন ভুল Directory থেকে — Staging-এর বদলে Production-এর State ছিল সেখানে। এই অধ্যায়ের উদ্দেশ্য: প্রতিটি Command কী করে, কী ঝুঁকি আছে, আর কীভাবে নিরাপদে চালাতে হয় — তা হাতে-কলমে শেখানো।

> **REAL WORLD:** TechNova-র Production Database-এ `prevent_destroy` না থাকায় একবার একটি ভুল Variable-এর কারণে Plan-এ Database "ধ্বংস করে আবার তৈরি" দেখাচ্ছিল। ভাগ্য ভালো, Engineer Plan পড়েছিলেন। Plan না পড়ে Apply দিলে ডেটা হারিয়ে যেত। শিক্ষা: Plan পড়া ঐচ্ছিক নয় — এটি Apply-এর অনুমোদন।

১১.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ৯-এর State Lifecycle ও অধ্যায় ১০-এর Backend/Locking জানা থাকতে হবে। Dependency Graph-এর ধারণা (অধ্যায় ৮) এখানে কাজে লাগবে।

১১.৪ Plan — কী গণনা করা হয়

Terraform Plan হলো একটি পূর্বাভাস — Apply দিলে কী হবে, তার তালিকা। Plan-এর সময় Terraform: Configuration পড়ে, State পড়ে, বাস্তব Infrastructure Refresh করে, তারপর তিনটি তুলনা করে সিদ্ধান্ত নেয় — কোন Resource তৈরি হবে (`+`), কোনটি বদলাবে (`~`), কোনটি মুছবে (`-`), কোনটি ধ্বংস করে আবার তৈরি হবে (`-/+`)। Plan কিছু বদলায় না — এটি Read-Only (State Refresh ছাড়া)।

```
$ terraform plan
local_file.config: Refreshing state... [id=9f86d081884c7d659a2feaa0c55ad015]

Terraform will perform the following actions:

  # local_file.app will be created
  + resource "local_file" "app" {
```

```
      + content  = "TechNova app"
      + filename = "./app.txt"
      + id       = (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.
```

লক্ষ করুন: `(known after apply)` মানে মানটি Apply-এর পরেই জানা যাবে — যেমন `id`। Plan-এর প্রতীকগুলো পড়তে শিখুন — এগুলোই আপনার নিরাপত্তা।

> **CONCEPT:** Plan হলো Terraform-এর "চুক্তি"। আপনি Plan পড়ে অনুমোদন দেন, তারপর Apply সেই চুক্তি কার্যকর করে। Plan না পড়ে Apply দেওয়া মানে না পড়ে চুক্তিতে সই করা।

১১.৪.১ Plan File ও -out

`terraform plan -out=tfplan` দিয়ে Plan একটি ফাইলে সংরক্ষণ করা যায়, আর `terraform apply tfplan` দিয়ে ঠিক সেই Plan-ই কার্যকর করা যায় — মাঝখানে Configuration বদলে গেলেও Apply সেই সংরক্ষিত Plan-ই চালাবে। CI/CD-তে এটিই নিরাপদ পদ্ধতি: Plan ধাপে Plan তৈরি ও পর্যালোচনা, Apply ধাপে সেই Plan-ই চালানো। `terraform show tfplan` দিয়ে সংরক্ষিত Plan মানুষের পড়ার মতো করে দেখা যায়, `terraform show -json tfplan` দিয়ে Machine-এ পড়ার মতো JSON পাওয়া যায় — Policy Check-এর জন্য এটি ব্যবহৃত হয়।

> **WARNING:** `-target` ব্যবহার করে শুধু একটি Resource-এ Apply দেওয়া বিপজ্জনক অভ্যাস। এতে Dependency-র বাইরের Resource অসম্পূর্ণ থাকে, State ও Configuration-এর মধ্যে অসঙ্গতি তৈরি হয়। শুধু জরুরি মেরামতের সময়, অস্থায়ীভাবে ব্যবহার করুন — আর তারপর পূর্ণ Plan দিয়ে যাচাই করুন।

১১.৫ Apply — কার্যকর করা

Terraform Apply Plan-এর পরিবর্তনগুলো কার্যকর করে। Apply-এর সময় Terraform Dependency Graph-এর Topological Order মেনে চলে — নির্ভরশীল Resource পরে, স্বাধীন Resource সমান্তরালে। Default Parallelism ১০ — অর্থাৎ একসঙ্গে সর্বোচ্চ ১০টি Resource তৈরি হতে পারে; `-parallelism` দিয়ে বদলানো যায়। Apply-এর আগে Terraform আবার Plan দেখায় ও অনুমোদন চায় — `-auto-approve` দিলে অনুমোদন ছাড়াই চলে, যা CI/CD-তে ব্যবহৃত হয়, কিন্তু হাতে চালানোর সময় বিপজ্জনক।

```
$ terraform apply
Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

Apply মাঝপথে ব্যর্থ হলে State আংশিক হালনাগাদ থাকে — যা হয়েছে তা State-এ আছে। আবার Apply দিলে Terraform বাকিটা শেষ করে। এজন্যই Apply Idempotent — একই Configuration-এ বারবার Apply দিলে দ্বিতীয়বার "No changes" দেখায়।

১১.৬ Destroy — ধ্বংস করা

Terraform Destroy Configuration-এর সব Resource মুছে ফেলে। এটি সবচেয়ে ধ্বংসাত্মক Command। নিরাপদ নিয়ম: প্রথমে `terraform plan -destroy` চালিয়ে দেখুন কী মুছবে, তারপর Destroy দিন। কখনোই Production-এ বিনা যাচাইয়ে Destroy চালাবেন না।

```
$ terraform plan -destroy
Plan: 0 to add, 0 to change, 2 to destroy.

$ terraform destroy
Do you really want to destroy all resources?
  Terraform will destroy all resources managed by this configuration.
  Only 'yes' will be accepted to confirm.

  Enter a value: yes
Destroy complete! Resources: 2 destroyed.
```

WARNING: Destroy State-এর সব Resource মুছে ফেলে, কিন্তু State ফাইলটি থেকে যায় (খালি অবস্থায়)। ভুল Directory থেকে Destroy চালানো সবচেয়ে সাধারণ বিপর্যয় — Apply/Destroy-এর আগে সবসময় `pwd` ও `terraform workspace show` দিয়ে যাচাই করুন আপনি সঠিক Environment-এ আছেন কি না।

১১.৭ Refresh ও Refresh-Only Mode

Refresh মানে বাস্তব Infrastructure পড়ে State হালনাগাদ করা। পুরোনো Terraform-এ `terraform refresh` Command ছিল; Terraform 1.9+-এ এটি `terraform apply -refresh-only` ও `terraform plan -refresh-only`-এর মাধ্যমে করা হয়। Refresh-Only কোনো Infrastructure বদলায় না — শুধু State হালনাগাদ করে। এটি Drift দেখার নিরাপদ উপায়। মনে রাখুন: সাধারণ Plan-ও Refresh করে (যদি না `-refresh=false` দেন) — তাই Plan-এর State সবসময় হালনাগাদ থাকে।

১১.৮ Lifecycle Rule — আচরণ নিয়ন্ত্রণ

`lifecycle` Block Resource-এর তৈরি/ধ্বংসের আচরণ নিয়ন্ত্রণ করে:

Rule	কাজ	উদাহরণ
`create_before_destroy`	পুরোনোটি মুছার আগে নতুনটি তৈরি করে	Zero-Downtime Replacement
`prevent_destroy`	Destroy/Replace আটকে দেয় — Error হয়	Production Database
`ignore_changes`	কিছু Attribute-এর বাইরের পরিবর্তন উপেক্ষা করে	বাইরে থেকে বদলানো Tag
`replace_triggered_by`	অন্য Resource বদলালে এই Resource Replace করে	Config বদলালে Server Replace
`precondition`	শর্ত না মিললে Plan/Apply আটকায়	পরিবেশ যাচাই
`postcondition`	Apply-এর পর শর্ত যাচাই করে	তৈরির পর যাচাই

```
resource "local_file" "critical" {
  filename = "${path.module}/critical.txt"
  content  = "TechNova critical config"

  lifecycle {
    prevent_destroy = true
  }
}

resource "local_file" "app" {
  filename = "${path.module}/app.txt"
  content  = "App v2"

  lifecycle {
    create_before_destroy = true
    ignore_changes        = [file_permission]
    replace_triggered_by  = [local_file.config]
  }
}

# precondition / postcondition উদাহরণ
variable "environment" {
  type = string
  validation {
    condition     = contains(["staging", "production"], var.environment)
    error_message = "Must be staging or production."
  }
}

resource "local_file" "checked" {
  filename = "${path.module}/checked.txt"
  content  = "Env: ${var.environment}"

  lifecycle {
    precondition {
      condition     = var.environment != ""
      error_message = "Environment must not be empty."
    }
    postcondition {
      condition     = self.content != ""
      error_message = "Content must not be empty after apply."
    }
  }
}
```

`prevent_destroy = true` থাকলে Terraform Destroy করতে গেলে Error দেয় — ইচ্ছাকৃতভাবে মুছতে হলে প্রথমে এই Rule সরাতে হয়। এটি Production Database-এর মতো অমূল্য Resource-এর শেষ প্রতিরক্ষা। `ignore_changes` সাবধানে ব্যবহার করুন — এটি Drift লুকিয়ে রাখে; শুধু সেই Attribute-এই ব্যবহার করুন যা বাইরের সিস্টেম বদলায়। `self` মানে Resource-টি নিজে — Postcondition-এ এর Attribute যাচাই করা যায়।

১১.৯ Rename, Move, Remove — moved ও removed Block

Resource-এর নাম বদলালে Terraform মনে করে পুরোনোটি মুছে নতুনটি তৈরি হবে — এটি এড়াতে `moved` Block ব্যবহার করুন:

```
moved {
  from = local_file.old_name
  to   = local_file.new_name
}

removed {
  from = local_file.legacy
  lifecycle {
    destroy = false
  }
}
```

`moved` Block State-এ Resource-এর Address বদলে দেয় — কোনো ধ্বংস বা পুনর্নির্মাণ ছাড়াই। `removed` Block Configuration থেকে Resource সরানোর সময় ব্যবহার করা হয় — `destroy = false` দিলে বাস্তব Resource থেকে যাবে, শুধু Terraform-এর পরিচালনা থেকে সরবে; `destroy = true` (Default) দিলে বাস্তব Resource-ও মুছে যাবে। Terraform 1.9+-এ এই Block-গুলোই Rename/Remove-এর প্রমিত পদ্ধতি — হাতে `state mv` চালানোর চেয়ে Configuration-এ লেখা ভালো, কারণ এটি Version Control-এ থাকে ও পর্যালোচনা করা যায়।

নাম বদল (কোনো ধ্বংস ছাড়াই):
`local_file.old_name` ── `moved` ──► `local_file.new_name`

সরানো (`Configuration` থেকে):
`local_file.legacy` ── `removed` ──► `State` থেকে সরানো
`destroy = false` → বাস্তব ফাইল থেকে যায়
`destroy = true` → বাস্তব ফাইলও মুছে যায়

১১.১০ Sensitive মান ও Plan Output

Plan Output-এ Sensitive মান লুকানো থাকে (`(sensitive value)`) — কিন্তু Plan File (`-out`) ও State-এ মানটি থাকে। Plan File কাউকে পাঠানোর আগে ভাবুন — এতে Sensitive মান থাকতে পারে। CI Log-এ Plan Output সংরক্ষণ করার সময় Sensitive মান ফাঁস হতে পারে কি না যাচাই করুন।

TROUBLESHOOTING: Plan-এ অপ্রত্যাশিত Replace (`-/+`) দেখলে কারণ খুঁজুন — কোন Argument বদলেছে? অনেক Argument-এর পরিবর্তনে Resource Replace হয় (ForceNew)। `terraform plan` -এর বিস্তারিত Output-এ "forces replacement" লেখা থাকে। Apply ব্যর্থ হলে Error-এর শেষ লাইনগুলো পড়ুন — কোন Resource, কোন ধাপে ব্যর্থ হয়েছে, তা সেখানে থাকে।

১১.১১ Production Engineering Practices

Apply-এর আগে সবসময় Plan পড়ুন — Production-এ `-auto-approve` ছাড়া Apply দিন, বা CI-তে Plan পর্যালোচনার ধাপ রাখুন। Plan File সংরক্ষণ করে সেই Plan-ই Apply দিন। `prevent_destroy` দিন Database ও Storage-এর মতো অমূল্য Resource-এ। Destroy-এর আগে `plan -destroy` চালান। Rename-এর সময় `moved` Block ব্যবহার করুন। আর সবচেয়ে গুরুত্বপূর্ণ: Production-এ Apply দেওয়ার আগে Workspace ও Directory যাচাই করুন।

১১.১২ অধ্যায়ের সারসংক্ষেপ

Plan পূর্বাভাস দেয়, Apply কার্যকর করে, Destroy মুছে ফেলে — আর তিনটিই State-এর উপর দাঁড়িয়ে। Plan File সংরক্ষণ করে সেই Plan-ই Apply দেওয়া CI/CD-র নিরাপদ পদ্ধতি। Lifecycle Rule — বিশেষ করে `prevent_destroy` — Production-এর শেষ প্রতিরক্ষা। Rename ও Remove-এর জন্য `moved` ও `removed` Block ব্যবহার করুন, হাতে State সম্পাদনা নয়।

১১.১৩ Knowledge Check

১. Plan-এর `+`, `~`, `-`, `-/+` প্রতীকের অর্থ কী? ২. Plan File কেন সংরক্ষণ করবেন? ৩. `-target` কেন বিপজ্জনক? ৪. `prevent_destroy` ও `create_before_destroy`-এর পার্থক্য কী? ৫. `moved` Block কী করে?

১১.১৪ Practical Assignment

TechNova-র Local উদাহরণে: (১) `plan -out=tfplan` দিয়ে Plan সংরক্ষণ করে `apply tfplan` চালান; (২) একটি Argument বদলে Plan-এ `~` দেখুন; (৩) একটি Resource-এ `prevent_destroy = true` দিয়ে Destroy-এর চেষ্টা করে Error-টি লিখুন; (৪) একটি Resource-এর নাম বদলে `moved` Block দিয়ে Plan-এ "No changes" (শুধু Move) দেখান; (৫) `plan -destroy` চালিয়ে Destroy-এর তালিকা পড়ুন — কিন্তু আসল Destroy দেওয়ার আগে দুবার যাচাই করুন আপনি সঠিক Directory-তে আছেন।

PART III • DEVOPS ENGINEERING MASTERY

Advanced Terraform Engineering

Meta-Argument (count, for_each, depends_on, lifecycle), Dynamic Block, Function, Advanced HCL, Module Design & Composition, Module Versioning, Registry, Import & Migration.

অধ্যায় ১২ • PART III — Advanced Terraform Engineering

Chapter 12 — Meta-Arguments — count, for_each, depends_on, and lifecycle

১২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি `count`, `for_each`, `depends_on` এবং `lifecycle` — এই চারটি **Meta-Argument** (মেটা-আর্গুমেন্ট, অর্থাৎ resource বা module-এর আচরণ নিয়ন্ত্রণকারী বিশেষ আর্গুমেন্ট) গভীরভাবে বুঝবেন। একই resource-এর একাধিক কপি তৈরি করা, শর্তসাপেক্ষে resource তৈরি বা না-করা, resource-গুলোর মধ্যে নির্ভরতা (dependency) ঘোষণা করা এবং resource-এর জীবনচক্র (lifecycle) নিয়ন্ত্রণ করা শিখবেন। TechNova-র একাধিক subnet ও server তৈরির বাস্তব উদাহরণ দিয়ে প্রতিটি ধারণা হাতে-কলমে প্রয়োগ করবেন, এবং জানবেন কখন `count` ব্যবহার করবেন আর কখন `for_each` নিরাপদ।

১২.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

বাস্তব infrastructure-এ কোনো resource-ই একা থাকে না। TechNova-র production VPC-তে তিনটি availability zone-এ তিনটি subnet, প্রতিটিতে দুটি করে application server, একটি করে NAT gateway — সব মিলিয়ে একই ধরনের resource বারবার দরকার হয়। প্রতিটি কপি আলাদা resource block লিখে কপি-পেস্ট করলে configuration বিশাল, ভুলপ্রবণ ও রক্ষণাবেক্ষণ-অযোগ্য হয়ে যায়। Meta-argument সেই পুনরাবৃত্তিকে একটি ঘোষণায় পরিণত করে। আবার, database তৈরি হওয়ার আগেই application server চালু হলে অ্যাপ crash করবে; পুরোনো server মুছে ফেলার আগে নতুন server তৈরি না হলে downtime হবে। এই ordering ও safety নিয়ন্ত্রণই meta-argument-এর কাজ।

> **REAL WORLD:** TechNova-র staging-এ একবার তিনটি subnet হাতে লেখা হয়েছিল। পরে দ্বিতীয় subnet মাঝখান থেকে সরানো হলে `count`-ভিত্তিক server-গুলোর index বদলে গিয়ে Terraform ভুল server মুছে নতুন server তৈরি করতে চেয়েছিল — প্রায় একটি staging outage। `for_each`-এ নামভিত্তিক পরিচয় থাকায় এমন দুর্ঘটনা ঘটে না।

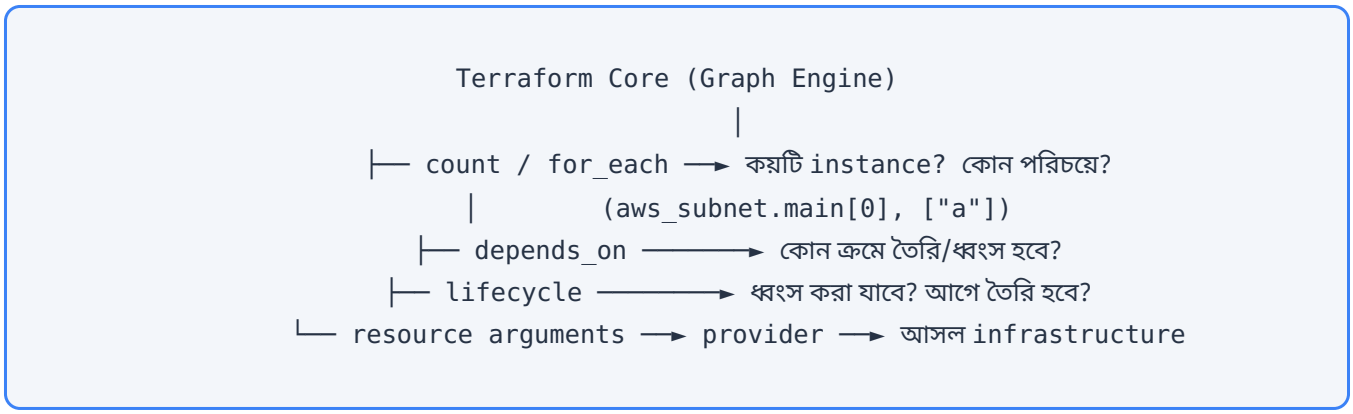
১২.৩ পূর্বশর্ত জ্ঞান

ধরে নেওয়া হচ্ছে আপনি Volume 08-এর Part I ও II থেকে resource block, variable, output, `terraform plan`/`apply`, state এবং provider ধারণা জানেন। List, map ও set-এর পার্থক্য, HCL-এর মৌলিক expression এবং TechNova-র VPC ধারণা জানা থাকলে এই অধ্যায় সহজ হবে।

১২.৪ Meta-Argument কী — প্রথম নীতি থেকে

Terraform-এ প্রতিটি resource block-এর দুই ধরনের আর্গুমেন্ট থাকে। প্রথমটি **resource argument** — যেমন `ami`, `instance_type`, `cidr_block` — যা provider-কে বলে resource-টি কেমন হবে। দ্বিতীয়টি **meta-argument** — যা Terraform core-কে বলে resource-টি কীভাবে পরিচালিত হবে: কয়টি কপি হবে, কিসের ওপর নির্ভর করবে, কখন ধ্বংস করা যাবে। Meta-argument provider-এর কাছে যায় না; এটি Terraform-এর নিজের ভাষা (HCL) ও graph engine-এর নির্দেশ।

> **CONCEPT:** Resource argument বলে *what* (কী তৈরি হবে), meta-argument বলে *how many*, *in what order*, *and with what safety* (কয়টি, কোন ক্রমে, কী নিরাপত্তায়)। Plan-এ অপ্রত্যাশিত destroy দেখলে প্রথমে meta-argument-গুলোই পরীক্ষা করুন।



১২.৫ count — সংখ্যা দিয়ে একাধিক কপি

`count` একটি পূর্ণসংখ্যা। `count = 3` মানে resource-এর তিনটি instance তৈরি হবে, ঠিকানা `aws_subnet.main[0]`, `[1]`, `[2]`। প্রতিটি instance-এর ভেতরে `count.index` দিয়ে বর্তমান সংখ্যাটি (0 থেকে শুরু) পাওয়া যায়।

```
resource "aws_subnet" "app" {
  count             = 3
  vpc_id            = aws_vpc.main.id
  cidr_block        = cidrsubnet(aws_vpc.main.cidr_block, 8, count.index)
  availability_zone = var.availability_zones[count.index]
  tags = { Name = "technova-app-subnet-${count.index + 1}" }
}
```

১২.৫.১ শর্তসাপেক্ষ তৈরি — count = 0/1

```
variable "enable_nat_gateway" { type = bool  default = true }
resource "aws_nat_gateway" "main" {
  count         = var.enable_nat_gateway ? 1 : 0
  allocation_id = aws_eip.nat[0].id
  subnet_id     = aws_subnet.public[0].id
}
```

`count = 0` হলে কোনো instance তৈরি হয় না। কিন্তু সতর্কতা: `aws_nat_gateway.main[0]` অন্য কোথাও refer করলে এবং count 0 হলে plan fail করবে। তাই শর্তসাপেক্ষ resource-কে refer করার সময় `length()` বা `try()` দিয়ে নিরাপদে লিখুন (অধ্যায় ১৩)।

১২.৫.২ count-এর বড় বিপদ — তালিকার ক্রম বদলে যাওয়া

`count`-এ instance-এর পরিচয় হলো তার **index**। তালিকা থেকে মাঝের উপাদান সরালে পরের সব index এক ঘর সরে যায়। Terraform তখন মনে করে পুরোনো `[1]` মুছে গেছে আর নতুন `[1]` তৈরি করতে হবে — অপ্রয়োজনীয় destroy ও recreate। Stateful resource (database, disk) হলে এটি ডেটা হারানোর ঝুঁকি। তাই নাম দিয়ে চেনা যায় এমন জিনিসের জন্য `count` নয়, `for_each` ব্যবহার করুন।

> **WARNING:** `count`-ভিত্তিক resource-এর list থেকে মাঝের item সরানোর আগে সবসময় `terraform plan` দেখুন। Plan-এ destroy দেখা গেলে বুঝবেন index shift হচ্ছে।

১২.৬ for_each — নামভিত্তিক পরিচয়, নিরাপদ পুনরাবৃত্তি

`for_each` একটি map বা set of strings নেয়। প্রতিটি key একটি instance-এর স্থায়ী পরিচয় হয়: `aws_subnet.app["az-a"]`। Map থেকে key সরালে শুধু সেই instance-টি মুছবে; বাকিগুলো অপরিবর্তিত থাকবে। প্রতিটি instance-এর ভেতরে `each.key` ও `each.value` পাওয়া যায়।

```
variable "subnets" {
  type = map(object({ cidr = string, az = string }))
  default = {
    "app-a" = { cidr = "10.0.1.0/24", az = "ap-south-1a" }
    "app-b" = { cidr = "10.0.2.0/24", az = "ap-south-1b" }
    "app-c" = { cidr = "10.0.3.0/24", az = "ap-south-1c" }
  }
}
resource "aws_subnet" "app" {
  for_each          = var.subnets
  vpc_id            = aws_vpc.main.id
  cidr_block        = each.value.cidr
  availability_zone = each.value.az
  tags = { Name = "technova-${each.key}" }
}
```

`for_each`-এ list সরাসরি দেওয়া যায় না — `toset()` দিয়ে set বানাতে হয়। Map of objects সবচেয়ে প্রকাশভঙ্গিসম্পন্ন পদ্ধতি। `count` ও `for_each` একই block-এ একসঙ্গে ব্যবহার করা যায় না।

১২.৬.১ count বনাম for_each — তুলনা ও সিদ্ধান্ত-নির্দেশিকা

বিষয়	count	for_each
পরিচয় (identity)	সংখ্যা: [0], [1], [2]	নাম/key: ["app-a"]
মাঝের item সরালে	পরের সব instance recreate হতে পারে	শুধু সেই instance মুছবে
ইনপুট	সংখ্যা (number)	map বা set of strings
শর্তসাপেক্ষ তৈরি	খুব সহজ: 0 বা 1	খালি map `{}`
উপযুক্ত যখন	কপিগুলো অভিন্ন, বিনিময়যোগ্য	প্রতিটি কপির আলাদা নাম/বৈশিষ্ট্য আছে

সিদ্ধান্ত-নির্দেশিকা: প্রতিটি instance-এর আলাদা নাম থাকলে `for_each`; শুধু সংখ্যা বাড়ানো-কমানোই উদ্দেশ্য হলে `count`; শর্তসাপেক্ষ একক resource-এর জন্য `count = condition ? 1 : 0`। সন্দেহ হলে `for_each` বেছে নিন।

১২.৭ depends_on — স্পষ্ট নির্ভরতা

Terraform সাধারণত resource argument-এর reference দেখে নিজে থেকেই নির্ভরতা বোঝে — একে বলে **implicit dependency**। যেমন `subnet_id = aws_subnet.app.id` লিখলেই Terraform জানে subnet আগে তৈরি করতে হবে।

`depends_on` দরকার হয় যখন নির্ভরতা reference-এ প্রকাশ পায় না: যেমন একটি script, IAM policy, অথবা module-এর ভেতরের hidden dependency।

```
resource "aws_instance" "app" {
  ami           = var.ami_id
  instance_type = "t3.micro"
  depends_on = [aws_db_instance.main]  # কোনো argument-এ DB-এর reference নেই, তবু DB আগে চাই
}
```

অতিরিক্ত ব্যবহারের বিপদ: অপ্রয়োজনীয় `depends_on` graph-কে অযথা serial করে দেয় — parallel তৈরি কমে গিয়ে apply ধীর হয়। নিয়ম: প্রথমে reference দিয়ে নির্ভরতা প্রকাশ করুন; কেবল reference দিয়ে প্রকাশ করা অসম্ভব হলে `depends_on` ব্যবহার করুন, এবং কেন দরকার তা comment-এ লিখুন।

১২.৮ lifecycle — জীবনচক্র নিয়ন্ত্রণ

১২.৮.১ create_before_destroy

সাধারণত replace করতে হলে Terraform আগে পুরোনোটি মুছে তারপর নতুনটি তৈরি করে — downtime হয়। `create_before_destroy = true` দিলে আগে নতুনটি তৈরি হবে, তারপর পুরোনোটি মুছবে। তবে unique নামের resource-এ (যেমন S3 bucket) এটি কাজ করে না — নামে random suffix বা `name_prefix` লাগে।

১২.৮.২ prevent_destroy

`prevent_destroy = true` থাকলে Terraform ওই resource মুছতে অস্বীকার করবে — ভুল করে `terraform destroy` করলেও plan fail করবে। Production database, state bucket, KMS key-এ এটি প্রায় বাধ্যতামূলক। এটি শুধু Terraform-এর নিরাপত্তা; AWS console থেকে মুছলে তা ঠেকাবে না।

১২.৮.৩ ignore_changes — শক্তিশালী কিন্তু বিপজ্জনক

```
resource "aws_autoscaling_group" "app" {
  desired_capacity = 2
  lifecycle { ignore_changes = [desired_capacity, tags] }
}
```

কিছু attribute বাইরের সিস্টেম বদলায় — যেমন auto-scaling-এর `desired_capacity`। তখন `ignore_changes` বললে Terraform সেই attribute-এর পরিবর্তন উপেক্ষা করবে।

> **WARNING:** `ignore_changes = all` প্রায় কখনোই ব্যবহার করবেন না — তাতে Terraform ওই resource-কে আর পরিচালনাই করে না, drift চিরতরে অদৃশ্য হয়ে যায়। শুধু নির্দিষ্ট attribute তালিকাভুক্ত করুন।

১২.৮.৪ replace_triggered_by ও preconditions/postconditions

`replace_triggered_by` বলে: তালিকার কোনো resource বদলালে এই resource-টিকে replace করো। `precondition` apply-এর আগে, `postcondition` apply-এর পরে শর্ত যাচাই করে; শর্ত ভাঙলে apply থেমে যায়।

```
resource "aws_instance" "app" {
  ami           = var.ami_id
  instance_type = var.instance_type
```

```
  lifecycle {
    precondition {
      condition     = var.instance_type != "t2.micro"
      error_message = "Production-এ t2.micro ব্যবহার নিষিদ্ধ।"
    }
    replace_triggered_by = [aws_launch_template.app.id]
  }
}
```

১২.৯ সম্পূর্ণ উদাহরণ — TechNova: একাধিক subnet ও server

```
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
  tags = { Name = "technova-${var.environment}" }
  lifecycle { prevent_destroy = true }
}
resource "aws_subnet" "app" {
  for_each = {
    a = { cidr = "10.0.1.0/24", az = "ap-south-1a" }
    b = { cidr = "10.0.2.0/24", az = "ap-south-1b" }
  }
  vpc_id            = aws_vpc.main.id
  cidr_block        = each.value.cidr
  availability_zone = each.value.az
  tags = { Name = "technova-app-${each.key}" }
}
resource "aws_eip" "nat" { count = var.enable_nat ? 1 : 0  domain = "vpc" }
resource "aws_nat_gateway" "main" {
  count         = var.enable_nat ? 1 : 0
  allocation_id = aws_eip.nat[0].id
  subnet_id     = aws_subnet.app["a"].id
  depends_on    = [aws_internet_gateway.main]
  lifecycle { create_before_destroy = true }
}
resource "aws_instance" "app" {
  for_each      = aws_subnet.app
  ami           = "ami-CHANGE-ME-0000000000000001"
  instance_type = "t3.micro"
  subnet_id     = each.value.id
  lifecycle { create_before_destroy = true  ignore_changes = [tags] }
}
```

১২.৯.১ ধাপে ধাপে প্রয়োগ ও প্রত্যাশিত আউটপুট

`terraform init`, `terraform plan` ও `terraform apply` চালান। Plan-এ দেখবেন: VPC ১টি, subnet ২টি (`["a"]`, `["b"]`), NAT ১টি, instance ২টি। `terraform state list`-এ ঠিকানাগুলো `aws_subnet.app["a"]` আকারে আসবে। তারপর `enable_nat = false` করে plan করুন: NAT ও EIP destroy হবে, subnet অপরিবর্তিত থাকবে।

```
$ terraform plan
Plan: 6 to add, 0 to change, 0 to destroy.
$ terraform state list
aws_vpc.main
aws_subnet.app["a"]
aws_subnet.app["b"]
```

```
aws_eip.nat[0]
aws_nat_gateway.main[0]
aws_instance.app["a"]
```

১২.১০ সাধারণ ভুল

১. `count` ও `for_each` একই block-এ ব্যবহার করা। **২.** `for_each`-এ list দেওয়া। **৩.** শর্তসাপেক্ষ resource-কে `[0]` দিয়ে refer করা যখন count 0 হতে পারে। **৪.** সব জায়গায় `depends_on` ছড়িয়ে দেওয়া। **৫.** `ignore_changes = all` ব্যবহার করা। **৬.** `create_before_destroy` দিয়ে unique নামের resource replace করার চেষ্টা করা।

১২.১১ Troubleshooting নির্দেশিকা

সমস্যা	কারণ	সমাধান
"Invalid index" / index out of range	count 0 হওয়া resource-কে [0] দিয়ে refer	`length()` পরীক্ষা বা `try()` ব্যবহার করুন
অপ্রত্যাশিত destroy in plan	count list-এর ক্রম বদলেছে	`for_each`-এ migrate করুন (অধ্যায় ১৬)
prevent_destroy-এ plan fail	resource মুছতে চাওয়া হচ্ছে	ইচ্ছাকৃত হলে সাময়িকভাবে false করুন
precondition failed	শর্ত ভাঙা হয়েছে	variable-এর মান ঠিক করুন

TROUBLESHOOTING: Plan-এ destroy দেখলে `terraform plan -out=tfplan` করে `terraform show -json tfplan` দিয়ে কোন resource কেন replace হচ্ছে দেখুন।

১২.১২ নিরাপত্তা বিবেচনা

`prevent_destroy` production ডেটা রক্ষার প্রথম লাইন। `ignore_changes` দিয়ে security group rule বা IAM policy উপেক্ষা করবেন না। `count` / `for_each` দিয়ে তৈরি resource-এর নামে পরিবেশ স্পষ্ট করুন, যাতে staging resource ভুলে production ভেবে মোছা না হয়।

১২.১৩ Production Engineering Practices

Production-এ: (১) stateful resource-এ সবসময় `for_each`; (২) database ও state bucket-এ `prevent_destroy`; (৩) zero-downtime resource-এ `create_before_destroy`; (৪) প্রতিটি `depends_on` ও `ignore_changes`-এর পাশে কারণ লেখা comment; (৫) `precondition` দিয়ে নিষিদ্ধ মান আগেই আটকানো।

১২.১৪ অধ্যায়ের সারাংশ

`count` সংখ্যা দিয়ে কপি তৈরি করে, শর্তসাপেক্ষ তৈরিতে সেরা, কিন্তু index shift-এর ঝুঁকি আছে। `for_each` নামভিত্তিক পরিচয় দেয়, তাই পরিবর্তনে নিরাপদ। `depends_on` শুধু তখনই ব্যবহার করুন যখন reference দিয়ে নির্ভরতা প্রকাশ করা যায় না। `lifecycle` দিয়ে create_before_destroy, prevent_destroy, ignore_changes ও replace_triggered_by নিয়ন্ত্রণ করুন।

১২.১৫ Knowledge Check

১. `count` ও `for_each`-এর পরিচয়ের পার্থক্য কী? ২. মাঝের subnet সরালে কেন `count`-এ recreate হয়? ৩. `depends_on` কখন দরকার, কখন নয়? ৪. `ignore_changes = all` কেন বিপজ্জনক? ৫. `create_before_destroy` কখন কাজ করবে না?

১২.১৬ Practical Assignment

TechNova staging-এর জন্য configuration লিখুন: `for_each` দিয়ে ৩টি subnet, `count` দিয়ে শর্তসাপেক্ষ ১টি NAT gateway, `create_before_destroy` সহ ২টি instance, এবং database-এ `prevent_destroy`। তারপর মাঝের subnet সরিয়ে plan করুন এবং `count` দিয়ে একই কাজ করলে কী হতো তা তুলনা করে একটি ছোট নোট লিখুন।

অধ্যায় ১৩ • PART III — Advanced Terraform Engineering

Chapter 13 — Dynamic Blocks, Functions, Expressions, and Advanced HCL

১৩.১ শেখার উদ্দেশ্য

এই অধ্যায়ে আপনি **Dynamic Block** (ডাইনামিক ব্লক — পুনরাবৃত্ত nested block তৈরি করার কৌশল), Terraform-এর **Function** (ফাংশন — মান transform করার built-in টুল), **Expression** (এক্সপ্রেশন — মান গণনার ভাষা), conditional, `for` expression, splat expression, `templatefile` এবং `try` / `can` / `coalesce`-এর মতো উন্নত expression শিখবেন। শেষে জানবেন কোন logic Terraform-এর ভেতরে রাখা উচিত আর কোনটি বাইরে রাখা উচিত।

১৩.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

একটি security group-এ পাঁচটি port খুলতে হলে পাঁচটি `ingress` block হাতে লেখা যায়, কিন্তু port তালিকা বদলালে আবার হাতে বদলাতে হয়। Dynamic block সেই তালিকা থেকে block তৈরি করে — configuration ডেটা-চালিত (data-driven) হয়। একইভাবে function ছাড়া CIDR গণনা, নাম তৈরি, string জোড়া দেওয়া — সব হাতে করতে হতো। HCL-এর expression ভাষা যত ভালো জানবেন, configuration তত ছোট, নিরাপদ ও পুনঃব্যবহারযোগ্য হবে। কিন্তু অতিরিক্ত জটিল logic Terraform-কে একটি প্রোগ্রামিং ভাষায় পরিণত করে — যা রক্ষণাবেক্ষণের দুঃস্বপ্ন। এই ভারসাম্যই এই অধ্যায়ের মূল শিক্ষা।

> **REAL WORLD:** TechNova-র security group-এ একবার ১২টি ingress rule হাতে লেখা ছিল। একটি port বদলাতে গিয়ে একজন engineer ভুল rule মুছে ফেলেন, আর production API দুই ঘণ্টা বন্ধ থাকে। Dynamic block + একটি port তালিকা থাকলে পরিবর্তনটি হতো এক লাইনে, review-ও সহজ হতো।

১৩.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ১২-এর `for_each`, list/map, variable type এবং `cidrsubnet()`-এর ধারণা জানা থাকতে হবে। Function-এর ধারণা অন্য যেকোনো ভাষা থেকে জানলেই চলবে — Terraform-এর function-গুলো pure: এগুলো শুধু মান নিয়ে মান ফেরত দেয়, বাইরের অবস্থা বদলায় না।

১৩.৪ Dynamic Block — প্রথম নীতি থেকে

Terraform-এ কিছু আর্গুমেন্ট একক মান নয়, **nested block** — যেমন security group-এর `ingress`, resource-এর `tag` নয়, `ebs_block_device`। Nested block-এর সংখ্যা আগে থেকে জানা না থাকলে বা তালিকা থেকে আসলে **dynamic block** ব্যবহার করা হয়। এটি `for_each`-এর মতোই কাজ করে, তবে resource-এর বদলে block তৈরি করে।

> **CONCEPT:** `count` / `for_each` resource-কে পুনরাবৃত্তি করে; dynamic block resource-এর ভেতরের block-কে পুনরাবৃত্তি করে। দুটোর মানসিক মডেল একই — একটি collection থেকে প্রতিটি item-এর জন্য একটি করে জিনিস তৈরি করা — শুধু স্তর (level) আলাদা।

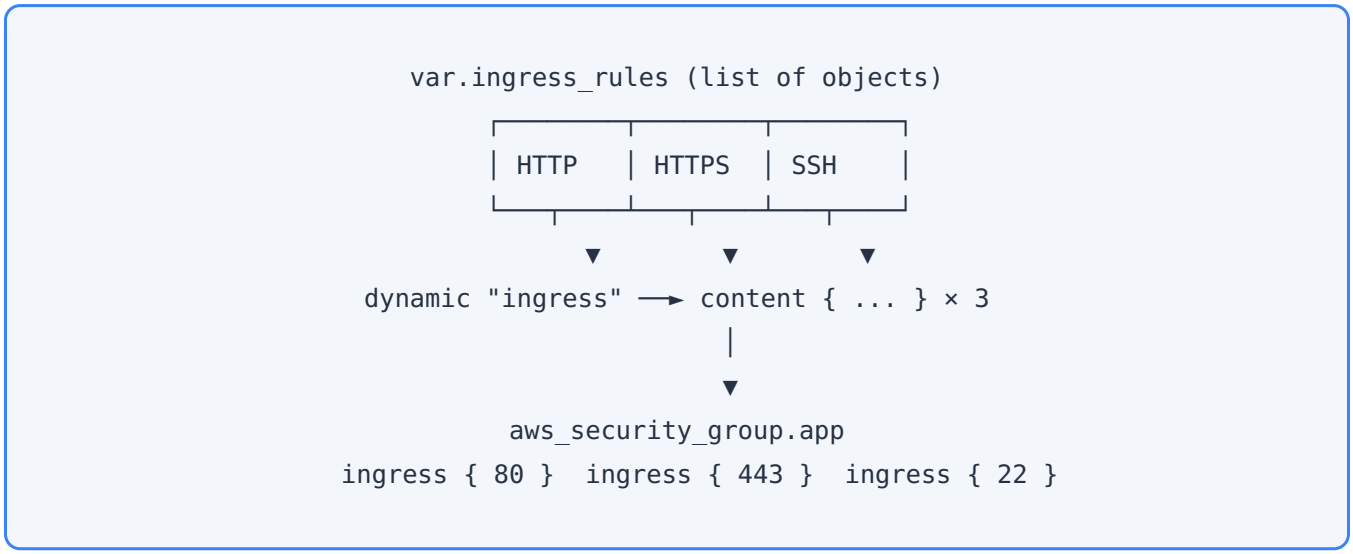
```
variable "ingress_rules" {
  type = list(object({
    port        = number
    protocol    = string
    cidr_blocks = list(string)
    description = string
  }))
  default = [
    { port = 80,  protocol = "tcp", cidr_blocks = ["0.0.0.0/0"], description = "HTTP" },
    { port = 443, protocol = "tcp", cidr_blocks = ["0.0.0.0/0"], description = "HTTPS" },
    { port = 22,  protocol = "tcp", cidr_blocks = ["203.0.113.0/24"], description = "SSH from
office" },
  ]
}

resource "aws_security_group" "app" {
  name   = "technova-app-sg"
  vpc_id = aws_vpc.main.id

  dynamic "ingress" {
    for_each = var.ingress_rules
    content {
      from_port   = ingress.value.port
      to_port     = ingress.value.port
      protocol    = ingress.value.protocol
      cidr_blocks = ingress.value.cidr_blocks
      description = ingress.value.description
    }
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

এখানে `dynamic "ingress"`-এর ভেতরে `ingress.value` হলো বর্তমান rule। নাম বদলাতে চাইলে `iterator` আর্গুমেন্ট ব্যবহার করুন: `iterator = rule` দিলে `rule.value.port` লেখা যায় — যখন nested block-এর নাম ও iterator-এর নাম গুলিয়ে যায় তখন এটি পঠনযোগ্যতা বাড়ায়। Dynamic block-এ `for_each`-এর মতো map-ও দেওয়া যায়; খালি list দিলে কোনো block তৈরি হয় না — শর্তসাপেক্ষ nested block-এর এটাই আদর্শ পদ্ধতি।



১৩.৫ Functions — সর্বাধিক ব্যবহৃত ফাংশনের তালিকা

Terraform-এর function-গুলো শ্রেণিবদ্ধ (categorised)। নিচে TechNova-র কাজে সবচেয়ে বেশি লাগে এমনগুলো দেওয়া হলো — প্রতিটি উদাহরণসহ।

শ্রেণি	Function	কাজ ও উদাহরণ
String	`format()`, `upper()`, `lower()`, `replace()`, `join()`, `split()`	`format("technova-%s-%02d", var.env, 1)` → "technova-staging-01"; `join(",", ["a","b"])` → "a,b"
Numeric	`max()`, `min()`, `ceil()`, `floor()`	`max(2, var.desired)` — কমপক্ষে ২টি instance নিশ্চিত করা
Collection	`length()`, `concat()`, `contains()`, `keys()`, `values()`, `merge()`, `lookup()`, `tolist()`, `toset()`, `tomap()`	`merge(var.common_tags, { Name = "x" })`; `contains(["a","b"], "a")` → true
Encoding	`jsonencode()`, `jsondecode()`, `base64encode()`, `base64decode()`, `urlencode()`	`jsonencode({ Version = "2012-10-17" })` — IAM policy লেখায় অপরিহার্য
Filesystem	`file()`, `fileset()`, `fileexists()`, `templatefile()`, `path.module`	`file("${path.module}/user-data.sh")`
Date/Time	`timestamp()`, `formatdate()`, `timeadd()`	`formatdate("YYYY-MM-DD", timestamp())`
Type conversion	`tostring()`, `tonumber()`, `tobool()`	`tonumber(var.port)`
Network	`cidrhost()`, `cidrsubnet()`, `cidrnetmask()`	`cidrsubnet("10.0.0.0/16", 8, 1)` → "10.0.1.0/24"

মনে রাখবেন: function-এর ভেতরে resource attribute ব্যবহার করা যায়, কিন্তু function-এর ফলাফল `count` বা `for_each`-এ ব্যবহার করলে সেই মান plan-এর সময় জানা থাকতে হবে — নইলে "value depends on resource attributes" error আসবে।

১৩.৫.১ jsonencode ও templatefile — দুটি কাজের উদাহরণ

```
# IAM policy — হাতে JSON লেখার বদলে jsonencode
data "aws_iam_policy_document" "app" {
  statement {
    actions   = ["s3:GetObject"]
    resources = ["arn:aws:s3:::technova-app-bucket/*"]
  }
}
# অথবা সরাসরি:
# policy = jsonencode({ Version = "2012-10-17", Statement = [...] })

# user-data — templatefile দিয়ে variable ঢোকানো
resource "aws_instance" "app" {
  ami           = var.ami_id
  instance_type = "t3.micro"
  user_data = templatefile("${path.module}/user-data.sh.tpl", {
    environment = var.environment
    app_port    = 8080
  })
}
```

`templatefile(path, vars)` একটি ফাইল পড়ে `${...}` জায়গায় vars-এর মান বসায়। `user-data.sh.tpl`-এ লেখা থাকে `APP_ENV=${environment}`। `path.module` হলো বর্তমান module-এর ফোল্ডার — relative path-এর বদলে এটি ব্যবহার করুন, নইলে module অন্য জায়গা থেকে call করলে path ভেঙে যায়।

১৩.৬ Conditionals, for Expressions ও Splat Expressions

১৩.৬.১ Conditional — `condition ? true_val : false_val`

```
instance_type = var.environment == "production" ? "t3.large" : "t3.micro"
name_prefix   = var.environment == "production" ? "prod-" : "stg-"
```

দুই শাখার type একই হতে হবে। গভীর nested ternary এড়িয়ে চলুন — তিন স্তরের বেশি হলে `lookup()` দিয়ে map ব্যবহার করুন, যা পড়তে সহজ।

১৩.৬.২ for expression — collection transform করা

```
locals {
  # list থেকে list: সব subnet-এর CIDR
  subnet_cidrs = [for s in aws_subnet.app : s.cidr_block]
  # list থেকে map: নাম → id
  subnet_ids = { for k, s in aws_subnet.app : k => s.id }
  # শর্তসহ (if): শুধু production subnet
  prod_cidrs = [for s in var.subnets : s.cidr if s.env == "production"]
}
```

`for` expression হলো HCL-এর সবচেয়ে শক্তিশালী টুল — এটি ছাড়া module-এর output গঠন করা প্রায় অসম্ভব। `if` clause দিয়ে filter করা যায়, `...` (grouping) দিয়ে এক key-এ একাধিক value জমানো যায়।

১৩.৬.৩ Splat expression — `.*` ও full splat `[*]`

```
output "instance_ips" {
  value = aws_instance.app[*].private_ip
}
```

`aws_instance.app[*].private_ip` সব instance-এর private IP-এর list দেয়। Legacy splat `aws_instance.app.*.private_ip` এখনো কাজ করে, তবে নতুন কোডে full splat `[*]` ব্যবহার করুন। মনে রাখবেন: splat-এর ফলাফল list-এর ক্রম `count`-এর index অনুযায়ী; `for_each` resource-এ splat-এর বদলে `for` expression ব্যবহার করুন, কারণ map-এর ক্রম অনিশ্চিত।

১৩.৭ try, can ও coalesce — নিরাপদ expression

```
locals {
  # try: প্রথমটি সফল হলে সেটি, না হলে পরেরটি — error না ছড়িয়ে
  nat_ip = try(aws_eip.nat[0].public_ip, "none")
  # can: expression-টি বৈধ কি না — boolean ফেরত দেয়
  has_db = can(aws_db_instance.main.address)
  # coalesce: প্রথম non-empty মানটি নেয়
  region = coalesce(var.region, "ap-south-1")
  # coalescelist / try-এর সমন্বয়ে optional attribute
  db_port = try(var.db.port, 5432)
}
```

`try()` দিয়ে শর্তসাপেক্ষ resource-এর অনুপস্থিত attribute নিরাপদে ধরা যায় (অধ্যায় ১২.৫.২-এর সমস্যার সমাধান)। `can()` validation-এ ও `precondition`-এ কাজে লাগে। তবে সতর্কতা: `try()` দিয়ে আসল ভুল ঢেকে ফেলবেন না — টাইপো করা attribute নাম `try()`-এর আড়ালে চুপচাপ ভুল মান দেবে। Debugging-এর সময় সাময়িকভাবে `try()` সরিয়ে আসল error দেখুন।

> **WARNING:** `timestamp()` প্রতিটি plan-এ বদলায় — একে resource argument-এ দিলে প্রতিটি apply-তে অপ্রয়োজনীয় পরিবর্তন (perpetual diff) হবে। শুধু tag বা output-এ ব্যবহার করুন, resource-এর মূল attribute-এ নয়।

১৩.৮ Logic কোথায় রাখবেন — Terraform বনাম বাইরে

Terraform-এর ভেতরে রাখুন	বাইরে রাখুন (script / CI / প্রোগ্রাম)
নাম, tag, CIDR গণনা, ছোট conditional	জটিল ব্যবসায়িক যুক্তি, API call, loop-এ loop
for expression দিয়ে output গঠন	ডেটা সংগ্রহ, CSV/Excel পার্স করা
validation ও precondition	দীর্ঘ string প্রক্রিয়াকরণ, regex-এর জটিল খেলা

নিয়ম: একটি expression যদি এক লাইনে পড়া না যায়, অথবা ব্যাখ্যা করতে comment লাগে, তবে সেটি `locals`-এ নাম দিয়ে ভাগ করুন। `locals` হলো নামকরণ করা intermediate মান — এতে জটিল গণনা এক জায়গায় থাকে, resource block পরিষ্কার থাকে।

```
locals {
  common_tags = {
    Project     = "TechNova"
    Environment = var.environment
    ManagedBy   = "terraform"
  }
  app_name = format("technova-%s-app", var.environment)
}
```

১৩.৯ সম্পূর্ণ উদাহরণ — TechNova security group ও outputs

```
locals {
  ports = [80, 443, 8080]
}
resource "aws_security_group" "web" {
  name   = "technova-web-sg"
  vpc_id = aws_vpc.main.id
  dynamic "ingress" {
    for_each = local.ports
    content {
      from_port   = ingress.value
      to_port     = ingress.value
      protocol    = "tcp"
      cidr_blocks = ["0.0.0.0/0"]
      description = "Port ${ingress.value}"
    }
  }
  tags = merge(local.common_tags, { Name = "technova-web-sg" })
}
output "open_ports" {
  value = [for p in local.ports : tostring(p)]
}
```

১৩.১০ সাধারণ ভুল

১. Dynamic block-এ `each.value`-এর বদলে ভুল নাম ব্যবহার করা (dynamic block-এর iterator-এর নাম block-এর নামের মতো হয় — `ingress.value`)। **২.** `for_each`-এ unknown মান (resource attribute) ব্যবহার করা। **৩.** `timestamp()` resource-এ দেওয়া। **৪.** Nested ternary-র জট। **৫.** `try()` দিয়ে আসল ভুল ঢাকা। **৬.** Splat দিয়ে `for_each` resource-এর মান নেওয়া।

১৩.১১ Troubleshooting নির্দেশিকা

TROUBLESHOOTING: "Invalid for_each argument" মানে for_each-এর মান plan-এর সময় জানা নেই — সেটি কোনো resource attribute থেকে আসছে কি না দেখুন; জানা মান (variable/local) দিয়ে সাজান। "Unsupported attribute" মানে function বা splat-এর নাম ভুল — `terraform console`-এ expression টাইপ করে পরীক্ষা করুন: `echo 'merge({a=1},{b=2})' | terraform console`।

১৩.১২ নিরাপত্তা বিবেচনা

`file()` দিয়ে secret ফাইল পড়ে resource-এ দেবেন না — secret-এর জন্য variable (sensitive) বা Vault/SSM ব্যবহার করুন। Dynamic ingress-এ `0.0.0.0/0` দিয়ে SSH (22) খুলবেন না; office CIDR (`203.0.113.0/24`) ব্যবহার করুন। `jsonencode`-এ policy লিখলে `terraform plan`-এ policy-টি পড়ে যাচাই করুন।

১৩.১৩ Production Engineering Practices

Function ও expression ছোট রাখুন, জটিল গণনা `locals`-এ নাম দিয়ে রাখুন, dynamic block-এর input variable-এর type স্পষ্ট করুন (`list(object({...}))`), `templatefile`-এর template-গুলো version control-এ রাখুন, এবং প্রতিটি জটিল expression-এর পাশে একটি উদাহরণ-মান comment-এ লিখুন।

১৩.১৪ অধ্যায়ের সারাংশ

Dynamic block nested block-কে ডেটা-চালিত করে; function মান transform করে; conditional, `for` ও splat expression collection-কে গঠন করে; `try` / `can` / `coalesce` expression-কে নিরাপদ করে। কিন্তু HCL একটি configuration ভাষা — জটিল প্রোগ্রামিং logic বাইরে রাখুন, Terraform-কে পড়ার মতো সহজ রাখুন।

১৩.১৫ Knowledge Check

১. Dynamic block ও `for_each`-এর পার্থক্য কী? ২. `iterator` কখন দরকার? ৩. `try()` ও `can()`-এর পার্থক্য কী? ৪. `for_each` resource-এ splat কেন এড়ানো উচিত? ৫. `timestamp()` resource-এ দিলে কী হয়?

১৩.১৬ Practical Assignment

TechNova-র জন্য একটি security group লিখুন যেখানে ingress rule-গুলো variable-এর list থেকে dynamic block দিয়ে আসে, SSH শুধু `203.0.113.0/24` থেকে খোলা, এবং output-এ `for` expression দিয়ে খোলা port-গুলোর তালিকা বের করুন। তারপর একটি port যোগ করে plan করুন — শুধু সেই rule যোগ হচ্ছে কি না যাচাই করুন।

অধ্যায় ১৪ • PART III — Advanced Terraform Engineering

Chapter 14 — Terraform Modules — Reusability, Design, and Module Composition

১৪.১ শেখার উদ্দেশ্য

এই অধ্যায়ে আপনি **Module** (মডিউল — পুনঃব্যবহারযোগ্য Terraform configuration-এর প্যাকেজ) প্রথম নীতি থেকে বুঝবেন: root module বনাম child module, input (variable), output, module source (local path, Git, registry), composition pattern, এবং পুনঃব্যবহারযোগ্য module ডিজাইনের নিয়ম। শেষে TechNova-র networking, compute ও storage-এর জন্য একটি সম্পূর্ণ working module লিখবেন।

১৪.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

একটি কোম্পানিতে দশটি service থাকলে দশবার VPC, subnet, security group লেখা যায় না — প্রতিবারই একই ভুল নতুন করে হবে। Module হলো সেই পুনরাবৃত্তির সমাধান: একবার ভালোভাবে লিখুন, সব পরিবেশে (staging, production) ও সব team-এ পুনঃব্যবহার করুন। ভালো module মানে কম ভুল, দ্রুত delivery, এবং নিরাপত্তা-নিয়ম এক জায়গায় প্রয়োগ করা। খারাপ module মানে লুকানো জটিলতা ও ভাঙা abstraction।

> **REAL WORLD:** TechNova-তে প্রথমে প্রতিটি service নিজের মতো VPC বানাত — একটিতে flow log ছিল, আরেকটিতে ছিল না; একটিতে private subnet ছিল, আরেকটিতে সব public। একটি shared VPC module চালু করার পর সব service একই নিরাপদ baseline পেল, আর audit-এ একটি জায়গাই পরীক্ষা করতে হয়।

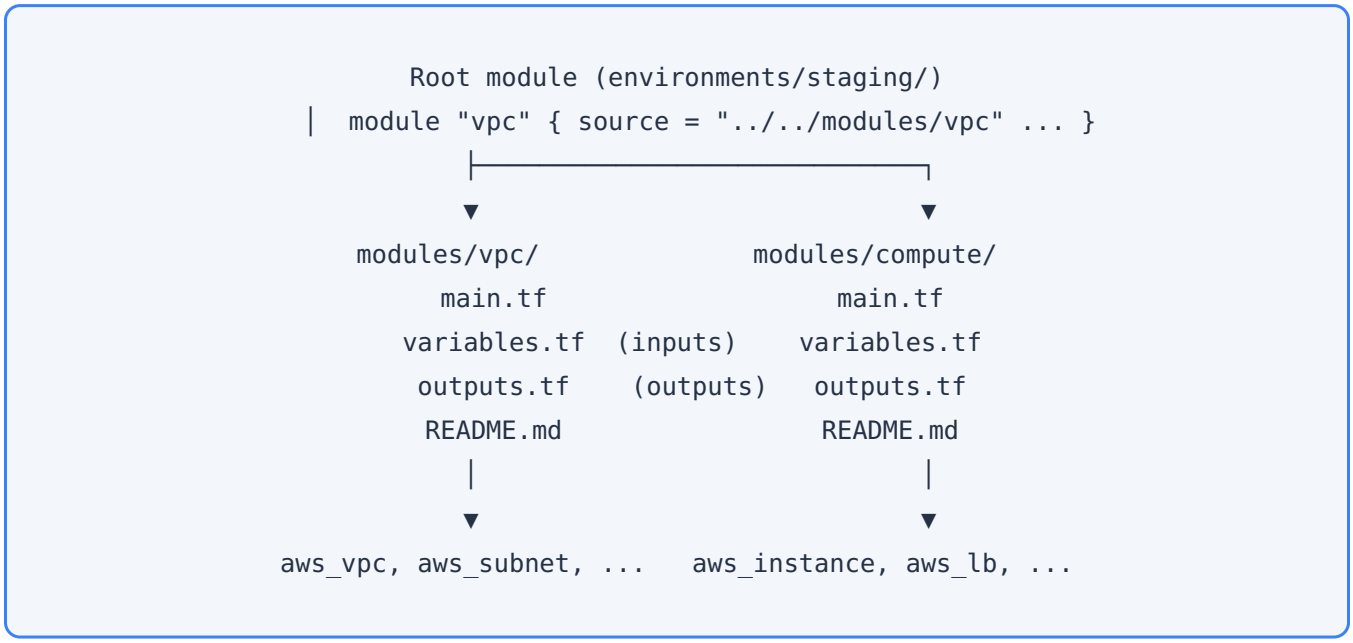
১৪.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ১২–১৩-এর variable, output, `for_each`, `locals` ও function জানা থাকতে হবে। Git-এর মৌলিক ধারণা (repository, tag, branch) জানা থাকলে module source বুঝতে সহজ হবে।

১৪.৪ Module কী — প্রথম নীতি থেকে

সহজ কথায়, **module হলো একটি ফোল্ডার, যেখানে `.tf` ফাইল আছে।** আপনি এতদিন যা লিখেছেন — `main.tf`, `variables.tf` — সেটিই **root module** (রুট মডিউল)। অন্য কোনো ফোল্ডারের `.tf` ফাইলগুলোকে `module` block দিয়ে call করলে সেটি **child module** (চাইল্ড মডিউল) হয়। Child module-এর ভেতরে resource থাকে; root module তাকে input দেয় (variable) আর output নেয়। Module-এর ভেতরের resource-এর ঠিকানা হয় `module.vpc.aws_subnet.app["a"]` — module-এর নাম ঠিকানার অংশ।

> **CONCEPT:** Module হলো Terraform-এর function-এর মতো: input = argument, output = return value, ভেতরের resource = function body। যেমন function-এর ভেতরটা না জেনে call করা যায়, তেমনি ভালো module-এর ভেতরটা না জেনেও ব্যবহার করা যায় — শুধু তার interface (variable + output) জানলেই চলে।



১৪.৫ Module call করা — source-এর তিন রূপ

```
module "vpc" {
  source = "../../modules/vpc"   # local path
  environment  = "staging"
  cidr_block   = "10.0.0.0/16"
  azs          = ["ap-south-1a", "ap-south-1b"]
}
module "vpc_remote" {
  source = "git::https://github.com/technova/terraform-modules.git//modules/vpc?ref=v1.2.0"
}
module "vpc_registry" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "5.0.0"
}
```

Local path একই repository-এর ভেতরে — দ্রুত, কিন্তু অন্য repo থেকে ব্যবহার করা যায় না। **Git source** যেকোনো repo থেকে, `?ref=v1.2.0` দিয়ে tag/branch pin করা যায়। **Registry source** Terraform Registry থেকে, `version` আর্গুমেন্ট দিয়ে pin করা যায় (অধ্যায় ১৫)। Module-এও `count` ও `for_each` ব্যবহার করা যায় — তখন প্রতিটি instance-এর ঠিকানা হয় `module.vpc["staging"]`।

১৪.৬ Inputs, Outputs ও Composition Pattern

Module-এর **input** হলো `variable` — প্রতিটির type, description ও প্রয়োজনে default ও validation থাকা উচিত। **Output** হলো `output` — যা root module বা অন্য module ব্যবহার করবে, যেমন `vpc_id`, `subnet_ids`। Composition-এর দুটি pattern: **flat** (root module সরাসরি ছোট ছোট module call করে — networking, compute, storage আলাদা) ও **nested** (একটি বড় module ভেতরে ছোট module call করে)। TechNova-র জন্য flat pattern ভালো: প্রতিটি concern আলাদা module, root module শুধু সেগুলো জোড়া দেয়। একটি module যেন একটি মাত্র concern সামলায় — VPC module-এ database থাকবে না।

```
# root module — composition: vpc-এর output compute-এর input
module "vpc" {
```

```
  source = "../../modules/vpc"
  environment = var.environment
  cidr_block  = "10.0.0.0/16"
}
module "compute" {
  source     = "../../modules/compute"
  vpc_id     = module.vpc.vpc_id
  subnet_ids = module.vpc.private_subnet_ids
  instance_count = 2
}
```

১৪.৭ পুনঃব্যবহারযোগ্য Module ডিজাইনের নিয়ম

১. Minimal interface: কম variable, স্পষ্ট নাম, প্রতিটির description। **২.** পরিবেশের মান hardcode করবেন না — `"production"` বা নির্দিষ্ট CIDR module-এর ভেতরে লিখবেন না; variable দিয়ে নিন। **৩.** Sensible default দিন, তবে নিরাপদ default (যেমন public access বন্ধ)। **৪.** Output-এ শুধু দরকারি মান দিন। **৫.** Versioning-ready গঠন: `main.tf`, `variables.tf`, `outputs.tf`, `versions.tf`, `README.md`, `examples/`। **৬.** নামকরণ: resource-এর নামে `var.environment` ও module-এর নাম ব্যবহার করুন।

১৪.৮ সম্পূর্ণ উদাহরণ — TechNova VPC Module

```
# modules/vpc/variables.tf
variable "environment" { type = string  description = "পরিবেশের নাম, যেমন staging" }
variable "cidr_block"  { type = string  description = "VPC-এর CIDR" }
variable "azs"         { type = list(string)  description = "Availability zone-এর তালিকা" }

# modules/vpc/main.tf
resource "aws_vpc" "this" {
  cidr_block           = var.cidr_block
  enable_dns_hostnames = true
  tags = { Name = "technova-${var.environment}" }
  lifecycle { prevent_destroy = true }
}
resource "aws_subnet" "private" {
  for_each          = toset(var.azs)
  vpc_id            = aws_vpc.this.id
  cidr_block        = cidrsubnet(var.cidr_block, 8, index(var.azs, each.key))
  availability_zone = each.key
  tags = { Name = "technova-${var.environment}-private-${each.key}" }
}

# modules/vpc/outputs.tf
output "vpc_id"             { value = aws_vpc.this.id }
output "private_subnet_ids" { value = [for s in aws_subnet.private : s.id] }

# modules/vpc/versions.tf
terraform {
  required_version = ">= 1.9, < 2.0"
  required_providers { aws = { source = "hashicorp/aws"  version = ">= 5.0, < 6.0" } }
}

# modules/vpc/README.md
# VPC Module — TechNova
# ব্যবহার: module "vpc" { source = "../../modules/vpc"
```

```
#   environment = "staging"  cidr_block = "10.0.0.0/16"
#   azs = ["ap-south-1a", "ap-south-1b"] }
# Output: vpc_id, private_subnet_ids
```

এই module-টি `terraform init` ও `terraform plan` দিয়ে যাচাই করুন: `examples/staging/`-এ একটি ছোট root configuration লিখে module call করুন। প্রতিটি module-এর সঙ্গে একটি `examples/` ফোল্ডার রাখলে নতুন ব্যবহারকারী কপি-পেস্ট করে শুরু করতে পারে।

> **WARNING:** Module-এর ভেতরে `terraform` backend block লিখবেন না — backend শুধু root module-এ থাকে। Child module-এ provider configuration-ও এড়িয়ে চলুন; provider root থেকে inherit হয়।

১৪.৯ সাধারণ ভুল

১. Module-এ পরিবেশ hardcode করা। **২.** অতিরিক্ত variable — ২০টি variable-এর module কেউ ব্যবহার করতে চায় না। **৩.** Output না দেওয়া — তখন অন্য module resource-এর ভেতরের ঠিকানা জানতে পারে না। **৪.** Module-এর ভেতরে module-এর গভীর nesting (৩ স্তরের বেশি)। **৫.** Registry module না পড়ে ব্যবহার করা — ভেতরে কী তৈরি হচ্ছে জানা জরুরি।

১৪.১০ Troubleshooting নির্দেশিকা

> **TROUBLESHOOTING:** "Module not installed" হলে `terraform init` চালান। Git source-এ "could not download module" হলে `?ref=` tag-টি আছে কি না ও নেটওয়ার্ক/credential যাচাই করুন। Module পরিবর্তনের পর plan-এ কিছু না বদলালে `terraform get -update` দিয়ে module cache refresh করুন।

১৪.১১ নিরাপত্তা বিবেচনা

তৃতীয় পক্ষের module ব্যবহারের আগে তার কোড পড়ুন — module আপনার নামে resource তৈরি করে, IAM policy দিতে পারে, এমনকি public access খুলে দিতে পারে। Registry-তে star/download সংখ্যা নিরাপত্তার প্রমাণ নয়। নিজের module-এ নিরাপদ default রাখুন: encryption on, public access off।

১৪.১২ Production Engineering Practices

প্রতিটি module-এ README, example ও version pin রাখুন; module-এর পরিবর্তনে semantic version বাড়ান (অধ্যায় ১৫); module-এর input validation-এ `validation` block ব্যবহার করুন; `terraform fmt` ও `terraform validate` CI-তে চালান।

১৪.১৩ অধ্যায়ের সারাংশ

Module হলো `.tf` ফাইলের ফোল্ডার — root module তা call করে, input দেয় variable দিয়ে, output নেয়। Local, Git ও Registry — তিন source-ই আছে। ভালো module-এর interface ছোট, পরিবেশ hardcode করা থাকে না, এবং একটি মাত্র concern সামলায়। Flat composition দিয়ে networking, compute ও storage জোড়া দিন।

১৪.১৪ Knowledge Check

১. Root module ও child module-এর পার্থক্য কী? ২. Module-এর output কেন দরকার? ৩. Flat বনাম nested composition কখন কোনটি? ৪. Module-এ backend কেন লেখা যায় না? ৫. তৃতীয় পক্ষের module ব্যবহারের আগে কী যাচাই করবেন?

১৪.১৫ Practical Assignment

উপরের VPC module-টি লিখে `examples/staging/` থেকে call করুন। তারপর একটি `compute` module লিখুন যা `vpc_id` ও `subnet_ids` input নেয় এবং ২টি instance তৈরি করে। Root module-এ দুটি module জোড়া দিয়ে plan করুন।

অধ্যায় ১৫ • PART III — Advanced Terraform Engineering

Chapter 15 — Module Versioning, Registry, and Dependency Management

১৫.১ শেখার উদ্দেশ্য

এই অধ্যায়ে আপনি **Semantic Versioning** (সেমান্টিক ভার্সনিং — MAJOR.MINOR.PATCH নিয়ম), Git tag দিয়ে module version করা, **Version Constraint** (ভার্সন সীমা — কোন version ব্যবহার করা যাবে), `.terraform.lock.hcl` lock file, Terraform Registry (public বনাম private), module প্রকাশ (publishing), breaking change ও migration, এবং monorepo বনাম multi-repo কৌশল শিখবেন।

১৫.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

গতকাল কাজ করা configuration আজ হঠাৎ ভেঙে গেলে তার কারণ প্রায়ই version: module-এর নতুন version-এ input-এর নাম বদলে গেছে, অথবা provider-এর নতুন version-এ আচরণ বদলেছে। Version pin না করলে প্রতিটি `terraform init` একটি লটারি। Production-এ reproducibility (পুনরুৎপাদনযোগ্যতা) মানে: ছয় মাস পরেও একই কোড, একই version, একই ফলাফল। Versioning সেই নিশ্চয়তা দেয়, আর lock file তা প্রমাণ করে।

> **REAL WORLD:** TechNova-র একটি team VPC module-এর `main` branch সরাসরি ব্যবহার করত। একদিন module-এ subnet-এর নামকরণ বদলানো হলো — পরের `terraform plan`-এ সব subnet recreate হতে চাইল। Tag দিয়ে version pin করা থাকলে এই পরিবর্তন শুধু ইচ্ছাকৃত upgrade-এই আসত।

১৫.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ১৪-এর module, Git tag/branch, এবং `terraform init`-এর ধারণা জানা থাকতে হবে।

১৫.৪ Semantic Versioning — প্রথম নীতি থেকে

Semantic versioning-এ version লেখা হয় `MAJOR.MINOR.PATCH` আকারে — যেমন `1.2.0`। **PATCH** (1.2.0 → 1.2.1): bug fix, আচরণ একই। **MINOR** (1.2.0 → 1.3.0): নতুন feature, পুরোনো ব্যবহার ভাঙে না। **MAJOR** (1.2.0 → 2.0.0): breaking change — input-এর নাম বদল, output সরানো, resource replace। Git-এ `git tag v1.2.0` দিয়ে version চিহ্নিত করা হয়, আর module source-এ `?ref=v1.2.0` দিয়ে সেই tag pin করা হয়। Registry module-এ `version = "1.2.0"` আর্গুমেন্টে লেখা হয়।

> **CONCEPT:** Version হলো একটি চুক্তি (contract): MAJOR বদল মানে চুক্তি ভেঙেছে, MINOR/PATCH মানে চুক্তি অটুট। এই চুক্তি না মানলে কোনো team নিরাপদে upgrade করতে পারে না।

১৫.৫ Version Constraint — কোন version চলবে

```
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "~> 5.0"   # 5.0.0 বা তার বেশি, কিন্তু 6.0.0-এর নিচে
}
terraform {
  required_version = ">= 1.9, < 2.0"
  required_providers {
    aws = { source = "hashicorp/aws"  version = ">= 5.0, < 6.0" }
  }
}
```

Constraint	অর্থ
`= 1.2.0`	ঠিক এই version
`~> 5.0`	5.x — MINOR/PATCH upgrade চলবে, MAJOR নয়
`>= 1.9, < 2.0`	সীমার ভেতরে যেকোনো version
কোনো constraint নেই	সবচেয়ে নতুন version — production-এ নিষিদ্ধ

`~>` (pessimistic operator) production-এর আদর্শ: নিরাপত্তা-fix (PATCH) আসে, breaking change আসে না।

১৫.৬ .terraform.lock.hcl ও Dependency Management

`terraform init` চালালে Terraform provider ও module-এর ঠিক কোন version নামিয়েছে তা `.terraform.lock.hcl` ফাইলে লিখে রাখে — প্রতিটি provider-এর version ও checksum (hash) সহ। এই ফাইলটি **version control-এ commit করুন** — তাতে সব engineer ও CI একই version পায়। নতুন version নিতে চাইলে `terraform init -upgrade` চালান, lock file-এর পরিবর্তন review করুন, তারপর commit করুন। Lock file মুছে ফেললে বা commit না করলে প্রতিটি পরিবেশে আলাদা version নামতে পারে — "আমার মেশিনে চলে, CI-তে চলে না" সমস্যার জন্ম।

```
$ terraform init -upgrade
$ git diff .terraform.lock.hcl   # কী বদলালো review করুন
$ terraform plan                 # upgrade-এর প্রভাব দেখুন
```

১৫.৭ Terraform Registry — Public বনাম Private

Public Registry (registry.terraform.io) — সবার জন্য উন্মুক্ত, যেমন `terraform-aws-modules/vpc/aws`। **Private Registry** — Terraform Cloud/Enterprise-এ নিজের সংস্থার module-এর জন্য; শুধু নিজের team ব্যবহার করতে পারে। Module প্রকাশ করতে হলে GitHub repository-এর নাম হতে হবে `terraform-<provider>-<name>` আকারে (যেমন `terraform-aws-technova-vpc`) এবং version-এর জন্য Git tag থাকতে হবে। প্রকাশের আগে module-এর README, example ও `versions.tf` সম্পূর্ণ করুন।

> **WARNING:** Version pinning একটি নিরাপত্তা নিয়ন্ত্রণ (security control): pin না করা module-এর নতুন version-এ ক্ষতিকর পরিবর্তন (যেমন public access খুলে দেওয়া) চুপচাপ ঢুকে যেতে পারে। সবসময় version pin করুন, আর upgrade-এর আগে changelog পড়ুন।

১৫.৮ Breaking Change, Migration ও Monorepo বনাম Multi-repo

Breaking change-এ MAJOR version বাড়ান, CHANGELOG-এ কী ভেঙেছে ও কীভাবে migrate করতে হবে লিখুন, এবং পুরোনো MAJOR version কিছুদিন সমর্থন করুন। Module-এর গঠন (structure) বদলালে state-এর ঠিকানা বদলায় — যেমন resource module-এর ভেতরে সরালে তার address `aws_subnet.app` থেকে `module.vpc.aws_subnet.app` হয়; তখন `moved` block দিয়ে ঠিকানা বদলের ঘোষণা দিতে হয় (অধ্যায় ১৬)। **Monorepo** (সব module এক repo-তে): পরিবর্তন একসঙ্গে review করা সহজ, কিন্তু version আলাদা করা কঠিন। **Multi-repo** (প্রতিটি module আলাদা repo): version স্বাধীন, কিন্তু পরিবর্তনের সমন্বয় কঠিন। TechNova-র মতো ছোট team-এর জন্য monorepo + Git tag দিয়ে শুরু করা, বড় হলে multi-repo-তে যাওয়া বাস্তবসম্মত।

১৫.৯ সাধারণ ভুল ও Troubleshooting

ভুল: version constraint না দেওয়া; lock file commit না করা; `-upgrade` চালিয়ে plan না দেখে commit করা; MAJOR upgrade-এ changelog না পড়া। **Troubleshooting:** "Failed to query available provider packages" হলে নেটওয়ার্ক/registry যাচাই করুন; lock file-এর checksum mismatch হলে `terraform init -upgrade` দিয়ে নতুন করে lock করুন; module-এর পুরোনো version cache-এ আটকে থাকলে `terraform get -update` চালান।

> **TROUBLESHOOTING:** Upgrade-এর পর plan-এ অপ্রত্যাশিত পরিবর্তন দেখলে প্রথমে lock file-এর diff দেখুন — কোন provider/module-এর version বদলেছে, সেটিই কারণের সবচেয়ে বড় সূত্র।

১৫.১০ নিরাপত্তা ও Production Practices

সব module ও provider version pin করুন; lock file commit করুন; upgrade staging-এ প্রথমে পরীক্ষা করুন; private module-এর জন্য private registry বা Git + tag ব্যবহার করুন; তৃতীয় পক্ষের module-এর নতুন version-এর কোড review করুন।

১৫.১১ অধ্যায়ের সারাংশ

Semantic versioning (MAJOR.MINOR.PATCH) module-এর চুক্তি; version constraint (`~>`) upgrade-এর সীমা নির্ধারণ করে; `.terraform.lock.hcl` ঠিক কোন version ব্যবহৃত হচ্ছে তা প্রমাণ করে; Registry module প্রকাশ ও ব্যবহারের জায়গা; breaking change-এ MAJOR বাড়ান ও migration নির্দেশনা দিন।

১৫.১২ Knowledge Check

১. MAJOR, MINOR, PATCH-এর পার্থক্য কী? ২. `~> 5.0`-এর অর্থ কী? ৩. Lock file commit করা কেন জরুরি? ৪. Public বনাম private registry কখন কোনটি? ৫. Monorepo বনাম multi-repo-এর trade-off কী?

১৫.১৩ Practical Assignment

অধ্যায় ১৪-এর VPC module-এর জন্য `git tag v1.0.0` দিন, একটি root configuration থেকে `?ref=v1.0.0` দিয়ে call করুন, lock file commit করুন। তারপর module-এ একটি নতুন output যোগ করে `v1.1.0` tag দিন এবং `terraform init -upgrade` দিয়ে upgrade করুন।

অধ্যায় ১৬ • PART III — Advanced Terraform Engineering

Chapter 16 — Importing Existing Infrastructure and Infrastructure Migration

১৬.১ শেখার উদ্দেশ্য

এই অধ্যায়ে আপনি হাতে তৈরি (brownfield) infrastructure-কে Terraform-এর অধীনে আনা শিখবেন: `terraform import` CLI ও **Import Block** (ইমপোর্ট ব্লক — Terraform 1.5+), import workflow (config লিখুন → import করুন → plan করুন → drift ঠিক করুন), `terraform plan -generate-config-out`, migration কৌশল, `moved` block, `terraform state mv`, `removed` block দিয়ে নিরাপদে resource সরানো, এবং drift detection ও reconciliation।

১৬.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

কোনো কোম্পানিই শূন্য থেকে শুরু করে না। TechNova-র production VPC, database, DNS — অনেক কিছু console বা CLI দিয়ে হাতে তৈরি হয়েছিল, Terraform আসার আগে। সেগুলো মুছে নতুন করে তৈরি করা অসম্ভব — downtime ও ডেটা হারানোর ঝুঁকি। Import হলো সেতু: চলমান infrastructure-কে না থামিয়ে Terraform-এর পরিচালনায় আনা। ভুলভাবে করলে state ও বাস্তবতার মধ্যে অমিল (drift) তৈরি হয়, আর ভুল state surgery ডেটা মুছে দিতে পারে।

> **REAL WORLD:** TechNova-র production database হাতে তৈরি ছিল। একজন engineer সেটি import না করে নতুন database resource লিখে apply করেছিলেন — Terraform পুরোনো database-এর পাশে একই নামে নতুন database বানাতে গিয়ে fail করে, আর অ্যাপের connection string বিভ্রান্তিতে পড়ে। Import আগে করলে এই ঝুঁকি থাকত না।

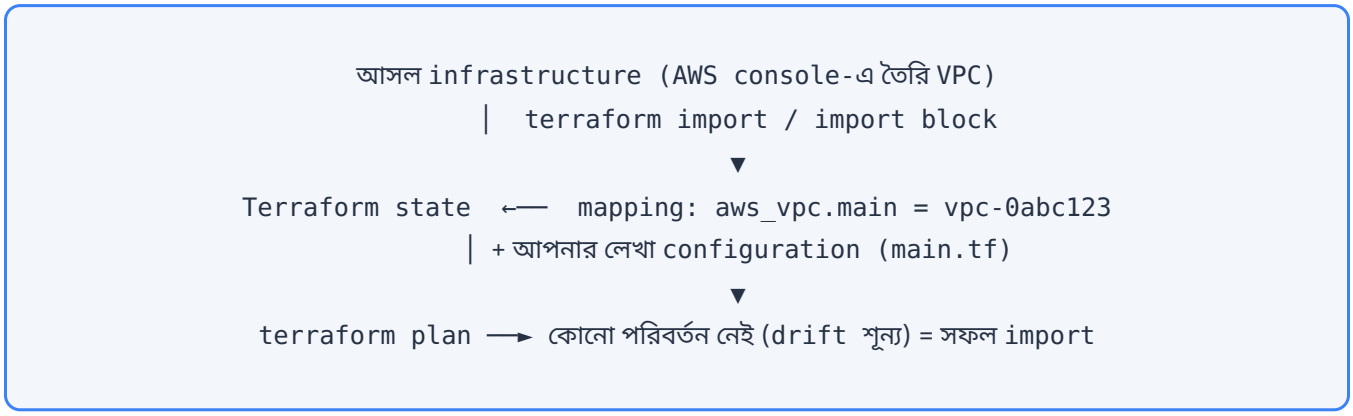
১৬.৩ পূর্বশর্ত জ্ঞান

State, resource address, `terraform plan` ও module (অধ্যায় ১২–১৫) জানা থাকতে হবে। Import করার আগে অবশ্যই state file-এর backup নেওয়ার অভ্যাস থাকতে হবে — remote backend-এ versioning চালু রাখুন।

১৬.৪ Import — প্রথম নীতি থেকে

Import মানে: Terraform-কে বলা — "এই আসল resource-টি (যার cloud ID আমি দিচ্ছি) এখন থেকে এই resource address-এর অধীনে পরিচালিত হবে।" Import resource তৈরি করে না, মুছে না, বদলায় না — শুধু state-এ একটি mapping যোগ করে। তাই import-এর আগে configuration লিখতে হয়, যাতে import-এর পর plan-এ পার্থক্য (drift) দেখা যায় ও ঠিক করা যায়।

> **CONCEPT:** Import হলো state-এ একটি ঠিকানা লেখা, configuration লেখা নয়। Terraform import করলে আপনার `.tf` ফাইল তৈরি করে দেয় না (generate-config ছাড়া) — configuration আপনাকেই লিখতে হবে, নইলে পরের plan-এ Terraform resource-টি মুছতে চাইবে।



১৬.৫ Import-এর দুটি পদ্ধতি

১৬.৫.১ CLI — terraform import

```
$ terraform import aws_vpc.main vpc-0abc123def456
$ terraform import 'aws_subnet.app["a"]' subnet-0abc123def456
```

সুবিধা: দ্রুত, একক resource-এর জন্য। অসুবিধা: আদেশটি state-এ সরাসরি লেখে, version control-এ থাকে না, পুনরাবৃত্তি করা যায় না।

১৬.৫.২ Import block (Terraform 1.5+) — প্রস্তাবিত পদ্ধতি

```
import {
  to = aws_vpc.main
  id = "vpc-0abc123def456"
}
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
  tags = { Name = "technova-production" }
}
```

Import block configuration-এর অংশ — review করা যায়, version control-এ থাকে, `terraform plan`-এ preview দেখা যায়। Apply করলে import সম্পন্ন হয়; তারপর import block সরিয়ে ফেলা যায়। `for_each` সহ import block-ও লেখা যায় — একসঙ্গে অনেক resource import করতে।

১৬.৫.৩ terraform plan -generate-config-out

```
$ terraform plan -generate-config-out=generated.tf
```

এই flag দিয়ে Terraform import করা resource-গুলোর জন্য configuration-এর খসড়া তৈরি করে দেয় — হাতে লেখার শুরু হিসেবে চমৎকার। তবে খসড়াটি যাচাই না করে ব্যবহার করবেন না: এতে অপ্রয়োজনীয় attribute ও provider-এর default মান থাকতে পারে।

১৬.৬ Import Workflow — ধাপে ধাপে

ধাপ ১: State backup নিন ও backend versioning যাচাই করুন। **ধাপ ২:** আসল resource-এর বর্তমান অবস্থা দেখে configuration লিখুন (console/API থেকে CIDR, tag, type সংগ্রহ করুন)। **ধাপ ৩:** Import block লিখে `terraform`

`plan` করুন — plan-এ import preview দেখা যাবে। **ধাপ ৪:** `terraform apply` দিয়ে import সম্পন্ন করুন। **ধাপ ৫:** আবার `terraform plan` করুন — "No changes" না আসা পর্যন্ত configuration ঠিক করুন (এটাই drift ঠিক করা)। **ধাপ ৬:** Import block সরিয়ে final plan যাচাই করুন।

> **WARNING:** Import করা resource-এ অপ্রত্যাশিত attribute থাকতে পারে — যেমন হাতে তৈরি security group-এ খোলা port, বা database-এ পুরোনো parameter। Import-এর পর plan-এর প্রতিটি attribute পড়ুন; যা বুঝতে পারছেন না তা না বুঝে apply করবেন না।

১৬.৭ Migration কৌশল — Brownfield Adoption

Gradual migration (ধাপে ধাপে): সবচেয়ে কম ঝুঁকির resource (যেমন VPC, subnet) দিয়ে শুরু করুন, তারপর database-এর মতো stateful resource। একবারে সব import করবেন না — প্রতিটি batch-এর পর plan যাচাই করুন। **Brownfield adoption** মানে চলমান সিস্টেমকে ধাপে ধাপে Terraform-এর অধীনে আনা, greenfield (নতুন করে তৈরি) নয়। Migration-এর সময় team-কে জানান: import চলাকালীন কেউ console থেকে resource বদলাবে না — নইলে drift তৈরি হবে।

১৬.৮ Moved Block ও terraform state mv — ঠিকানা বদলানো

Resource-কে এক module থেকে আরেক module-এ, বা এক নাম থেকে আরেক নামে সরাতে হলে তার state address বদলাতে হয়। **`moved` block** (প্রস্তাবিত):

```
moved {
  from = aws_subnet.app
  to   = module.vpc.aws_subnet.private
}
```

এটি configuration-এ লেখা থাকে, plan-এ preview দেখা যায়, apply-এর পর সরানো যায়। **`terraform state mv`** CLI আদেশ একই কাজ করে, তবে সরাসরি state বদলায় — ভুল ঠিকানা দিলে state নষ্ট হতে পারে। নিয়ম: `moved` block প্রথম পছন্দ; CLI শুধু জরুরি ক্ষেত্রে, backup নিয়ে।

> **WARNING:** হাতে state file সম্পাদনা (manual state editing) কখনোই স্বাভাবিক workflow নয় — এটি শেষ উপায়, backup ছাড়া করবেন না, এবং ভুল করলে পুরো state নষ্ট হতে পারে। সবসময় `moved` block বা `state mv`-এর মতো নিরাপদ পদ্ধতি ব্যবহার করুন।

১৬.৯ Removed Block — নিরাপদে resource সরানো

```
removed {
  from = aws_instance.legacy
  lifecycle { destroy = false }  # আসল resource রেখে শুধু state থেকে সরান
}
```

`removed` block দিয়ে resource-কে Terraform-এর পরিচালনা থেকে সরানো যায় — `destroy = false` দিলে আসল resource থেকে যায়, শুধু state থেকে মুছে যায়; `destroy = true` (default) দিলে আসল resource-ও মুছে যায়। Configuration থেকে resource block মুছে ফেলার আগে `removed` block দিয়ে ইচ্ছা স্পষ্ট করুন।

১৬.১০ Drift Detection ও Reconciliation

Drift হলো state-এর সঙ্গে বাস্তব infrastructure-এর অমিল — কেউ console থেকে কিছু বদলালে drift হয়। `terraform plan` (বা `terraform refresh`-সহ plan) drift দেখায়; নিয়মিত plan চালানো (CI-তে scheduled plan) drift detection-এর সবচেয়ে সহজ পদ্ধতি। Reconciliation মানে: drift ইচ্ছাকৃত হলে configuration-এ তা লিখুন; অনিচ্ছাকৃত হলে `terraform apply` দিয়ে আগের অবস্থায় ফেরান। `ignore_changes` দিয়ে drift ঢাকবেন না — তা সমস্যা লুকায়, সমাধান করে না।

১৬.১১ সাধারণ ভুল ও Troubleshooting

ভুল: Configuration না লিখে import করা; backup ছাড়া state surgery; একবারে সব resource import করা; import-এর পর plan-এর drift না দেখে apply করা। **Troubleshooting:** "Cannot import, resource already managed" হলে ঠিকানাটি state-এ আছে কি না `terraform state list` দিয়ে দেখুন; import-এর পর plan-ে বড় পরিবর্তন দেখালে configuration-এর মান আসল অবস্থার সঙ্গে মেলান; ভুল import হলে `terraform state rm <address>` দিয়ে শুধু state থেকে সরান (আসল resource মুছবে না), তারপর আবার import করুন।

> **TROUBLESHOOTING:** Import-এর পর plan "No changes" না দেখালে হতাশ হবেন না — এটি স্বাভাবিক। প্রতিটি পার্থক্য একে একে ঠিক করুন: tag, CIDR, instance type — আসল অবস্থা দেখে configuration মেলান, উল্টোটা নয়।

১৬.১২ নিরাপত্তা ও Production Practices

Import-এর আগে state backup ও backend versioning নিশ্চিত করুন; production-এ import staging-এ প্রথমে অনুশীলন করুন; import করা resource-এর IAM policy ও security group rule হাতে যাচাই করুন; migration-এর সময় console পরিবর্তন freeze করুন; প্রতিটি import batch-এর পর plan review করুন।

১৬.১৩ অধ্যায়ের সারাংশ

Import চলমান infrastructure-কে Terraform-এর অধীনে আনে — configuration লিখুন, import করুন, plan দিয়ে drift ঠিক করুন। Import block (1.5+) CLI-এর চেয়ে ভালো, কারণ তা review-যোগ্য। `moved` block দিয়ে ঠিকানা বদলান, `removed` block দিয়ে নিরাপদে সরান, হাতে state সম্পাদনা এড়িয়ে চলুন। Drift নিয়মিত plan দিয়ে ধরুন ও মেলান।

১৬.১৪ Knowledge Check

১. Import কী তৈরি করে, কী করে না? ২. Import block কেন CLI-এর চেয়ে ভালো? ৩. `-generate-config-out` কী দেয়? ৪. `moved` block বনাম `state mv`-এর পার্থক্য কী? ৫. `removed` block-এ `destroy = false`-এর অর্থ কী?

১৬.১৫ Practical Assignment

একটি হাতে তৈরি (বা `terraform import` অনুশীলনের জন্য তৈরি) VPC import করুন: প্রথমে configuration লিখুন, import block দিয়ে import করুন, plan-এ "No changes" না আসা পর্যন্ত drift ঠিক করুন। তারপর resource-টিকে একটি module-এ সরাতে `moved` block ব্যবহার করুন এবং plan যাচাই করুন।

PART IV • DEVOPS ENGINEERING MASTERY

Cloud Infrastructure Provisioning

AWS, Microsoft Azure, Google Cloud Platform ও IBM Cloud-এ Terraform দিয়ে Infrastructure Provisioning; Networking, Compute, Storage, Database ও IAM Automation।

অধ্যায় ১৭ • PART IV — Cloud Infrastructure Provisioning

Chapter 17 — AWS Infrastructure Provisioning with Terraform

১৭.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি AWS-এ Terraform দিয়ে সম্পূর্ণ অবকাঠামো তৈরি করতে পারবেন। আপনি `provider` কনফিগারেশন, IAM Role ও Policy, VPC নেটওয়ার্ক, Security Group, EC2 Instance, S3 Bucket, Application Load Balancer এবং RDS Database তৈরি ও পরিচালনা করতে শিখবেন। আরও শিখবেন S3 + DynamoDB দিয়ে Remote State ও State Locking কীভাবে করতে হয়, এবং `terraform destroy` দিয়ে নিরাপদে সম্পদ মুছে ফেলতে হয়।

১৭.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

AWS পৃথিবীর সবচেয়ে বেশি ব্যবহৃত ক্লাউড প্ল্যাটফর্ম। একটি আধুনিক কোম্পানির প্রায় সব workload — ওয়েব সার্ভার, ডাটাবেস, স্টোরেজ, নেটওয়ার্ক — AWS-এ চলে। হাতে ক্লিক করে এসব তৈরি করলে ভুল হয়, সময় লাগে এবং একই পরিবেশ দুইবার তৈরি করা যায় না। Terraform দিয়ে AWS অবকাঠামো কোড হিসেবে লিখলে তা version control-এ থাকে, review করা যায়, বারবার একইভাবে তৈরি করা যায় এবং ভুল হলে দ্রুত ফিরিয়ে আনা যায়। DevOps Engineer-এর দৈনন্দিন কাজের বড় অংশই AWS + Terraform।

১৭.৩ পূর্বশর্ত জ্ঞান

এই অধ্যায়ের জন্য Terraform-এর মৌলিক ধারণা (Resource, Data Source, Variable, Output, State), HCL সিনট্যাক্স এবং AWS-এর মৌলিক ধারণা (Region, Availability Zone, IAM) জানা থাকা ভালো। Linux terminal এবং Git-এর মৌলিক ব্যবহার জানতে হবে।

১৭.৪ প্রথম নীতিমালা: AWS কী এবং Terraform কীভাবে AWS-এর সঙ্গে কথা বলে

AWS হলো Amazon-এর ক্লাউড সেবা — অর্থাৎ অন্যের ডেটা সেন্টারে ভাড়া করা কম্পিউটার, নেটওয়ার্ক ও স্টোরেজ। Terraform AWS-এর সঙ্গে কথা বলে AWS API-র মাধ্যমে। আপনি HCL ফাইলে লেখেন `resource "aws_instance" "web" { ... }` — Terraform AWS Provider সেই লেখাকে AWS API call-এ রূপান্তর করে, যেমন `RunInstances`। Provider হলো অনুবাদক: HCL থেকে API। AWS Provider-এর ভেতরে প্রতিটি Resource Type একটি নির্দিষ্ট AWS API-র সঙ্গে যুক্ত।

> **CONCEPT:** Terraform নিজে কোনো সার্ভার তৈরি করে না। এটি শুধু AWS API-কে নির্দেশ দেয়। AWS-ই আসল কাজটি করে। তাই Terraform-এর কাজ হলো — কী চাই তা ঘোষণা করা (Declarative), বর্তমান অবস্থা জানা (State), এবং পার্থক্য (Diff) বের করে শুধু প্রয়োজনীয় API call করা।

Authentication বা প্রমাণীকরণের জন্য Terraform-এর AWS Access Key ও Secret Key দরকার, অথবা IAM Role। সবচেয়ে নিরাপদ পদ্ধতি হলো IAM Role ব্যবহার করা — বিশেষ করে EC2 Instance বা CI/CD Runner-এর ক্ষেত্রে। Access Key কখনো কোডে লেখা যাবে না।

১৭.৫ Provider কনফিগারেশন

AWS Provider কনফিগার করার সময় Region, Profile এবং Default Tags নির্ধারণ করা হয়। Region হলো ভৌগোলিক এলাকা, যেমন `ap-south-1` (মুম্বাই) বা `eu-west-1` (আয়ারল্যান্ড)। Default Tags দিলে প্রতিটি Resource-এ স্বয়ংক্রিয়ভাবে Tag যুক্ত হয় — এটি খরচের হিসাব ও মালিকানা নির্ণয়ে অত্যন্ত গুরুত্বপূর্ণ।

```
terraform {
  required_version = ">= 1.9, < 2.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region  = "ap-south-1"
  profile = "technova-dev"

  default_tags {
    tags = {
      Project     = "TechNova"
      Environment = "dev"
      ManagedBy   = "terraform"
      Owner       = "platform-team"
    }
  }
}
```

এখানে `profile` হলো `~/.aws/credentials` ফাইলে সংরক্ষিত Profile-এর নাম। `assume_role` ব্যবহার করে একটি Role-এর পরিচয় ধারণ করা যায় — এটি Cross-Account কাজে ব্যবহৃত হয়:

```
provider "aws" {
  region = "ap-south-1"
  assume_role {
    role_arn     = "arn:aws:iam::CHANGE-ME-ACCOUNT-ID:role/terraform-deployer"
    session_name = "terraform-session"
  }
}
```

`assume_role` মানে হলো — আমার নিজের পরিচয়ের বদলে নির্দিষ্ট Role-এর অনুমতি নিয়ে কাজ করা। এটি নিরাপত্তার জন্য ভালো, কারণ Role-এর অনুমতি সীমিত রাখা যায় এবং তা অস্থায়ী হয়।

১৭.৬ IAM: User, Role ও Policy — Least Privilege

IAM (Identity and Access Management) হলো AWS-এর পরিচয় ও অনুমতি ব্যবস্থা। User হলো একজন মানুষ বা অ্যাপ্লিকেশন, Role হলো অস্থায়ী পরিচয় যা কেউ ধারণ করতে পারে, আর Policy হলো JSON নথি যা বলে দেয় কে কী করতে পারবে। Least Privilege নীতি মানে — যতটুকু দরকার ঠিক ততটুকু অনুমতি দেওয়া, তার বেশি নয়।

```
resource "aws_iam_role" "ec2_web" {
  name = "technova-ec2-web-role"
  assume_role_policy = jsonencode({
```

```
    Version = "2012-10-17"
    Statement = [{
      Action    = "sts:AssumeRole"
      Effect    = "Allow"
      Principal = { Service = "ec2.amazonaws.com" }
    }]
  })
}

resource "aws_iam_role_policy" "ec2_s3_read" {
  name = "technova-ec2-s3-read"
  role = aws_iam_role.ec2_web.id
  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Action   = ["s3:GetObject", "s3:ListBucket"]
      Effect   = "Allow"
      Resource = ["arn:aws:s3:::technova-app-data", "arn:aws:s3:::technova-app-data/*"]
    }]
  })
}

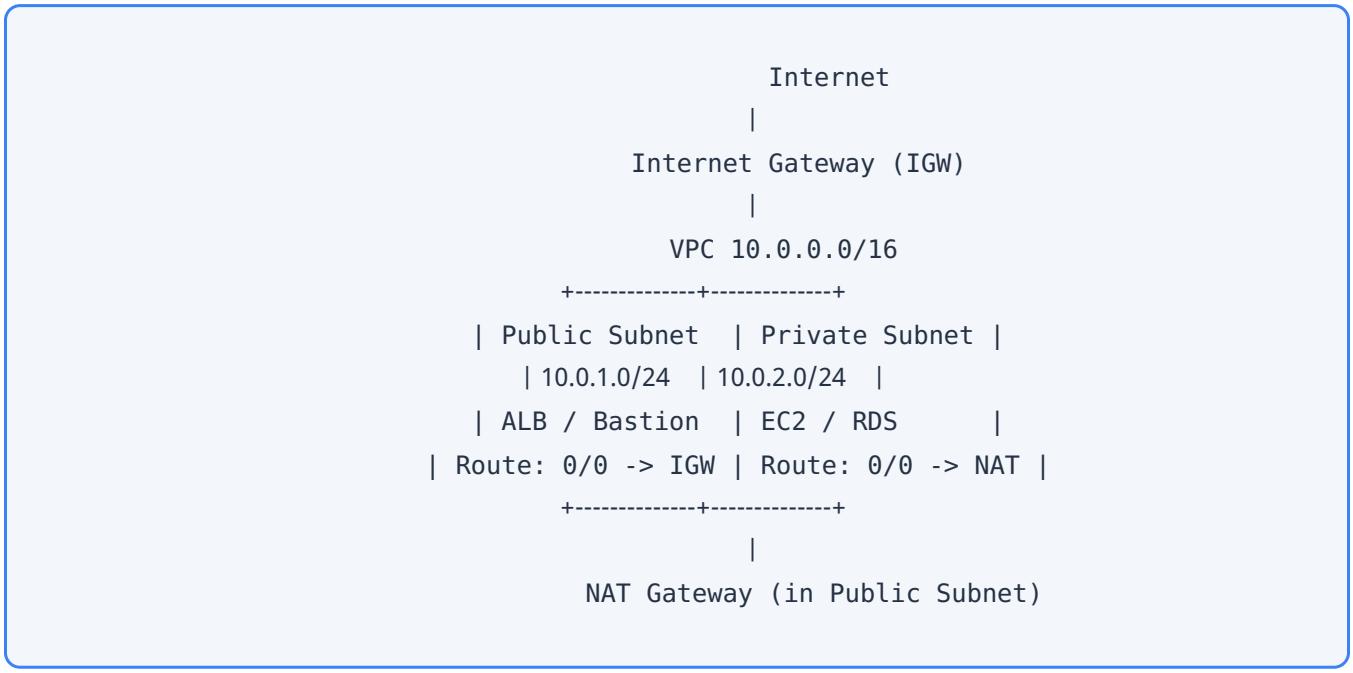
resource "aws_iam_instance_profile" "ec2_web" {
  name = "technova-ec2-web-profile"
  role = aws_iam_role.ec2_web.name
}
```

`assume_role_policy` বলে দেয় কে এই Role ধারণ করতে পারবে — এখানে শুধু EC2 Service। `aws_iam_instance_profile` হলো Role-কে EC2 Instance-এর সঙ্গে যুক্ত করার সেতু। Instance Profile ছাড়া EC2 Role পায় না।

> **WARNING:** কখনো `AdministratorAccess` Policy Terraform-এর জন্য ব্যবহার করবেন না। Terraform-এর জন্য শুধু প্রয়োজনীয় অনুমতির Policy তৈরি করুন। Admin অনুমতি থাকলে একটি ভুল `apply` পুরো অ্যাকাউন্ট মুছে ফেলতে পারে।

১৭.৭ VPC, Subnet, Route Table, Internet Gateway ও NAT Gateway

VPC (Virtual Private Cloud) হলো AWS-এর ভেতরে আপনার নিজের আলাদা নেটওয়ার্ক। এটি একটি CIDR Block দিয়ে তৈরি হয়, যেমন `10.0.0.0/16` — অর্থাৎ ৬৫,৫৩৬টি IP ঠিকানা। Subnet হলো VPC-র ভেতরে ছোট ভাগ। Public Subnet-এর Route Table-এ Internet Gateway (IGW) থাকে, Private Subnet-এ NAT Gateway থাকে।




```
resource "aws_vpc" "main" {
  cidr_block           = "10.0.0.0/16"
  enable_dns_hostnames = true
  enable_dns_support   = true
  tags = { Name = "technova-vpc" }
}

resource "aws_subnet" "public" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = "10.0.1.0/24"
  availability_zone       = "ap-south-1a"
  map_public_ip_on_launch = true
  tags = { Name = "technova-public-a" }
}

resource "aws_subnet" "private" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = "10.0.2.0/24"
  availability_zone = "ap-south-1a"
  tags = { Name = "technova-private-a" }
}

resource "aws_internet_gateway" "main" {
  vpc_id = aws_vpc.main.id
  tags   = { Name = "technova-igw" }
}

resource "aws_eip" "nat" { domain = "vpc" }

resource "aws_nat_gateway" "main" {
  allocation_id = aws_eip.nat.id
  subnet_id     = aws_subnet.public.id
  tags          = { Name = "technova-nat" }
  depends_on    = [aws_internet_gateway.main]
}

resource "aws_route_table" "public" {
```

```
  vpc_id = aws_vpc.main.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.main.id
  }
  tags = { Name = "technova-rt-public" }
}

resource "aws_route_table_association" "public" {
  subnet_id      = aws_subnet.public.id
  route_table_id = aws_route_table.public.id
}
```

`enable_dns_hostnames` true করলে Instance-গুলো DNS Hostname পায়। NAT Gateway-এর জন্য Elastic IP (EIP) দরকার — এটি একটি স্থির Public IP। NAT Gateway সবসময় Public Subnet-এ থাকে, কারণ তার ইন্টারনেটে যাওয়ার পথ দরকার। Private Subnet-এর Instance NAT-এর মাধ্যমে বাইরে যেতে পারে, কিন্তু বাইরে থেকে কেউ সরাসরি তাদের কাছে আসতে পারে না — এটাই নিরাপত্তা।

> **CONCEPT:** Route Table হলো নেটওয়ার্কের দিকনির্দেশনা। `0.0.0.0/0` মানে "সব ঠিকানা"। Public Subnet বলে — সব ট্রাফিক IGW-তে যাও। Private Subnet বলে — সব ট্রাফিক NAT-এ যাও। Subnet নিজে Public বা Private নয়; তার Route Table-ই তা নির্ধারণ করে।

১৭.৮ Security Group

Security Group হলো Virtual Firewall যা Instance-স্তরে কাজ করে। এটি Stateful — অর্থাৎ Outbound Response স্বয়ংক্রিয়ভাবে অনুমোদিত হয়। Ingress হলো ভেতরে আসা ট্রাফিক, Egress হলো বাইরে যাওয়া ট্রাফিক।

```
resource "aws_security_group" "web" {
  name        = "technova-web-sg"
  description = "Allow HTTP and SSH to web servers"
  vpc_id      = aws_vpc.main.id

  ingress {
    description = "HTTP from anywhere"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  ingress {
    description = "SSH from office only"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["203.0.113.0/24"]
  }
  egress {
    description = "All outbound"
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
```

```
  tags = { Name = "technova-web-sg" }
}
```

SSH-এর জন্য `0.0.0.0/0` কখনো ব্যবহার করবেন না — শুধু অফিসের IP (এখানে উদাহরণস্বরূপ `203.0.113.0/24`) অনুমোদন করুন। `protocol = "-1"` মানে সব Protocol।

১৭.৯ EC2 Instance, AMI Data Source ও User Data

EC2 (Elastic Compute Cloud) হলো ভার্চুয়াল সার্ভার। AMI (Amazon Machine Image) হলো সার্ভারের ছবি — Operating System ও সফটওয়্যারের টেমপ্লেট। Data Source দিয়ে সবশেষ Ubuntu AMI খুঁজে নেওয়া যায়, যাতে AMI ID হার্ডকোড করতে না হয়। `user_data` হলো Instance প্রথম চালু হওয়ার সময় চলা Script।

```
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"] # Canonical (Ubuntu)
  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
  }
}

resource "aws_instance" "web" {
  ami                    = data.aws_ami.ubuntu.id
  instance_type          = "t3.micro"
  subnet_id              = aws_subnet.public.id
  vpc_security_group_ids = [aws_security_group.web.id]
  iam_instance_profile   = aws_iam_instance_profile.ec2_web.name

  user_data = <<-EOF
    #!/bin/bash
    apt-get update -y
    apt-get install -y nginx
    systemctl enable --now nginx
    echo "<h1>Hello from TechNova</h1>" > /var/www/html/index.html
  EOF

  tags = { Name = "technova-web-01" }
}
```

`most_recent = true` মানে সবশেষ AMI নেওয়া হবে। `owners` হলো AMI-র মালিকের Account ID — Canonical-এর ID `099720109477`। `t3.micro` হলো ছোট ও সস্তা Instance Type, শেখার জন্য আদর্শ। `user_data` Script শুধু প্রথম Boot-এ চলে — পরে পরিবর্তন করলে Instance Replace হয়।

১৭.১০ S3 Bucket: Versioning ও Encryption

S3 (Simple Storage Service) হলো Object Storage — ফাইল রাখার জায়গা। Bucket হলো Container। Versioning চালু করলে ফাইলের প্রতিটি সংস্করণ সংরক্ষিত থাকে। Encryption চালু করলে ফাইল Disk-এ Encrypted অবস্থায় থাকে।

```
resource "aws_s3_bucket" "app" {
  bucket = "technova-app-data-CHANGE-ME-UNIQUE"
  tags   = { Name = "technova-app-data" }
}
```

```
resource "aws_s3_bucket_versioning" "app" {
  bucket = aws_s3_bucket.app.id
  versioning_configuration { status = "Enabled" }
}

resource "aws_s3_bucket_server_side_encryption_configuration" "app" {
  bucket = aws_s3_bucket.app.id
  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}

resource "aws_s3_bucket_public_access_block" "app" {
  bucket                  = aws_s3_bucket.app.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}
```

Bucket-এর নাম বিশ্বজুড়ে Unique হতে হয়। `aws_s3_bucket_public_access_block` সবসময় চালু রাখুন — ভুলে Public হয়ে গেলে ডেটা ফাঁস হয়।

১৭.১১ Application Load Balancer ও RDS ধারণা

ALB (Application Load Balancer) একাধিক EC2 Instance-এর মধ্যে ট্রাফিক ভাগ করে দেয়। এটি Layer 7 (HTTP) স্তরে কাজ করে। Target Group হলো Instance-গুলোর দল, Listener হলো Port 80-এ শোনার নিয়ম, Health Check হলো Instance সুস্থ আছে কি না তা যাচাই।

```
resource "aws_lb" "web" {
  name               = "technova-alb"
  internal           = false
  load_balancer_type = "application"
  security_groups    = [aws_security_group.web.id]
  subnets            = [aws_subnet.public.id]
  tags               = { Name = "technova-alb" }
}

resource "aws_lb_target_group" "web" {
  name     = "technova-tg"
  port     = 80
  protocol = "HTTP"
  vpc_id   = aws_vpc.main.id
  health_check {
    path                = "/"
    healthy_threshold   = 2
    unhealthy_threshold = 2
    interval            = 30
  }
}
```

RDS (Relational Database Service) হলো Managed Database — AWS ডাটাবেস সার্ভারটি পরিচালনা করে। `aws_db_instance` দিয়ে Database তৈরি হয়, `aws_db_subnet_group` বলে দেয় কোন Subnet-এ ডাটাবেস বসবে,

আর `aws_db_parameter_group` দিয়ে Database-এর Setting বদলানো যায়। Production-এ RDS সবসময় Private Subnet-এ রাখুন, Publicly Accessible করবেন না।

```
resource "aws_db_subnet_group" "main" {
  name       = "technova-db-subnet"
  subnet_ids = [aws_subnet.private.id]
  tags       = { Name = "technova-db-subnet" }
}

resource "aws_db_instance" "main" {
  identifier             = "technova-db"
  engine                 = "postgres"
  engine_version         = "16.3"
  instance_class         = "db.t3.micro"
  allocated_storage      = 20
  db_name                = "technova"
  username               = "technova_admin"
  password               = var.db_password # sensitive variable
  db_subnet_group_name   = aws_db_subnet_group.main.name
  vpc_security_group_ids = [aws_security_group.web.id]
  multi_az               = false
  skip_final_snapshot    = true
  tags                   = { Name = "technova-db" }
}
```

`skip_final_snapshot = true` শুধু Dev-এর জন্য — Production-এ false রাখুন, যাতে মুছে ফেলার আগে Snapshot থাকে। Password কখনো কোডে লিখবেন না — `var.db_password` Sensitive Variable হিসেবে দিন।

১৭.১২ Remote State: S3 + DynamoDB Locking

Local State File দলগত কাজে চলে না। S3 Backend ব্যবহার করলে State ফাইল S3-এ থাকে এবং DynamoDB Table Lock দেয়, যাতে দুজন একসাথে Apply করতে না পারে।

```
resource "aws_s3_bucket" "tfstate" {
  bucket = "technova-tfstate-CHANGE-ME-UNIQUE"
}
resource "aws_s3_bucket_versioning" "tfstate" {
  bucket = aws_s3_bucket.tfstate.id
  versioning_configuration { status = "Enabled" }
}
resource "aws_dynamodb_table" "tflock" {
  name         = "technova-tf-lock"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "LockID"
  attribute {
    name = "LockID"
    type = "S"
  }
}

terraform {
  backend "s3" {
    bucket         = "technova-tfstate-CHANGE-ME-UNIQUE"
    key            = "dev/terraform.tfstate"
    region         = "ap-south-1"
```

```
    dynamodb_table = "technova-tf-lock"
    encrypt        = true
  }
}
```

Backend Block-এ Variable ব্যবহার করা যায় না — Bucket-এর নাম সরাসরি লিখতে হয়। Backend বদলালে `terraform init -migrate-state` চালাতে হয়।

১৭.১৩ TechNova সম্পূর্ণ উদাহরণ: VPC, Subnet, Security Group, EC2, S3

নিচে একটি সম্পূর্ণ কাজের উদাহরণ দেওয়া হলো। ফাইল গঠন: `main.tf`, `variables.tf`, `outputs.tf`।

```
# variables.tf
variable "region" { default = "ap-south-1" }
variable "db_password" {
  type      = string
  sensitive = true
}

# outputs.tf
output "web_public_ip" {
  value = aws_instance.web.public_ip
}
output "s3_bucket" {
  value = aws_s3_bucket.app.bucket
}
```

চালানোর ধাপ:

```
$ terraform init
$ terraform fmt -check
$ terraform validate
$ terraform plan -out=tfplan
$ terraform apply tfplan
```

প্রত্যাশিত আউটপুটের অংশ:

```
Plan: 12 to add, 0 to change, 0 to destroy.
aws_vpc.main: Creating...
aws_subnet.public: Creating...
aws_instance.web: Creating...
Apply complete! Resources: 12 added, 0 changed, 0 destroyed.
Outputs:
web_public_ip = "203.0.113.10"
```

IP ঠিকানাটি ব্রাউজারে খুললে "Hello from TechNova" দেখা যাবে।

১৭.১৪ সাধারণ ভুল ও Troubleshooting

সমস্যা	কারণ	সমাধান
	AWS Credential নেই	

`Error: No valid credential sources`		`aws configure` করুন বা Environment Variable দিন
`InvalidAMIID.NotFound`	ভুল Region-এ AMI	Data Source ব্যবহার করুন, AMI হার্ডকোড নয়
EC2-তে SSH হচ্ছে না	Security Group-এ 22 বন্ধ	নিজের IP দিয়ে Ingress খুলুন
`BucketAlreadyExists`	S3 নাম Unique নয়	নামে Random Suffix যোগ করুন
State Lock আটকে আছে	আগের Apply মাঝপথে থেমেছে	নিশ্চিত হয়ে `terraform force-unlock LOCK-ID`

TROUBLESHOOTING: `terraform plan`-এ হঠাৎ সব Resource "to destroy" দেখালে ভয় পাবেন না — সম্ভবত Workspace, Backend বা Variable বদলেছে। `terraform state list` দিয়ে দেখুন State ঠিক আছে কি না, তারপর Apply করুন।

১৭.১৫ নিরাপত্তা ও Production অনুশীলন

Production-এ সব Resource Private Subnet-এ রাখুন, শুধু ALB Public রাখুন। সব S3 Bucket-এ Encryption ও Public Access Block চালু করুন। IAM Policy-তে Least Privilege মেনে চলুন। State Bucket-এ Versioning ও Encryption বাধ্যতামূলক করুন। সব Resource-এ Tag দিন — `Environment`, `Owner`, `CostCenter`।

REAL WORLD: TechNova-র একটি Dev পরিবেশে একজন Engineer ভুলে S3 Bucket Public করে দিয়েছিলেন — Customer ডেটা Search Engine-এ দেখা যাচ্ছিল। এরপর থেকে `aws_s3_bucket_public_access_block` এবং SCP (Service Control Policy) দিয়ে Public Bucket নিষিদ্ধ করা হয়েছে। একটি ভুল থেকে শিক্ষা নিয়ে Policy হিসেবে কোডে লেখা হয়েছে।

WARNING: NAT Gateway ঘণ্টা প্রতি চার্জ হয় এবং ডেটা Transfer-এরও খরচ আছে। Dev পরিবেশে দীর্ঘদিন NAT Gateway চালু রাখলে বিল বাড়ে। কাজ শেষে `terraform destroy` চালান।

১৭.১৬ অধ্যায়ের সারাংশ

এই অধ্যায়ে আপনি AWS Provider, IAM, VPC, Security Group, EC2, S3, ALB, RDS এবং S3 + DynamoDB Remote State শিখেছেন। মূল শিক্ষা: AWS-এ যা তৈরি করবেন সব কোডে লিখুন, Least Privilege মেনে চলুন, এবং Private Subnet-এ রাখুন।

১৭.১৭ Knowledge Check

১. VPC কী এবং Public ও Private Subnet-এর পার্থক্য কী? ২. NAT Gateway কেন Public Subnet-এ থাকে? ৩. Security Group Stateful বলতে কী বোঝায়? ৪. State Locking কেন দরকার? ৫. `user_data` কখন চলে?

১৭.১৮ Practical Assignment

TechNova-র জন্য একটি সম্পূর্ণ AWS পরিবেশ তৈরি করুন: VPC (`10.1.0.0/16`), দুটি Public ও দুটি Private Subnet (দুটি AZ), একটি EC2 Instance (Nginx), একটি S3 Bucket (Versioning + Encryption সহ) এবং S3 Backend। তারপর `terraform destroy` চালিয়ে সব মুছে ফেলুন এবং খরচ যাচাই করুন। ফলাফল একটি README-তে লিখুন।

অধ্যায় ১৮ • PART IV — Cloud Infrastructure Provisioning

Chapter 18 — Microsoft Azure Infrastructure Provisioning

১৮.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি Azure-এ Terraform দিয়ে Resource Group, Virtual Network, Subnet, Network Security Group, Virtual Machine, Storage Account ও Managed Identity তৈরি করতে পারবেন। Azure Remote State (azurerm Backend) কনফিগার করতে পারবেন এবং সম্পূর্ণ TechNova Azure উদাহরণ চালাতে পারবেন।

১৮.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

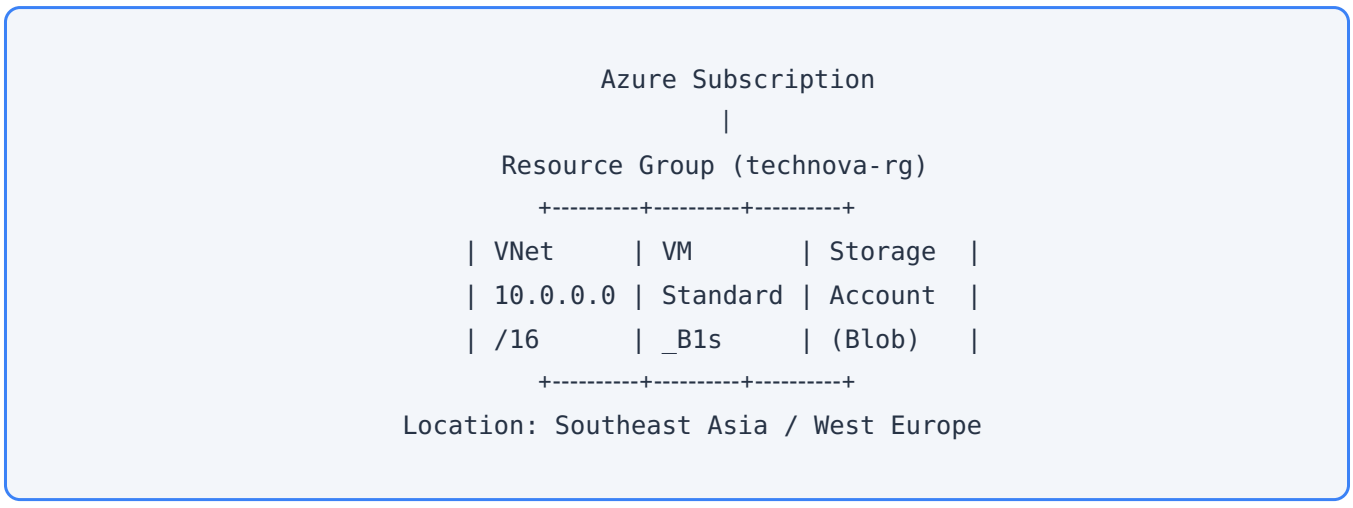
Azure হলো Microsoft-এর ক্লাউড — অনেক Enterprise, সরকারি প্রতিষ্ঠান ও .NET পরিবেশ Azure ব্যবহার করে। AWS জানলেই যথেষ্ট নয়; বহু কোম্পানিতে Azure-ই মূল ক্লাউড। Azure-এর ধারণাগুলো AWS-এর সঙ্গে মিলে যায়, কিন্তু নাম ও গঠন আলাদা — যেমন VPC-র বদলে Virtual Network, Security Group-এর বদলে Network Security Group, আর সবকিছুর ওপরে থাকে Resource Group। এই অধ্যায়ে সেই অনুবাদ শিখবেন।

১৮.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ১৭-এর AWS ধারণা, Terraform-এর মৌলিক জ্ঞান এবং Azure-এর মৌলিক ধারণা (Subscription, Resource Group) জানা থাকলে ভালো হয়। Azure CLI (`az`) ইনস্টল থাকা দরকার।

১৮.৪ প্রথম নীতিমালা: Azure-এর গঠন — Management Group থেকে Resource

Azure-এর স্তরবিন্যাসটি এমন: Management Group → Subscription → Resource Group → Resource। Subscription হলো বিলিং ও সীমার একক। Resource Group হলো Resource-গুলোর ফোল্ডার — একসাথে তৈরি, পরিচালনা ও মুছে ফেলার একক। একটি Resource Group মুছে ফেললে তার ভেতরের সব Resource মুছে যায় — এটি যেমন সুবিধা, তেমনি বিপদ।



> **CONCEPT:** Resource Group কোনো নেটওয়ার্ক সীমানা নয় — এটি শুধু পরিচালনার সীমানা। দুটি Resource Group-এর Resource একই Virtual Network-এ থাকতে পারে। Location মানে Resource Group-এর Metadata কোথায় থাকবে; ভেতরের Resource অন্য Region-এও থাকতে পারে, তবে সাধারণত একই Location-এ রাখাই ভালো।

১৮.৫ Provider কনফিগারেশন: Subscription ও Features Block

Azure Provider-এর নাম `azurerm`। এর বিশেষত্ব হলো `features {}` Block — এটি বাধ্যতামূলক। `features`-এর ভেতরে Provider-এর আচরণ নিয়ন্ত্রণ করা যায়, যেমন Key Vault-এর Soft Delete।

```
terraform {
  required_version = ">= 1.9, < 2.0"
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 4.0"
    }
  }
}

provider "azurerm" {
  features {}
  subscription_id = var.subscription_id
}
```

Authentication-এর জন্য Azure CLI-তে লগইন করুন: `az login`। Terraform তখন CLI-র Credential ব্যবহার করে। CI/CD-তে Service Principal বা OIDC ব্যবহার করুন — কখনো Password কোডে লিখবেন না। `subscription_id` Variable হিসেবে দিন, যাতে একই কোড অন্য Subscription-এ চালানো যায়।

`features` Block-এর একটি দরকারি ব্যবহার:

```
provider "azurerm" {
  features {
    resource_group {
      prevent_deletion_if_contains_resources = true
    }
    key_vault {
      purge_soft_delete_on_destroy = false
    }
  }
}
```

`prevent_deletion_if_contains_resources = true` মানে — ভেতরে Resource থাকলে Resource Group মুছে ফেলা যাবে না। এটি Production-এ দুর্ঘটনা রোধ করে।

১৮.৬ Resource Group, Virtual Network ও Subnet

Resource Group তৈরি করা Terraform-এর সবচেয়ে সহজ Resource — শুধু নাম ও Location লাগে। Virtual Network (VNet) হলো Azure-এর VPC। Address Space হলো CIDR Block। Subnet হলো VNet-এর ভাগ।

```
variable "location" { default = "southeastasia" }
variable "subscription_id" { type = string }

resource "azurerm_resource_group" "main" {
  name     = "technova-rg"
  location = var.location
  tags = {
    Project     = "TechNova"
    Environment = "dev"
    ManagedBy   = "terraform"
  }
}

resource "azurerm_virtual_network" "main" {
  name                = "technova-vnet"
  address_space       = ["10.0.0.0/16"]
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  tags                = { Environment = "dev" }
}

resource "azurerm_subnet" "public" {
  name                 = "technova-public"
  resource_group_name  = azurerm_resource_group.main.name
  virtual_network_name = azurerm_virtual_network.main.name
  address_prefixes     = ["10.0.1.0/24"]
}

resource "azurerm_subnet" "private" {
  name                 = "technova-private"
  resource_group_name  = azurerm_resource_group.main.name
  virtual_network_name = azurerm_virtual_network.main.name
  address_prefixes     = ["10.0.2.0/24"]
}
```

লক্ষ্য করুন — Azure-এ প্রায় প্রতিটি Resource-এ `resource_group_name` ও `location` লিখতে হয়। Terraform-এ `azurerm_resource_group.main.location` লিখে এক জায়গা থেকে নেওয়া যায়, যাতে Location বদলাতে একটাই জায়গা বদলাতে হয়।

১৮.৭ Network Security Group (NSG)

NSG হলো Azure-এর Firewall। AWS Security Group-এর মতোই, তবে NSG-তে প্রতিটি Rule-এর Priority নম্বর থাকে — ছোট নম্বরের Rule আগে মেলে। NSG Subnet বা Network Interface-এর সঙ্গে যুক্ত করা যায়।

```
resource "azurerm_network_security_group" "web" {
  name                = "technova-web-nsg"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name

  security_rule {
    name                       = "Allow-HTTP"
    priority                   = 100
    direction                  = "Inbound"
    access                     = "Allow"
    protocol                   = "Tcp"
```

```
    source_port_range          = "*"
    destination_port_range     = "80"
    source_address_prefix      = "*"
    destination_address_prefix = "*"
  }
  security_rule {
    name                       = "Allow-SSH-Office"
    priority                   = 110
    direction                  = "Inbound"
    access                     = "Allow"
    protocol                   = "Tcp"
    source_port_range          = "*"
    destination_port_range     = "22"
    source_address_prefix      = "203.0.113.0/24"
    destination_address_prefix = "*"
  }
  tags = { Environment = "dev" }
}

resource "azurerm_subnet_network_security_group_association" "public" {
  subnet_id                 = azurerm_subnet.public.id
  network_security_group_id = azurerm_network_security_group.web.id
}
```

Priority 100–4096-এর মধ্যে হয়। একই Priority দুই Rule-এ দেওয়া যায় না। SSH-এর জন্য আবারও — শুধু অফিসের IP।

১৮.৮ Virtual Machine, Public IP ও Network Interface

Azure-এ VM তৈরি করতে তিনটি জিনিস লাগে: Public IP, Network Interface (NIC) এবং VM নিজে। NIC হলো VM-এর নেটওয়ার্ক কার্ড — এটি Subnet ও NSG-র সঙ্গে যুক্ত হয়।

```
resource "azurerm_public_ip" "web" {
  name                = "technova-web-pip"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  allocation_method   = "Static"
  sku                 = "Standard"
  tags                = { Environment = "dev" }
}

resource "azurerm_network_interface" "web" {
  name                = "technova-web-nic"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  ip_configuration {
    name                          = "internal"
    subnet_id                     = azurerm_subnet.public.id
    private_ip_address_allocation = "Dynamic"
    public_ip_address_id          = azurerm_public_ip.web.id
  }
}

resource "azurerm_linux_virtual_machine" "web" {
  name                = "technova-web-01"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
  size                = "Standard_B1s"
```

```
  admin_username      = "technova"
  network_interface_ids = [azurerm_network_interface.web.id]

  admin_ssh_key {
    username   = "technova"
    public_key = file("~/.ssh/id_ed25519.pub")
  }

  os_disk {
    caching              = "ReadWrite"
    storage_account_type = "Standard_LRS"
  }

  source_image_reference {
    publisher = "Canonical"
    offer     = "0001-com-ubuntu-server-jammy"
    sku       = "22_04-lts"
    version   = "latest"
  }

  custom_data = base64encode(<<-EOF
    #!/bin/bash
    apt-get update -y
    apt-get install -y nginx
    systemctl enable --now nginx
  EOF
  )
  tags = { Environment = "dev" }
}
```

`Standard_B1s` হলো সবচেয়ে ছোট ও সস্তা VM Size। `custom_data` হলো Azure-এর `user_data` — তবে এটি Base64 Encode করতে হয়। Password-এর বদলে SSH Key ব্যবহার করুন — Password Authentication বন্ধ রাখাই নিরাপদ।

> **REAL WORLD:** TechNova-র একটি দল VM-এর Password Terraform-এ লিখে Git-এ Push করেছিল। Password বদলাতে গিয়ে দেখা গেল VM-টি Replace হয়ে গেছে, কারণ `admin_password` বদলালে Azure VM পুনর্নির্মাণ হয়। শিক্ষা: SSH Key ব্যবহার করুন, Password নয়; আর Sensitive মান বদলানোর আগে `terraform plan` দেখুন।

১৮.৯ Storage Account ও Managed Identity

Storage Account হলো Azure-এর S3-এর সমতুল্য — Blob, File, Queue ও Table রাখার জায়গা। নাম বিশ্বজুড়ে Unique এবং শুধু ছোট হাতের অক্ষর ও সংখ্যা ব্যবহার করা যায়। Managed Identity হলো Azure Resource-এর নিজস্ব পরিচয় — কোনো Secret ছাড়াই অন্য Azure সেবায় প্রবেশের অধিকার দেয়।

```
resource "azurerm_storage_account" "app" {
  name                     = "technovaappdataCHANGE-ME"
  resource_group_name      = azurerm_resource_group.main.name
  location                 = azurerm_resource_group.main.location
  account_tier             = "Standard"
  account_replication_type = "LRS"
  min_tls_version          = "TLS1_2"
```

```
  tags                     = { Environment = "dev" }
}

resource "azurerm_storage_container" "app" {
  name                  = "appdata"
  storage_account_name  = azurerm_storage_account.app.name
  container_access_type = "private"
}
```

`LRS` (Locally Redundant Storage) মানে ডেটা এক ডেটা সেন্টারে তিন কপি থাকে। `GRS` হলে অন্য Region-এও কপি থাকে — বেশি নিরাপদ, বেশি খরচ। `min_tls_version = "TLS1_2"` পুরনো অনিরাপদ TLS বন্ধ করে।

```
resource "azurerm_user_assigned_identity" "web" {
  name                = "technova-web-identity"
  location            = azurerm_resource_group.main.location
  resource_group_name = azurerm_resource_group.main.name
}

# VM-এ System-Assigned Identity চালু করা
# azurerm_linux_virtual_machine-এর ভেতরে:
# identity { type = "SystemAssigned" }
```

System-Assigned Identity VM-এর সঙ্গে জন্মায় ও মরে — VM মুছলে Identity-ও মুছে যায়। User-Assigned Identity আলাদা Resource — একাধিক VM ব্যবহার করতে পারে। দুটোরই উদ্দেশ্য এক: Secret ছাড়া প্রমাণীকরণ।

১৮.১০ Azure Remote State: azurerm Backend

Azure-এ Remote State রাখা হয় Storage Account-এর Blob Container-এ। প্রথমে একটি Storage Account তৈরি করতে হয় (হাতে বা আলাদা Bootstrap Terraform দিয়ে), তারপর Backend কনফিগার করতে হয়।

```
terraform {
  backend "azurerm" {
    resource_group_name  = "technova-tfstate-rg"
    storage_account_name = "technovatfstateCHANGE-ME"
    container_name       = "tfstate"
    key                  = "dev.terraform.tfstate"
  }
}
```

azurerm Backend নিজেই Blob Lease দিয়ে State Locking করে — আলাদা DynamoDB-র মতো কিছু লাগে না। Backend-এ Variable ব্যবহার করা যায় না; `terraform init -backend-config="key=dev.terraform.tfstate"` দিয়ে মান দেওয়া যায়।

১৮.১১ TechNova সম্পূর্ণ Azure উদাহরণ

ফাইল গঠন: `main.tf` (উপরের সব Resource), `variables.tf`, `outputs.tf`।

```
# outputs.tf
output "web_public_ip" {
  value = azurerm_public_ip.web.ip_address
}
output "resource_group" {
```

```
  value = azurerm_resource_group.main.name
}
```

```
$ az login
$ terraform init
$ terraform plan -out=tfplan
$ terraform apply tfplan
```

প্রত্যাশিত আউটপুট:

```
Plan: 8 to add, 0 to change, 0 to destroy.
Apply complete! Resources: 8 added, 0 changed, 0 destroyed.
Outputs:
web_public_ip = "203.0.113.20"
```

IP ঠিকানায় ব্রাউজার খুললে Nginx-এর স্বাগত পাতা দেখা যাবে।

১৮.১২ সাধারণ ভুল ও Troubleshooting

সমস্যা	কারণ	সমাধান
`features block is required`	Provider-এ features নেই	`features {}` যোগ করুন
Storage Account নাম ভুল	বড় হাতের অক্ষর বা বিশেষ চিহ্ন	শুধু ছোট হাতের অক্ষর ও সংখ্যা
`ResourceGroupNotFound`	Backend-এর RG আগে তৈরি হয়নি	Bootstrap ধাপে আগে তৈরি করুন
VM-এ SSH হচ্ছে না	NSG বা Public IP সমস্যা	NSG Rule ও NIC যাচাই করুন

TROUBLESHOOTING: Azure-এর ভুল বার্তায় প্রায়ই Resource ID দেওয়া থাকে। `az resource show --ids <ID>` দিয়ে Azure-এ আসল অবস্থা দেখুন, তারপর Terraform State-এর সঙ্গে মেলান।

১৮.১৩ নিরাপত্তা ও Production অনুশীলন

Storage Account-এ TLS 1.2 বাধ্যতামূলক করুন, Public Blob Access বন্ধ রাখুন। VM-এ Password নয়, SSH Key ব্যবহার করুন। Managed Identity দিয়ে Secret ছাড়া প্রমাণীকরণ করুন। Resource Group মুছে ফেলা থেকে রক্ষা পেতে `prevent_deletion_if_contains_resources` চালু করুন। সব Resource-এ Tag দিন।

WARNING: Azure VM, Public IP ও Storage — সবকিছুরই ঘণ্টা বা মাস প্রতি খরচ আছে। `Standard_B1s` সস্তা হলেও ভুলে বড় Size চালু রাখলে বিল বাড়ে। কাজ শেষে `terraform destroy` চালান এবং Azure Portal-এ Cost Management দেখে যাচাই করুন।

১৮.১৪ অধ্যায়ের সারাংশ

এই অধ্যায়ে আপনি Azure-এর স্তরবিন্যাস, azurerm Provider, Resource Group, VNet, NSG, VM, Storage Account, Managed Identity ও azurerm Backend শিখেছেন। মূল শিক্ষা: Azure-এ সবকিছু Resource Group-এর ভেতরে থাকে, এবং Managed Identity Secret-মুক্ত প্রমাণীকরণের শ্রেষ্ঠ পথ।

১৮.১৫ Knowledge Check

১. Resource Group কী? ২. NSG-তে Priority কেন দরকার? ৩. System-Assigned ও User-Assigned Identity-এর পার্থক্য কী? ৪. azurerm Backend কীভাবে Lock করে? ৫. `features {}` কেন বাধ্যতামূলক?

১৮.১৬ Practical Assignment

TechNova-র জন্য Azure-এ একটি পরিবেশ তৈরি করুন: Resource Group, VNet, দুটি Subnet, NSG, একটি Linux VM (Nginx সহ), একটি Storage Account (Private Container)। তারপর azurerm Backend-এ State সরান। শেষে `terraform destroy` দিয়ে সব মুছে ফেলুন এবং খরচ যাচাই করুন।

অধ্যায় ১৯ • PART IV — Cloud Infrastructure Provisioning

Chapter 19 — Google Cloud Platform Infrastructure Provisioning

১৯.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি GCP-তে Terraform দিয়ে Project, IAM Binding, Service Account, VPC Network, Subnet, Firewall Rule, Compute Engine Instance ও Cloud Storage Bucket তৈরি করতে পারবেন। GCS Backend দিয়ে Remote State কনফিগার করতে পারবেন।

১৯.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

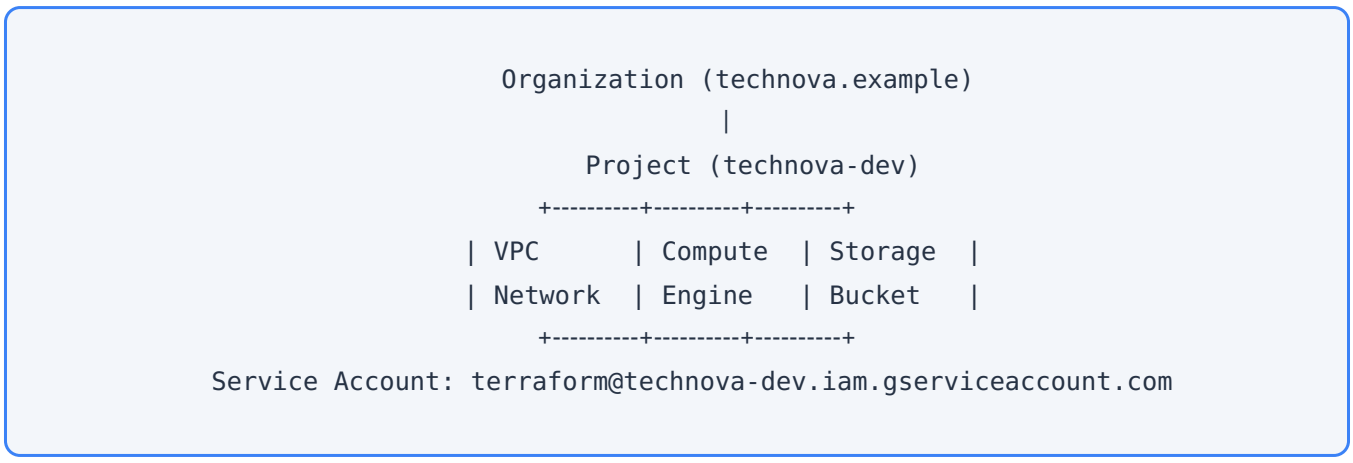
GCP (Google Cloud Platform) Google-এর ক্লাউড — Kubernetes-এর জন্মস্থান, BigQuery ও AI সেবার জন্য বিখ্যাত। বহু Startup ও Data কোম্পানি GCP ব্যবহার করে। GCP-এর ধারণাগুলো AWS/Azure-এর সঙ্গে মিলে যায়, কিন্তু নাম আলাদা: EC2-র বদলে Compute Engine, S3-র বদলে Cloud Storage, VPC-র বদলে VPC Network। GCP-এর IAM ও Project গঠন বুঝলে তিনটি ক্লাউডের তুলনা করা সহজ হয়।

১৯.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ১৭–১৮-এর নেটওয়ার্ক ও IAM ধারণা, Terraform-এর মৌলিক জ্ঞান এবং `gcloud` CLI-এর মৌলিক ব্যবহার জানা থাকলে ভালো হয়।

১৯.৪ প্রথম নীতিমালা: Project, Organization ও IAM

GCP-এর স্তরবিন্যাস: Organization → Folder → Project → Resource। Project হলো সবকিছুর ধারক — API, বিলিং, IAM সব Project-এর সঙ্গে যুক্ত। একটি VM, একটি Bucket — সবই কোনো না কোনো Project-এর ভেতরে থাকে। IAM-এ Member (User, Service Account, Group) -কে Role দেওয়া হয় — যেমন `roles/compute.admin`। Binding মানে Member + Role + Resource-এর সংযোগ।



> **CONCEPT:** Service Account হলো অ্যাপ্লিকেশন বা Terraform-এর পরিচয় — মানুষের নয়, যন্ত্রের User। Terraform চালানোর জন্য একটি Service Account তৈরি করে তাকে প্রয়োজনীয় Role দিন। নিজের ব্যক্তিগত Google অ্যাকাউন্ট দিয়ে Terraform চালাবেন না — তাতে অনুমতি বেশি থাকে এবং Audit করা কঠিন হয়।

১৯.৫ Provider কনফিগারেশন: Project ও Region

```
terraform {
  required_version = ">= 1.9, < 2.0"
  required_providers {
    google = {
      source  = "hashicorp/google"
      version = "~> 6.0"
    }
  }
}

provider "google" {
  project = var.project_id
  region  = "asia-south1"
  zone    = "asia-south1-a"
}
```

Authentication: `gcloud auth application-default login` চালালে Terraform Application Default Credentials ব্যবহার করে। CI/CD-তে Service Account Key বা Workload Identity Federation ব্যবহার করুন — Key ফাইল Git-এ কখনো দেবেন না। `region` হলো Region, `zone` হলো তার ভেতরের Zone (যেমন `asia-south1-a`)। VM Zone-স্তরে তৈরি হয়, Bucket Region-স্তরে।

১৯.৬ IAM Binding ও Service Account

```
resource "google_service_account" "terraform" {
  account_id   = "terraform-deployer"
  display_name = "Terraform Deployer"
  project      = var.project_id
}

resource "google_project_iam_member" "terraform_compute" {
  project = var.project_id
  role    = "roles/compute.admin"
  member  = "serviceAccount:${google_service_account.terraform.email}"
}

resource "google_project_iam_member" "terraform_storage" {
  project = var.project_id
  role    = "roles/storage.admin"
  member  = "serviceAccount:${google_service_account.terraform.email}"
}
```

`google_project_iam_member` একটি Member-কে একটি Role দেয় — আগের Binding মুছে ফেলে না। এর বদলে `google_project_iam_binding` ব্যবহার করলে সেই Role-এর সব Member প্রতিস্থাপিত হয় — ভুলে অন্যের অনুমতি মুছে যেতে পারে। তাই `_member` ব্যবহার করাই নিরাপদ। Least Privilege মনে রাখুন: `roles/owner` কখনো দেবেন না।

১৯.৭ VPC Network, Subnet ও Firewall Rule

GCP-তে VPC Network হলো Global — একটিই Network সব Region জুড়ে থাকে। Subnet Region-স্তরে তৈরি হয়। Firewall Rule আলাদা Resource হিসেবে তৈরি হয় — AWS Security Group-এর মতো VM-এর সঙ্গে নয়, Network-এর সঙ্গে যুক্ত হয়।

```
resource "google_compute_network" "main" {
  name                    = "technova-vpc"
  auto_create_subnetworks = false
  project                 = var.project_id
}

resource "google_compute_subnetwork" "public" {
  name          = "technova-public"
  ip_cidr_range = "10.0.1.0/24"
  region        = "asia-south1"
  network       = google_compute_network.main.id
  project       = var.project_id
}

resource "google_compute_subnetwork" "private" {
  name          = "technova-private"
  ip_cidr_range = "10.0.2.0/24"
  region        = "asia-south1"
  network       = google_compute_network.main.id
  project       = var.project_id
}

resource "google_compute_firewall" "allow_http" {
  name    = "technova-allow-http"
  network = google_compute_network.main.name
  project = var.project_id
  allow {
    protocol = "tcp"
    ports    = ["80"]
  }
  source_ranges = ["0.0.0.0/0"]
  target_tags   = ["web"]
}

resource "google_compute_firewall" "allow_ssh" {
  name    = "technova-allow-ssh"
  network = google_compute_network.main.name
  project = var.project_id
  allow {
    protocol = "tcp"
    ports    = ["22"]
  }
  source_ranges = ["203.0.113.0/24"]
  target_tags   = ["web"]
}
```

`auto_create_subnetworks = false` মানে Custom Mode VPC — Subnet নিজে তৈরি করবেন। `target_tags` মানে এই Rule শুধু `web` Tag-যুক্ত VM-এ প্রযোজ্য হবে। Tag দিয়ে Firewall নিয়ন্ত্রণ করা GCP-এর একটি শক্তিশালী ধারণা।

১৯.৮ Compute Engine Instance

```
resource "google_compute_instance" "web" {
  name         = "technova-web-01"
  machine_type = "e2-micro"
  zone         = "asia-south1-a"
  project      = var.project_id
  tags         = ["web"]

  boot_disk {
    initialize_params {
      image = "ubuntu-os-cloud/ubuntu-2204-lts"
      size  = 10
    }
  }

  network_interface {
    subnetwork = google_compute_subnetwork.public.id
    access_config {
      # Ephemeral public IP
    }
  }

  metadata_startup_script = <<-EOF
    #!/bin/bash
    apt-get update -y
    apt-get install -y nginx
    systemctl enable --now nginx
  EOF

  service_account {
    email  = google_service_account.terraform.email
    scopes = ["cloud-platform"]
  }
}
```

`e2-micro` হলো সবচেয়ে ছোট Machine Type। `access_config` থাকলে Public IP মেলে — না থাকলে শুধু Private IP। `metadata_startup_script` হলো GCP-এর `user_data`। Service Account VM-কে Google API ব্যবহারের অনুমতি দেয়।

> **REAL WORLD:** TechNova-র একটি দল `e2-micro`-এর বদলে ভুলে `n2-standard-8` চালু করেছিল — এক সপ্তাহে বিল বহুগুণ বেড়ে গিয়েছিল। শিক্ষা: Machine Type সবসময় Variable হিসেবে দিন এবং `terraform plan`-এ Size যাচাই করুন।

১৯.৯ Cloud Storage Bucket

```
resource "google_storage_bucket" "app" {
  name          = "technova-app-data-CHANGE-ME-UNIQUE"
  location      = "ASIA-SOUTH1"
  project       = var.project_id
  force_destroy = false
  uniform_bucket_level_access = true
```

```
  versioning { enabled = true }
  labels = {
    project     = "technova"
    environment = "dev"
  }
}
```

`uniform_bucket_level_access = true` মানে Bucket-স্তরে IAM নিয়ন্ত্রণ — Object-স্তরের ACL বন্ধ। `force_destroy = false` মানে Bucket-এ ফাইল থাকলে মুছে ফেলা যাবে না — দুর্ঘটনা রোধ করে। Bucket-এর নাম বিশ্বজুড়ে Unique হতে হয়।

১৯.১০ Remote State: GCS Backend

```
terraform {
  backend "gcs" {
    bucket = "technova-tfstate-CHANGE-ME-UNIQUE"
    prefix = "dev"
  }
}
```

GCS Backend নিজেই Object Lock দিয়ে State Locking করে। প্রথমে Bucket তৈরি করতে হয় (Bootstrap ধাপে) এবং Versioning চালু রাখতে হয়, যাতে State-এর পুরনো সংস্করণ ফিরিয়ে আনা যায়।

১৯.১১ TechNova সম্পূর্ণ GCP উদাহরণ

```
$ gcloud auth application-default login
$ terraform init
$ terraform plan -out=tfplan
$ terraform apply tfplan
```

প্রত্যাশিত আউটপুট:

```
Plan: 7 to add, 0 to change, 0 to destroy.
Apply complete! Resources: 7 added, 0 changed, 0 destroyed.
Outputs:
web_external_ip = "203.0.113.30"
```

Output-এর জন্য `outputs.tf`-এ লিখুন: `output "web_external_ip" { value = google_compute_instance.web.network_interface[0].access_config[0].nat_ip }`।

১৯.১২ সাধারণ ভুল ও Troubleshooting

সমস্যা	কারণ	সমাধান
`Permission denied`	Service Account-এ Role নেই	প্রয়োজনীয় Role Binding দিন
API not enabled	Compute API চালু নেই	`gcloud services enable compute.googleapis.com`
Bucket নাম ভুল	নাম Unique নয়	Random Suffix যোগ করুন

Firewall কাজ করছে না	VM-এ Tag নেই	`target_tags` -এর সঙ্গে VM Tag মেলান

TROUBLESHOOTING: GCP-এ API Enable করতে ভুললে Terraform ভুল দেয়। `gcloud services list --enabled` দিয়ে চালু API দেখুন। নতুন Project-এ Compute, Storage ও IAM API আগে চালু করুন।

১৯.১৩ নিরাপত্তা ও Production অনুশীলন

Service Account-এ Least Privilege দিন, Key ফাইল Git-এ দেবেন না। Bucket-এ Uniform Access ও Versioning চালু করুন। Firewall-এ SSH শুধু অফিস IP থেকে খুলুন। সব Resource-এ Label দিন — GCP-এ Label-ই Tag-এর সমতুল্য।

WARNING: Compute Engine Instance ও Public IP-এর ঘণ্টা প্রতি খরচ আছে। `e2-micro` কিছু Region-এ Free Tier-এ পড়ে, কিন্তু ভুল Machine Type বা অতিরিক্ত Disk বিল বাড়ায়। কাজ শেষে `terraform destroy` চালান।

১৯.১৪ অধ্যায়ের সারাংশ

এই অধ্যায়ে আপনি GCP-এর Project ও IAM, VPC Network, Firewall, Compute Engine, Cloud Storage ও GCS Backend শিখেছেন। মূল শিক্ষা: GCP-এ VPC Global, Firewall Tag-ভিত্তিক, এবং Service Account-ই যন্ত্রের পরিচয়।

১৯.১৫ Knowledge Check

১. Project কী? ২. Service Account কেন দরকার? ৩. `iam_member` ও `iam_binding`-এর পার্থক্য কী? ৪. GCP Firewall কীভাবে Tag ব্যবহার করে? ৫. GCS Backend কীভাবে Lock করে?

১৯.১৬ Practical Assignment

TechNova-র জন্য GCP-তে একটি পরিবেশ তৈরি করুন: VPC, দুটি Subnet, Firewall Rule, একটি Compute Engine Instance (Nginx সহ), একটি Cloud Storage Bucket। GCS Backend-এ State সরান। শেষে `terraform destroy` দিয়ে সব মুছে ফেলুন।

অধ্যায় ২০ • PART IV — Cloud Infrastructure Provisioning

Chapter 20 — IBM Cloud Infrastructure Provisioning

২০.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি IBM Cloud-এ Terraform দিয়ে IAM Access Policy, VPC, Subnet, Virtual Server Instance (VSI), Cloud Object Storage (COS) ও Security Group তৈরি করতে পারবেন। IBM Cloud-এর State ব্যবস্থাপনার সীমা বুঝবেন এবং সম্পূর্ণ TechNova উদাহরণ চালাতে পারবেন।

২০.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

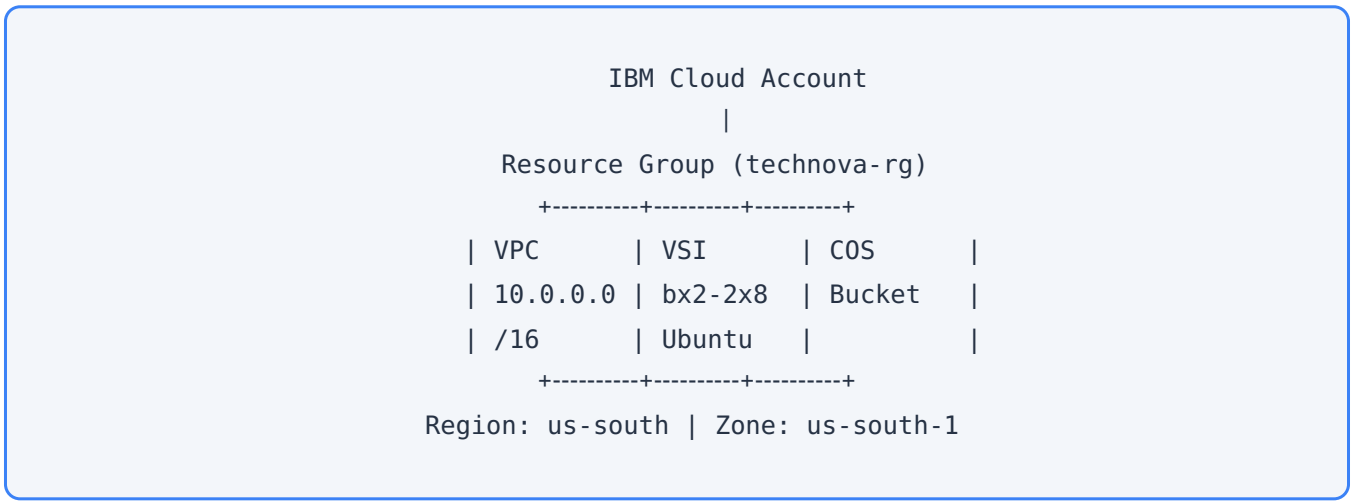
IBM Cloud বহু Enterprise, ব্যাংক ও সরকারি প্রতিষ্ঠানে ব্যবহৃত হয় — বিশেষ করে যেখানে IBM-এর সফটওয়্যার বা Red Hat OpenShift চলে। AWS/Azure/GCP জানা Engineer-এর জন্য IBM Cloud চতুর্থ ক্লাউড হিসেবে মূল্যবান। IBM Cloud-এর ধারণাগুলো মিলে যায়, কিন্তু Resource-এর নাম ও Provider আলাদা — যেমন `ibm_is_vpc`, `ibm_is_instance`। এই অধ্যায়ে সেই অনুবাদ শিখবেন।

২০.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ১৭–১৯-এর VPC, IAM ও VM ধারণা এবং Terraform-এর মৌলিক জ্ঞান প্রয়োজন। IBM Cloud CLI (`ibmcloud`) ইনস্টল থাকা ভালো।

২০.৪ প্রথম নীতিমালা: IBM Cloud-এর গঠন — Account, Resource Group ও Region

IBM Cloud-এর স্তরবিন্যাস: Account → Resource Group → Resource। Resource Group এখানেও পরিচালনার একক। Region হলো ভৌগোলিক এলাকা, যেমন `us-south` (ডালাস) বা `eu-gb` (লন্ডন)। Zone হলো Region-এর ভেতরের Data Center, যেমন `us-south-1`।



> **CONCEPT:** IBM Cloud-এ দুই ধরনের সেবা আছে — Classic Infrastructure ও VPC Infrastructure। আধুনিক কাজে VPC Infrastructure ব্যবহার করুন। Terraform Provider-এ VPC Resource-গুলোর নাম `ibm_is_` দিয়ে শুরু হয় (`is` মানে Infrastructure Services), আর Classic Resource-গুলো `ibm_compute_` দিয়ে শুরু হয়। দুটো মেশাবেন না।

২০.৫ Provider কনফিগারেশন: API Key ও Region

```
terraform {
  required_version = ">= 1.9, < 2.0"
  required_providers {
    ibm = {
      source  = "IBM-Cloud/ibm"
      version = "~> 1.0"
    }
  }
}

variable "ibmcloud_api_key" {
  type      = string
  sensitive = true
}

provider "ibm" {
  ibmcloud_api_key = var.ibmcloud_api_key
  region           = "us-south"
}
```

API Key কখনো কোডে লিখবেন না — Variable হিসেবে দিন এবং Environment Variable (`TF_VAR_ibmcloud_api_key`) বা `terraform.tfvars` (যা Git-এ যাবে না) দিয়ে দিন। IBM Cloud Console থেকে API Key তৈরি করা যায়। `region` বদলালে সব VPC Resource সেই Region-এ তৈরি হয়।

```
$ export TF_VAR_ibmcloud_api_key="CHANGE-ME-API-KEY"
$ terraform init
$ terraform plan
```

২০.৬ IAM ও Access Policy

IBM Cloud IAM-এ Access Group-কে Policy দিয়ে অনুমতি দেওয়া হয়। Policy বলে দেয় কোন Service-এ কী Role দেওয়া হবে — যেমন VPC-তে `Editor` Role।

```
resource "ibm_iam_access_group" "platform" {
  name = "technova-platform"
}

resource "ibm_iam_access_group_policy" "vpc_editor" {
  access_group_id = ibm_iam_access_group.platform.id
  roles           = ["Editor"]
  resources {
    service = "is"
```

```
  }
}
```

`service = "is"` মানে VPC Infrastructure সেবা। Role-গুলো হলো Viewer, Operator, Editor ও Administrator — Least Privilege অনুযায়ী সবচেয়ে ছোট Role দিন।

২০.৭ VPC, Subnet ও Security Group

```
resource "ibm_resource_group" "main" {
  name = "technova-rg"
}

resource "ibm_is_vpc" "main" {
  name           = "technova-vpc"
  resource_group = ibm_resource_group.main.id
}

resource "ibm_is_subnet" "public" {
  name            = "technova-public"
  vpc             = ibm_is_vpc.main.id
  zone            = "us-south-1"
  ipv4_cidr_block = "10.0.1.0/24"
  resource_group  = ibm_resource_group.main.id
}

resource "ibm_is_subnet" "private" {
  name            = "technova-private"
  vpc             = ibm_is_vpc.main.id
  zone            = "us-south-1"
  ipv4_cidr_block = "10.0.2.0/24"
  resource_group  = ibm_resource_group.main.id
}

resource "ibm_is_security_group" "web" {
  name           = "technova-web-sg"
  vpc            = ibm_is_vpc.main.id
  resource_group = ibm_resource_group.main.id
}

resource "ibm_is_security_group_rule" "http" {
  group     = ibm_is_security_group.web.id
  direction = "inbound"
  remote    = "0.0.0.0/0"
  tcp {
    port_min = 80
    port_max = 80
  }
}

resource "ibm_is_security_group_rule" "ssh" {
  group     = ibm_is_security_group.web.id
  direction = "inbound"
  remote    = "203.0.113.0/24"
  tcp {
    port_min = 22
    port_max = 22
```

```
  }
}
```

IBM Cloud-এ Security Group Rule আলাদা Resource — AWS-এর মতো Inline নয়। `remote` হলো Source CIDR। `ipv4_cidr_block` দিয়ে Subnet-এর CIDR নির্ধারণ করা হয়।

২০.৮ Virtual Server Instance (VSI)

VSI হলো IBM Cloud-এর VM। Profile হলো Size — যেমন `bx2-2x8` (2 vCPU, 8 GB RAM)। Image হলো OS — Ubuntu Image-এর ID Data Source দিয়ে খুঁজে নেওয়া যায়।

```
data "ibm_is_image" "ubuntu" {
  name = "ibm-ubuntu-22-04-4-minimal-amd64-1"
}

resource "ibm_is_ssh_key" "main" {
  name       = "technova-ssh-key"
  public_key = file("~/.ssh/id_ed25519.pub")
}

resource "ibm_is_instance" "web" {
  name    = "technova-web-01"
  vpc     = ibm_is_vpc.main.id
  zone    = "us-south-1"
  profile = "bx2-2x8"
  image   = data.ibm_is_image.ubuntu.id
  keys    = [ibm_is_ssh_key.main.id]

  primary_network_interface {
    subnet          = ibm_is_subnet.public.id
    security_groups = [ibm_is_security_group.web.id]
  }

  user_data = <<-EOF
    #!/bin/bash
    apt-get update -y
    apt-get install -y nginx
    systemctl enable --now nginx
  EOF
}

resource "ibm_is_floating_ip" "web" {
  name   = "technova-web-fip"
  target = ibm_is_instance.web.primary_network_interface[0].id
}
```

Floating IP হলো Public IP — VSI-র Private Interface-এর সঙ্গে যুক্ত হয়। SSH Key Resource হিসেবে তৈরি করা হয় — এটি IBM Cloud-এর বিশেষত্ব। `bx2-2x8` হলো Balanced Profile — শেখার জন্য উপযুক্ত।

২০.৯ Cloud Object Storage (COS)

```
resource "ibm_resource_instance" "cos" {
  name              = "technova-cos"
  service           = "cloud-object-storage"
```

```
  plan              = "standard"
  location          = "global"
  resource_group_id = ibm_resource_group.main.id
}

resource "ibm_cos_bucket" "app" {
  bucket_name          = "technova-app-data-CHANGE-ME-UNIQUE"
  resource_instance_id = ibm_resource_instance.cos.id
  region_location      = "us-south"
  storage_class        = "standard"
}
```

COS Instance প্রথমে তৈরি করতে হয় — তারপর তার ভেতরে Bucket। `plan = "standard"` হলো সাধারণ Plan। Bucket-এর নাম বিশ্বজুড়ে Unique হতে হয়।

২০.১০ State ব্যবস্থাপনা: সত্য কথা

> **WARNING:** IBM Cloud Object Storage Terraform-এর Native Backend নয়। AWS-এর S3 Backend বা Azure-র azurerm Backend-এর মতো IBM Cloud-এর জন্য কোনো Official Backend নেই। COS-কে S3-Compatible Backend হিসেবে ব্যবহার করা যায় — কারণ COS S3 API সমর্থন করে — কিন্তু এটি Official নয় এবং Locking আচরণ যাচাই করে নিতে হয়। Production-এ নিরাপদ পদ্ধতি হলো: Terraform Cloud, Spacelift, অথবা S3-Compatible Backend — যেটাই ব্যবহার করুন, Locking ও Encryption পরীক্ষা করে নিন।

```
# S3-compatible backend উদাহরণ (পরীক্ষা করে ব্যবহার করুন)
terraform {
  backend "s3" {
    bucket                      = "technova-tfstate-CHANGE-ME-UNIQUE"
    key                         = "dev/terraform.tfstate"
    region                      = "us-south"
    endpoints { s3 = "https://s3.us-south.cloud-object-storage.appdomain.cloud" }
    skip_credentials_validation = true
    skip_region_validation      = true
    skip_requesting_account_id  = true
  }
}
```

এই Backend-এ Endpoint COS-এর S3 Endpoint। Credential হিসেবে COS-এর HMAC Key ব্যবহার করা হয়। Locking-এর জন্য DynamoDB নেই — তাই Concurrent Apply এড়াতে CI/CD-এ Lock বা Approval Gate ব্যবহার করুন।

২০.১১ TechNova সম্পূর্ণ IBM Cloud উদাহরণ

```
$ export TF_VAR_ibmcloud_api_key="CHANGE-ME-API-KEY"
$ terraform init
$ terraform plan -out=tfplan
$ terraform apply tfplan
```

প্রত্যাশিত আউটপুট:

```
Plan: 8 to add, 0 to change, 0 to destroy.
Apply complete! Resources: 8 added, 0 changed, 0 destroyed.
```

Floating IP ঠিকানায় ব্রাউজার খুললে Nginx পাতা দেখা যাবে। Output-এর জন্য: `output "web_floating_ip" { value = ibm_is_floating_ip.web.address }`।

২০.১২ সাধারণ ভুল ও Troubleshooting

সমস্যা	কারণ	সমাধান
`Invalid API key`	ভুল বা মেয়াদোত্তীর্ণ Key	নতুন API Key তৈরি করুন
Image পাওয়া যাচ্ছে না	Image-এর নাম বদলেছে	Data Source-এ Filter যাচাই করুন
VSI-তে SSH হচ্ছে না	Floating IP যুক্ত হয়নি	Floating IP Resource যাচাই করুন
State Lock নেই	Native Backend নেই	CI Lock বা Terraform Cloud ব্যবহার করুন

TROUBLESHOOTING: IBM Cloud-এ Resource-এর নাম Region-ভেদে বদলাতে পারে। Image-এর নাম না মিললে `ibmcloud is images` দিয়ে উপলব্ধ Image দেখুন।

২০.১৩ নিরাপত্তা ও Production অনুশীলন

API Key কখনো Git-এ দেবেন না — Environment Variable ব্যবহার করুন। Access Group-এ Least Privilege দিন। SSH Key ব্যবহার করুন, Password নয়। State File-এর Encryption ও Access Control নিশ্চিত করুন — বিশেষ করে যেহেতু Native Locking নেই।

WARNING: VSI ও Floating IP-এর ঘণ্টা প্রতি খরচ আছে। কাজ শেষে `terraform destroy` চালান এবং IBM Cloud Console-এ বিল যাচাই করুন।

REAL WORLD: TechNova-র একটি দল IBM Cloud-এ State File Local-এ রেখে দুজন একসাথে Apply করেছিল — State Corrupt হয়ে গিয়েছিল। এরপর থেকে State Remote-এ রাখা এবং CI-তে একসাথে একটি Apply-ই চালানোর নিয়ম করা হয়েছে।

২০.১৪ অধ্যায়ের সারাংশ

এই অধ্যায়ে আপনি IBM Cloud Provider, IAM Access Group, VPC, Subnet, Security Group, VSI, COS এবং State-এর সীমা শিখেছেন। মূল শিক্ষা: IBM Cloud-এ কাজ করা যায়, তবে Native Backend নেই — তাই State ব্যবস্থাপনায় অতিরিক্ত সতর্ক থাকুন।

২০.১৫ Knowledge Check

১. `ibm_is_` উপসর্গ কী বোঝায়? ২. Floating IP কী? ৩. IBM Cloud-এ Native Terraform Backend নেই কেন — এর বিকল্প কী? ৪. VSI Profile কী? ৫. API Key নিরাপদে কীভাবে দেবেন?

২০.১৬ Practical Assignment

TechNova-র জন্য IBM Cloud-এ একটি পরিবেশ তৈরি করুন: Resource Group, VPC, দুটি Subnet, Security Group, একটি VSI (Nginx সহ), একটি COS Bucket। State-এর জন্য S3-Compatible Backend বা Terraform Cloud ব্যবহারের পরিকল্পনা লিখুন। শেষে `terraform destroy` দিয়ে সব মুছে ফেলুন।

অধ্যায় ২১ • PART IV — Cloud Infrastructure Provisioning

Chapter 21 — Networking, Compute, Storage, Database, and IAM Automation

২১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি চারটি ক্লাউডের সমতুল্য সেবা চিনতে পারবেন এবং Networking, Compute, Storage, Database ও IAM Automation-এর Cross-Provider Pattern লিখতে পারবেন। Module দিয়ে Abstraction, Tagging Strategy ও Environment Parity বুঝবেন এবং TechNova-র জন্য Provider-Agnostic Module ডিজাইন করতে পারবেন।

২১.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

বাস্তব কোম্পানি প্রায়ই একাধিক ক্লাউড ব্যবহার করে — একটিতে মূল অ্যাপ, অন্যটিতে Backup বা বিশেষ সেবা। প্রতিটি ক্লাউডের নাম আলাদা হলেও সমস্যাগুলো একই: নেটওয়ার্ক ভাগ করা, সার্ভার চালানো, ফাইল রাখা, ডাটাবেস চালানো, অনুমতি দেওয়া। এই Pattern-গুলো বুঝলে নতুন ক্লাউড শেখা সহজ হয় এবং একই ডিজাইন সব জায়গায় প্রয়োগ করা যায়।

২১.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ১৭–২০-এর চারটি ক্লাউডের মৌলিক Resource এবং Terraform Module-এর ধারণা জানা প্রয়োজন।

২১.৪ প্রথম নীতিমালা: একই সমস্যা, আলাদা নাম

সব ক্লাউডে পাঁচটি মৌলিক চাহিদা থাকে: (১) Networking — Resource-গুলো কীভাবে একে অপরের সঙ্গে কথা বলবে; (২) Compute — কোড কোথায় চলবে; (৩) Storage — ফাইল কোথায় থাকবে; (৪) Database — গঠনমূলক ডেটা কোথায় থাকবে; (৫) IAM — কে কী করতে পারবে। Terraform-এর কাজ হলো এই পাঁচটি চাহিদাকে প্রতিটি ক্লাউডের ভাষায় অনুবাদ করা।

> **CONCEPT:** Abstraction মানে পার্থক্য লুকিয়ে মিলটুকু সামনে আনা। একটি Module-এর Input হতে পারে `cidr_block`, `instance_type`, `environment` — ভেতরে AWS হলে VPC তৈরি হবে, Azure হলে VNet তৈরি হবে। ব্যবহারকারীকে ক্লাউডের নাম জানতে হয় না; শুধু চাহিদা জানাতে হয়।

২১.৫ চার ক্লাউডের সমতুল্য সেবা — তুলনা সারণি

ধারণা	AWS	Azure	GCP	IBM Cloud
Virtual Network	VPC	Virtual Network	VPC Network	VPC
Firewall	Security Group	Network Security Group	Firewall Rule	Security Group
Virtual Server	EC2	Virtual Machine	Compute Engine	Virtual Server Instance
Object Storage	S3		Cloud Storage	Cloud Object Storage

		Storage Account (Blob)		
Managed Database	RDS	Azure Database	Cloud SQL	Databases for PostgreSQL
Load Balancer	ALB / NLB	Load Balancer	Cloud Load Balancing	Load Balancer for VPC
Identity	IAM Role	Managed Identity	Service Account	Access Group / IAM
Remote State	S3 + DynamoDB	azurerm Backend	GCS Backend	S3-Compatible (Non-Native)

২১.৬ Networking Automation

Networking Automation-এর মূল নিয়ম: CIDR Block পরিকল্পনা করে লিখুন, Subnet-গুলো AZ বা Zone জুড়ে ছড়িয়ে দিন, Public ও Private আলাদা করুন। নিচে একটি Module-এর উদাহরণ:

```
# modules/network/variables.tf
variable "cidr_block" { type = string }
variable "public_cidrs" { type = list(string) }
variable "private_cidrs" { type = list(string) }
variable "environment" { type = string }
```

Module-এর ভেতরে `for_each` দিয়ে একাধিক Subnet তৈরি করা যায় — প্রতিটি CIDR-এর জন্য একটি Subnet। এতে Subnet-এর সংখ্যা বাড়াতে শুধু List-এ মান যোগ করতে হয়, কোড বদলাতে হয় না।

> **REAL WORLD:** TechNova-র Dev ও Prod পরিবেশে একই Module ব্যবহার করা হয় — শুধু CIDR আলাদা। Dev-এ `10.1.0.0/16`, Prod-এ `10.2.0.0/16`। এতে দুই পরিবেশের গঠন একই থাকে, শুধু ঠিকানা আলাদা হয় — এটাই Environment Parity।

২১.৭ Compute, Storage, Database ও IAM Automation

Compute Automation: Instance Type, Image ও User Data Variable হিসেবে দিন; Auto Scaling Group বা Instance Template ব্যবহার করুন। Storage Automation: Bucket-এ Versioning, Encryption ও Lifecycle Rule (পুরনো ফাইল Archive করা) চালু করুন। Database Automation: Subnet Group, Parameter Group ও Backup Retention কোডে লিখুন; Password Sensitive Variable হিসেবে দিন। IAM Automation: Role ও Policy কোডে লিখুন; মানুষের জন্য Group, যন্ত্রের জন্য Role বা Service Account ব্যবহার করুন।

```
# Database Module-এর ধারণা
variable "engine" { default = "postgres" }
variable "instance_class" { default = "db.t3.micro" }
variable "backup_retention_days" { default = 7 }
variable "multi_az" { default = false }
```

Database-এর Backup Retention মানে কতদিন Backup রাখা হবে। Production-এ অন্তত ৭ দিন রাখুন। Multi-AZ মানে Database দুটি Zone-এ চলবে — একটি নষ্ট হলে অন্যটি কাজ করবে।

২১.৮ Tagging Strategy ও Environment Parity

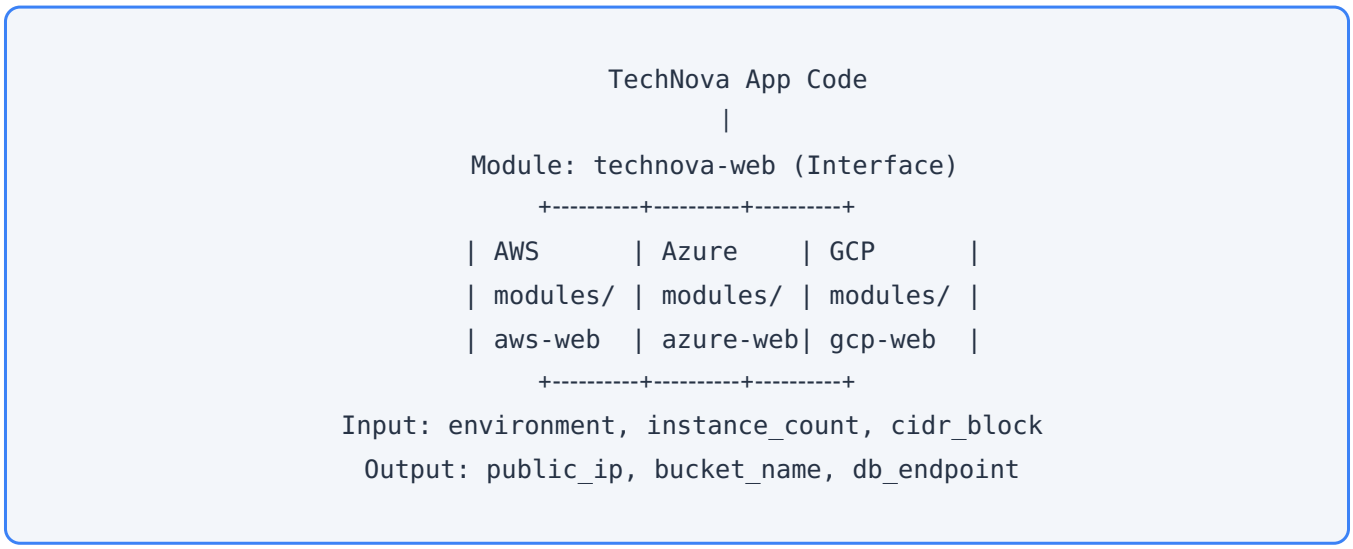
Tagging Strategy মানে সব Resource-এ একই Tag দেওয়া: `Project`, `Environment`, `Owner`, `CostCenter`। AWS-এ `default_tags`, Azure-এ প্রতিটি Resource-এ `tags`, GCP-এ `labels` ব্যবহার করুন। Environment Parity মানে Dev, Staging ও Prod-এর গঠন একই — শুধু Size ও সংখ্যা আলাদা। Parity থাকলে Dev-এ পরীক্ষা করা পরিবর্তন Prod-এ একইভাবে কাজ করে।

```
# environments/dev/terraform.tfvars
environment   = "dev"
instance_type = "t3.micro"
instance_count = 1

# environments/prod/terraform.tfvars
environment   = "prod"
instance_type = "t3.medium"
instance_count = 3
```

একই কোড, আলাদা `.tfvars` ফাইল — এটাই Environment Parity-র সহজ রূপ।

২১.৯ Provider-Agnostic Module Design — TechNova



ডিজাইনের নিয়ম: (১) প্রতিটি ক্লাউডের জন্য আলাদা Module, কিন্তু একই Input/Output নাম; (২) Root Module শুধু সিদ্ধান্ত নেয় কোন ক্লাউডের Module ডাকবে; (৩) কোনো Secret Module-এর ভেতরে হার্ডকোড করবেন না; (৪) প্রতিটি Module-এর README লিখুন — Input, Output ও উদাহরণ সহ।

```
module "web_aws" {
  source         = "./modules/aws-web"
  environment    = "dev"
  cidr_block     = "10.1.0.0/16"
  instance_count = 1
}
```

২১.১০ সাধারণ ভুল ও Troubleshooting

সমস্যা	কারণ	সমাধান
Module-এ Hardcoded Region	Module শুধু এক Region-এ চলে	Region Variable হিসেবে দিন
Tag-এর নাম অসামঞ্জস্য	প্রতিটি দল আলাদা Tag ব্যবহার করে	একটি Tagging নীতিমালা লিখুন
Dev ও Prod-এর গঠন আলাদা	আলাদা কোড লেখা হয়েছে	একই Module + আলাদা tfvars ব্যবহার করুন

TROUBLESHOOTING: Module-এ ভুল হলে `terraform plan`-এর আউটপুটে Module-এর পথ দেখা যায় — যেমন `module.web_aws.aws_instance.web`। সেই পথ ধরে ভেতরের Resource খুঁজুন।

২১.১১ অধ্যায়ের সারাংশ

এই অধ্যায়ে আপনি চার ক্লাউডের সমতুল্য সেবা, পাঁচটি Automation Pattern, Module দিয়ে Abstraction, Tagging ও Environment Parity শিখেছেন। মূল শিক্ষা: সমস্যা একই, নাম আলাদা — Pattern বুঝলে ক্লাউড বদলানো সহজ।

২১.১২ Knowledge Check

১. Environment Parity কী? ২. Tagging কেন দরকার? ৩. Module-এর Input/Output একই রাখা কেন ভালো? ৪. Database-এ Multi-AZ কী? ৫. `for_each` দিয়ে Subnet কীভাবে তৈরি করবেন?

২১.১৩ Practical Assignment

TechNova-র জন্য একটি Network Module তৈরি করুন — Input: CIDR, Public/Private Subnet List, Environment। Module-টি দিয়ে Dev ও Prod — দুটি পরিবেশে আলাদা CIDR দিয়ে নেটওয়ার্ক তৈরি করুন। একটি তুলনা সারণি লিখুন: আপনার Module AWS ও Azure-এ কীভাবে আলাদা হবে।

PART V • DEVOPS ENGINEERING MASTERY

Production Infrastructure as Code

Terraform Security, Secret Management, Validation, Policy as Code, CI/CD, Testing, Collaboration, GitOps, Production Architecture, Governance, Disaster Recovery & Cost Management

অধ্যায় ২২ • PART V — Production Infrastructure as Code

Chapter 22 — Terraform Security, Secrets, Validation, and Policy as Code

২২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি Terraform-এ নিরাপত্তা, Secret ব্যবস্থাপনা, State File-এর গোপনীয়তা, Least Privilege IAM, Backend নিরাপত্তা, Environment Isolation এবং Policy as Code (OPA, Checkov, TFLint, Trivy, Sentinel) বুঝবেন ও প্রয়োগ করতে পারবেন।

২২.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

Terraform দিয়ে পুরো অবকাঠামো তৈরি করা যায় — তাই Terraform-এর একটি ভুল পুরো কোম্পানির নিরাপত্তা ভেঙে দিতে পারে। একটি Public S3 Bucket, একটি খোলা Security Group, একটি ফাঁস হওয়া Password — সবই Terraform কোড থেকে আসতে পারে। নিরাপত্তা তাই Terraform-এর পরের ধাপ নয় — প্রথম ধাপ।

২২.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ১৭–২১-এর Resource, State ও Backend ধারণা এবং IAM-এর Least Privilege নীতি জানা প্রয়োজন।

২২.৪ প্রথম নীতিমালা: Secret কোথায় থাকে?

Secret মানে Password, API Key, Token — যা জানলে কেউ আপনার সিস্টেমে ঢুকতে পারে। Terraform-এ Secret তিন জায়গায় থাকতে পারে: (১) `.tf` ফাইলে — সবচেয়ে খারাপ; (২) Variable-এ — ভালো, যদি Sensitive চিহ্নিত করা হয়; (৩) State File-এ — প্রায়ই ভুলে যাওয়া হয়। State File-এ Resource-এর সব মান থাকে — Password-ও। তাই State File-এর নিরাপত্তা Password-এর নিরাপত্তার সমান।

> **WARNING:** কখনো Secret বা State File Public Git Repository-তে Commit করবেন না। একবার Push হলে তা ইতিহাস থেকে মুছে ফেলাও কঠিন — Git History-তে থেকে যায়। ভুলে Commit হলে Secret-টি বাতিল (Revoke) করে নতুন Secret নিন।

২২.৫ Secret ব্যবস্থাপনা: কী করবেন, কী করবেন না

```
# ভুল — কখনো করবেন না
resource "aws_db_instance" "bad" {
  password = "MyPassword123" # NEVER DO THIS
}

# সঠিক — Sensitive Variable
variable "db_password" {
  type      = string
  sensitive = true
}
```

```
# সঠিক — Secret Manager থেকে পড়া (AWS)
data "aws_secretsmanager_secret_version" "db" {
  secret_id = "technova/db-password"
}
```

Secret দেওয়ার নিরাপদ পথ: (১) Environment Variable — `TF_VAR_db_password`; (২) Secret Manager বা Vault — AWS Secrets Manager, Azure Key Vault, HashiCorp Vault থেকে Data Source দিয়ে পড়া; (৩) `.tfvars` ফাইল — যা `.gitignore`-এ রাখা হয়েছে। `.gitignore`-এ অবশ্যই রাখুন: `*.tfstate`, `*.tfvars` (শুধু উদাহরণ ফাইল ছাড়া), `.terraform/`।

> **CONCEPT:** `sensitive = true` মানে Terraform Output-এ মানটি দেখাবে না — কিন্তু State File-এ মানটি Plain Text-এ থাকে। Sensitive চিহ্ন শুধু চোখের আড়াল করে, নিরাপত্তা দেয় না। আসল নিরাপত্তা আসে Backend-এর Encryption ও Access Control থেকে। এই সীমা বুঝতে হবে।

২২.৬ State File-এর গোপনীয়তা ও Backend নিরাপত্তা

State File-এ সব Resource-এর মান থাকে — IP, Password, Key। তাই: (১) Backend-এ Encryption চালু করুন (S3-এ `encrypt = true`); (২) State Bucket-এ শুধু Terraform চালানো মানুষ বা CI Role-এর Access দিন; (৩) Versioning চালু করুন — ভুল State ফিরিয়ে আনা যায়; (৪) State File Local-এ রাখবেন না।

```
# .gitignore — অবশ্যই রাখুন
*.tfstate
*.tfstate.*
*.tfvars
.terraform/
crash.log
```

২২.৭ Provider Credential: কখনো Hardcode নয়

Provider Credential দেওয়ার নিরাপদ পথ: (১) Environment Variable — `AWS_ACCESS_KEY_ID`, `ARM_CLIENT_ID`, `GOOGLE_APPLICATION_CREDENTIALS`; (২) IAM Role / Managed Identity / Workload Identity — Secret ছাড়াই প্রমাণীকরণ; (৩) OIDC — CI/CD থেকে Short-Lived Token। সবচেয়ে নিরাপদ হলো OIDC — কোনো দীর্ঘমেয়াদি Secret থাকে না, প্রতিটি Run-এ নতুন Token আসে।

২২.৮ Least Privilege, Environment Isolation ও State Locking

Least Privilege: Terraform-এর জন্য আলাদা Role তৈরি করুন — শুধু প্রয়োজনীয় অনুমতি দিন। Dev ও Prod-এর জন্য আলাদা Role, আলাদা Account বা Subscription ব্যবহার করুন। Environment Isolation: Dev ও Prod-এর State File আলাদা Bucket বা আলাদা Key-তে রাখুন — একই State দুই পরিবেশে ব্যবহার করবেন না। State Locking: একসাথে দুটি Apply চললে State Corrupt হয় — S3 + DynamoDB, azurerm Blob Lease বা GCS Lock ব্যবহার করুন। Concurrent Execution এড়াতে CI-তে একসাথে একটি Apply-ই চালান।

২২.৯ Destructive Change, Recovery ও Cost Control

Destructive Change মানে Resource মুছে নতুন করে তৈরি করা — যেমন AMI বদলালে EC2 Replace হয়। `terraform plan`-এ "to destroy" দেখলে থামুন ও যাচাই করুন। `lifecycle { prevent_destroy = true }` দিয়ে গুরুত্বপূর্ণ Resource মুছে ফেলা বন্ধ করা যায়:

```
resource "aws_s3_bucket" "important" {
  bucket = "technova-important-data"
  lifecycle {
    prevent_destroy = true
  }
}
```

Recovery: State Backup রাখুন (Versioning), `terraform state` Command দিয়ে State মেরামত করুন। Cost Control: Tag দিয়ে খরচের হিসাব করুন, Budget Alert চালু করুন, অপ্রয়োজনীয় Resource মুছে ফেলুন।

২২.১০ Policy as Code: OPA, Checkov, TFLint, Trivy, Sentinel

Policy as Code মানে নিরাপত্তা নিয়ম কোড হিসেবে লেখা — মানুষের মনে রাখার ওপর নির্ভর না করে, স্বয়ংক্রিয়ভাবে যাচাই করা।

Tool	ধরন	কী করে
TFLint	Free / Open Source	Terraform কোডের ভুল, পুরনো Syntax ও Best Practice যাচাই
Checkov	Free / Open Source	Terraform Plan-এ নিরাপত্তা নিয়ম যাচাই (যেমন Encryption চালু আছে কি না)
Trivy	Free / Open Source	IaC ও Container Image-এর Vulnerability স্ক্যান
OPA (Open Policy Agent)	Free / Open Source	Rego ভাষায় নিজের নীতি লেখা ও যাচাই
HashiCorp Sentinel	Commercial (Terraform Cloud / Enterprise)	Terraform Cloud-এ Policy Enforcement

Sentinel শুধু Terraform Cloud বা Enterprise-এ পাওয়া যায় — এটি Free নয়। OPA, Checkov, TFLint ও Trivy Open Source — নিজের CI-তে বিনামূল্যে চালানো যায়।

```
# Checkov চালানো
$ checkov -d . --framework terraform
# TFLint চালানো
$ tflint --init
$ tflint
# OPA নীতির উদাহরণ (Rego) — S3 Public নিষিদ্ধ
# package terraform
# deny[msg] { resource := input.resource.aws_s3_bucket[_]; ... }
```

OPA-তে Rego ভাষায় নীতি লেখা হয় — যেমন "কোনো S3 Bucket Public হতে পারবে না"। CI-তে Checkov ব্যর্থ হলে Pipeline থেমে যায় — ভুল কোড Prod-এ যায় না।

> **REAL WORLD:** TechNova-তে Checkov CI-তে যুক্ত করার পর প্রথম সপ্তাহেই ১২টি Encryption-বিহীন Resource ধরা পড়েছিল। এর আগে কেউ খেয়াল করেনি। Policy as Code মানুষের ভুল ধরার জন্য — মানুষকে বদলানোর জন্য নয়।

২২.১১ সাধারণ ভুল ও Troubleshooting

সমস্যা	কারণ	সমাধান
Secret Git-এ চলে গেছে	.gitignore-এ ছিল না	Secret Revoke করুন, ইতিহাস পরিষ্কার করুন
State Lock আটকে আছে	Apply মাঝপথে থেমেছে	নিশ্চিত হয়ে `force-unlock`
Checkov ব্যর্থ হচ্ছে	Encryption নেই	নিয়ম অনুযায়ী Resource ঠিক করুন

> **TROUBLESHOOTING:** `terraform plan`-এ Sensitive মান `(sensitive value)` হিসেবে দেখায় — এটি স্বাভাবিক। মান দেখতে হলে `terraform output -json` নয়, Secret Manager থেকে দেখুন।

২২.১২ অধ্যায়ের সারাংশ

এই অধ্যায়ে আপনি Secret ব্যবস্থাপনা, State নিরাপত্তা, Least Privilege, Environment Isolation এবং Policy as Code শিখেছেন। মূল শিক্ষা: `sensitive = true` যথেষ্ট নয় — Backend-এর Encryption ও Access Control-ই আসল নিরাপত্তা; আর নিয়ম কোডে লিখে স্বয়ংক্রিয়ভাবে যাচাই করুন।

২২.১৩ Knowledge Check

১. Sensitive মান State File-এ কীভাবে থাকে? ২. Secret Git-এ গেলে কী করবেন? ৩. OIDC কেন নিরাপদ? ৪. Checkov ও Sentinel-এর পার্থক্য কী? ৫. `prevent_destroy` কী করে?

২২.১৪ Practical Assignment

TechNova-র একটি Terraform Project-এ Checkov ও TFLint চালান। একটি নীতি লিখুন: কোনো S3 Bucket Public হতে পারবে না এবং কোনো Security Group-এ SSH `0.0.0.0/0` থেকে খোলা যাবে না। একটি `.gitignore` তৈরি করুন এবং State Bucket-এর Encryption যাচাই করুন।

অধ্যায় ২৩ • PART V — Production Infrastructure as Code

Chapter 23 — Terraform CI/CD, Testing, Collaboration, and GitOps Integration

২৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি Terraform-এর জন্য CI/CD Pipeline তৈরি করতে পারবেন — `fmt`, `validate`, `plan`, Approval Gate ও `apply` সহ। GitHub Actions ও GitLab CI উদাহরণ লিখতে পারবেন, `terraform test` ও Terratest ধারণা বুঝবেন, PR-ভিত্তিক Collaboration ও Drift Detection প্রয়োগ করতে পারবেন এবং Atlantis ও GitOps ধারণা ব্যাখ্যা করতে পারবেন।

২৩.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

একা Terraform চালানো সহজ; দলগতভাবে চালানো কঠিন। দুজন একসাথে Apply করলে State নষ্ট হয়, Review ছাড়া Apply করলে ভুল Prod-এ যায়। CI/CD Pipeline এই সমস্যার সমাধান — প্রতিটি পরিবর্তন Review হয়, Plan দেখা যায়, Approval-এর পরেই Apply হয়। GitOps মানে Git-ই সত্যের উৎস — Git-এ যা আছে, অবকাঠামোতে তাই থাকবে।

২৩.৩ পূর্বশর্ত জ্ঞান

Terraform-এর মৌলিক Command, Git ও Pull Request ধারণা এবং CI/CD-এর মৌলিক জ্ঞান প্রয়োজন।

২৩.৪ প্রথম নীতিমালা: Pipeline-এর ধাপগুলো

একটি ভালো Terraform Pipeline-এর ধাপ: (১) `fmt -check` — কোডের গঠন ঠিক আছে কি না; (২) `validate` — Syntax ও Reference ঠিক আছে কি না; (৩) `plan` — কী বদলাবে তা দেখানো; (৪) Plan Review — মানুষ Plan পড়ে Approval দেয়; (৫) `apply` — শুধু Approval-এর পর। Apply সরাসরি `main` Branch-এ নয় — PR Merge-এর পর হয়।



CONCEPT: Plan হলো Terraform-এর সবচেয়ে গুরুত্বপূর্ণ নিরাপত্তা ব্যবস্থা। Apply-এর আগে Plan দেখলে বোঝা যায় কী তৈরি হবে, কী মুছবে। Plan ছাড়া Apply করা মানে চোখ বন্ধ করে গাড়ি চালানো।

২৩.৫ GitHub Actions উদাহরণ (সম্পূর্ণ YAML)

```yaml
name: Terraform
on:
  pull_request:
    paths: ["**.tf"]
  push:
    branches: ["main"]
permissions:
  contents: read
  pull-requests: write
jobs:
  terraform:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: "1.9.8"
      - name: Terraform Format
        run: terraform fmt -check -recursive
      - name: Terraform Init
        run: terraform init
      - name: Terraform Validate
        run: terraform validate
      - name: Terraform Plan
        run: terraform plan -no-color -out=tfplan
      - name: Terraform Apply
        if: github.ref == 'refs/heads/main' && github.event_name == 'push'
        run: terraform apply -auto-approve tfplan
```

এখানে `permissions` দিয়ে শুধু প্রয়োজনীয় অনুমতি দেওয়া হয়েছে। Plan শুধু PR-এ চলে; Apply শুধু `main`-এ Push হলে চলে। Authentication-এর জন্য OIDC ব্যবহার করুন — `id-token: write` Permission দিয়ে।

২৩.৬ GitLab CI উদাহরণ

```yaml
stages: [validate, plan, apply]
terraform:fmt:
  stage: validate
  image: hashicorp/terraform:1.9
  script: [terraform fmt -check -recursive, terraform validate]
terraform:plan:
  stage: plan
  image: hashicorp/terraform:1.9
  script: [terraform init, terraform plan -out=tfplan]
  artifacts: { paths: [tfplan] }
terraform:apply:
  stage: apply
  image: hashicorp/terraform:1.9
  script: [terraform apply -auto-approve tfplan]
  when: manual
  only: [main]
```

`when: manual` মানে Apply-তে মানুষের ক্লিক দরকার — এটাই Approval Gate।

২৩.৭ Testing: terraform test, Terratest ও Plan-ভিত্তিক যাচাই

`terraform test` হলো Terraform-এর নিজস্ব Testing Framework — `.tftest.hcl` ফাইলে Assertion লেখা হয়। Terratest হলো Go-ভিত্তিক Framework — আসল অবকাঠামো তৈরি করে, যাচাই করে, তারপর মুছে ফেলে। Plan-ভিত্তিক যাচাই মানে Apply না করে Plan-এর JSON দেখে নিয়ম যাচাই করা — যেমন "কোনো Resource মুছে যাবে না"।

```
# main.tftest.hcl
run "vpc_has_correct_cidr" {
  command = plan
  assert {
    condition     = aws_vpc.main.cidr_block == "10.0.0.0/16"
    error_message = "VPC CIDR must be 10.0.0.0/16"
  }
}
```

Test চালান: `terraform test`। Terratest-এর ধারণা: Go Test-এ Terraform Apply চালানো, তারপর HTTP Request দিয়ে যাচাই করা যে Nginx চলছে, শেষে Destroy করা।

২৩.৮ Collaboration: State Locking, Plan Review ও PR Workflow

দলগত কাজের নিয়ম: (১) Remote State ও Locking ব্যবহার করুন; (২) সব পরিবর্তন PR-এর মাধ্যমে আনুন; (৩) Plan PR-এ Comment হিসেবে দেখান — GitHub Actions-এ `terraform plan`-এর আউটপুট Comment করা যায়; (৪) অন্তত একজন Reviewer Approval দেবে; (৫) `main` Branch-এ সরাসরি Push নিষিদ্ধ করুন। Plan Review সংস্কৃতি মানে — Plan না পড়ে Approve করবেন না।

> **REAL WORLD:** TechNova-তে একজন Engineer Plan না পড়ে Approve করেছিলেন — Plan-এ ৩টি Database "to destroy" ছিল। ভাগ্য ভালো, Apply-এর আগে আরেকজন দেখে ফেলেছিলেন। এরপর থেকে Plan-এ "destroy" থাকলে দুজন Reviewer বাধ্যতামূলক করা হয়েছে।

২৩.৯ GitOps Integration: Atlantis ও Terraform Controller

Atlantis হলো Terraform-এর জন্য Pull Request Automation Tool — PR খুললে Atlantis স্বয়ংক্রিয়ভাবে `plan` চালায়, Comment করে, আর `atlantis apply` Comment লিখলে Apply করে। Flux-এর Terraform Controller (TF-Controller) হলো Kubernetes-ভিত্তিক GitOps — Git Repository-এ Terraform কোড থাকলে Controller তা Cluster-এ Apply করে। দুটোরই মূল নীতি এক: Git-ই সত্যের উৎস, পরিবর্তন Git-এর মাধ্যমেই আসে।

২৩.১০ Drift Detection

Drift মানে Git-এর কোড ও আসল অবকাঠামোর মধ্যে পার্থক্য — কেউ হাতে Console থেকে কিছু বদলেছে। CI-তে নিয়মিত `terraform plan -detailed-exitcode` চালালে Drift ধরা পড়ে — Exit Code 2 মানে পার্থক্য আছে। Drift পেলে হয় কোডে লিখুন, নয়তো হাতের পরিবর্তন ফিরিয়ে আনুন।

```
$ terraform plan -detailed-exitcode
# Exit 0 = কোনো পরিবর্তন নেই, 1 = ভুল, 2 = পরিবর্তন আছে
```

২৩.১১ সাধারণ ভুল ও Troubleshooting

সমস্যা	কারণ	সমাধান
CI-তে Plan ব্যর্থ	Credential নেই	OIDC বা Secret কনফিগার করুন
State Lock CI-তে আটকে আছে	আগের Job থেমেছে	নিশ্চিত হয়ে Unlock করুন
Apply সরাসরি PR-এ চলছে	Condition ভুল	Apply শুধু main-এ সীমাবদ্ধ করুন

TROUBLESHOOTING: CI-তে `terraform init` ব্যর্থ হলে Backend Credential যাচাই করুন। Local-এ কাজ করলেও CI-তে Credential আলাদা হয়।

২৩.১২ অধ্যায়ের সারাংশ

এই অধ্যায়ে আপনি Terraform CI/CD Pipeline, GitHub Actions ও GitLab CI, Testing, PR Workflow, Atlantis, GitOps ও Drift Detection শিখেছেন। মূল শিক্ষা: Plan দেখে, Review করে, Approval নিয়ে Apply করুন — কখনো সরাসরি নয়।

২৩.১৩ Knowledge Check

১. Pipeline-এর ধাপগুলো কী? ২. Drift কী? ৩. Atlantis কী করে? ৪. `terraform test` কী? ৫. Plan Review কেন দরকার?

২৩.১৪ Practical Assignment

TechNova-র জন্য একটি GitHub Actions Workflow লিখুন: `fmt`, `validate`, `plan` PR-এ এবং `apply` শুধু `main`-এ। একটি `.tftest.hcl` Test লিখুন যা VPC CIDR যাচাই করে। Drift Detection-এর জন্য একটি দৈনিক CI Job-এর পরিকল্পনা লিখুন।

অধ্যায় ২৪ • PART V — Production Infrastructure as Code

Chapter 24 — Production IaC Architecture, Governance, Disaster Recovery, and Cost Management

২৪.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি Production IaC-এর Repository গঠন, Governance (Naming, Tagging, Ownership), Disaster Recovery (State Backup, Multi-Region, Runbook), Cost Management (Infracost, Budget, Right-Sizing), Change Management, Audit ও Documentation মান বুঝবেন এবং TechNova-র জন্য সম্পূর্ণ Production Reference Architecture তৈরি করতে পারবেন।

২৪.২ কেন এই বিষয়টি DevOps-এ গুরুত্বপূর্ণ

Dev-এ Terraform চালানো আর Production-এ চালানো এক নয়। Production-এ ভুলের মূল্য বেশি — ডেটা হারানো, সেবা বন্ধ, বড় বিল। Production IaC মানে শুধু কোড লেখা নয় — Governance, Recovery ও খরচের ব্যবস্থা একসাথে করা। এই অধ্যায়ে সেই সম্পূর্ণ ছবি দেখবেন।

২৪.৩ পূর্বশর্ত জ্ঞান

অধ্যায় ১৭–২৩-এর সব ধারণা — Provider, State, Security, CI/CD — জানা প্রয়োজন।

২৪.৪ প্রথম নীতিমালা: Production IaC-এর চার স্তম্ভ

Production IaC চারটি স্তম্ভের ওপর দাঁড়ায়: (১) Governance — কে কী করতে পারবে, কীভাবে নাম হবে; (২) Reliability — ভুল হলে কীভাবে ফিরে আসা যাবে; (৩) Security — Secret ও অনুমতি কীভাবে নিয়ন্ত্রিত হবে; (৪) Cost — খরচ কীভাবে নিয়ন্ত্রণে থাকবে। একটি স্তম্ভ ভেঙে গেলে পুরো ব্যবস্থা ঝুঁকিতে পড়ে।

> **CONCEPT:** Governance মানে নিয়মের শাসন — কে Approve করবে, কী নাম হবে, কী Tag থাকবে। এটি আমলাতন্ত্র নয়; এটি দুর্ঘটনা রোধের ব্যবস্থা। ভালো Governance দলকে ধীর করে না — দলকে নিরাপদে দ্রুত কাজ করতে দেয়।

২৪.৫ Repository গঠন

```
technova-iac/
├── modules/
│   ├── network/        # VPC / VNet / Subnet
│   ├── compute/        # EC2 / VM
│   └── database/       # RDS / Cloud SQL
├── environments/
│   ├── dev/            # main.tf + terraform.tfvars
│   ├── staging/
│   └── prod/
```

```
├── policies/           # OPA / Checkov নীতি
├── .github/workflows/  # CI/CD
└── README.md           # কীভাবে চালাতে হয়
```

নিয়ম: Module-গুলো একবার লেখা হয়, Environment-গুলো সেই Module ব্যবহার করে। Prod-এর কোড Dev-এর কোডের মতোই — শুধু `.tfvars` আলাদা।

২৪.৬ Governance: Naming, Tagging ও Ownership

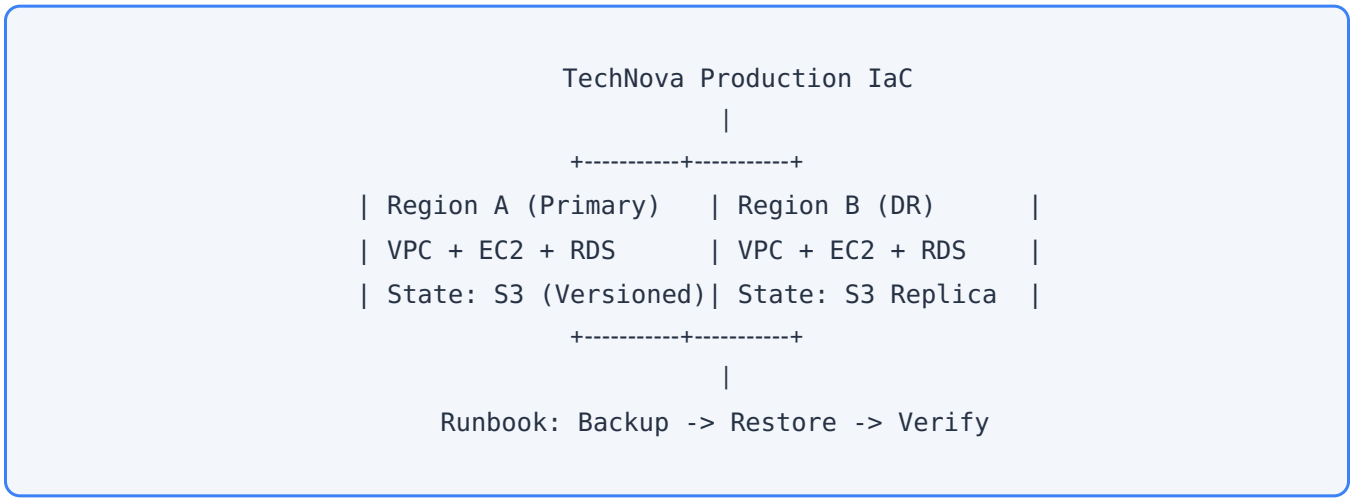
Naming Convention: `{project}-{environment}-{resource}-{instance}` — যেমন `technova-prod-web-01`। Tagging: প্রতিটি Resource-এ `Project`, `Environment`, `Owner`, `CostCenter` বাধ্যতামূলক — Policy as Code দিয়ে যাচাই করুন। Ownership: প্রতিটি Module-এর একজন Owner থাকবে — CODEOWNERS ফাইলে লিখুন।

```
# .github/CODEOWNERS
/modules/network/  @technova/platform-team
/environments/prod/ @technova/platform-leads
```

CODEOWNERS মানে ওই পথের পরিবর্তনে নির্দিষ্ট দলের Approval দরকার।

২৪.৭ Disaster Recovery: State Backup, Multi-Region ও Runbook

State Backup: Backend Bucket-এ Versioning চালু রাখুন — State-এর পুরনো সংস্করণ ফিরিয়ে আনা যায়। নিয়মিত State File-এর Backup নিন। Multi-Region: গুরুত্বপূর্ণ সেবা দুটি Region-এ চালান — একটি Region বন্ধ হলে অন্যটি কাজ করবে। তবে Multi-Region মানে দ্বিগুণ খরচ ও জটিলতা — শুধু প্রয়োজনে করুন।



Recovery Runbook-এর রূপরেখা: (১) ঘটনা ঘোষণা করুন; (২) State Backup থেকে শেষ ভালো State ফিরিয়ে আনুন; (৩) `terraform plan` দিয়ে পার্থক্য দেখুন; (৪) DR Region-এ Apply করুন; (৫) DNS বদলে ট্রাফিক DR-এ নিন; (৬) সেবা যাচাই করুন; (৭) ঘটনার বিবরণ লিখুন। Runbook নিয়মিত অনুশীলন (Game Day) করুন — বিপদের দিনে প্রথমবার পড়লে হবে না।

২৪.৮ Cost Management: Infracost, Budget ও Right-Sizing

Infracost হলো একটি Tool যা Pull Request-এ Terraform পরিবর্তনের মাসিক খরচের হিসাব দেখায় — Apply-এর আগেই জানা যায় বিল কত বাড়বে। Budget: AWS Budgets, Azure Cost Management বা GCP Budget Alert চালু

করুন — খরচ সীমা ছাড়ালে সতর্কবার্তা আসবে। Right-Sizing: Instance-এর Size ব্যবহার অনুযায়ী বদলান — বড় Instance-এ কম ব্যবহার হলে ছোট করুন। Tag দিয়ে খরচের হিসাব করুন — কোন দল, কোন Project কত খরচ করছে।

```
# Infracost চালানো
$ infracost breakdown --path .
# আউটপুট: প্রতিটি Resource-এর মাসিক খরচের হিসাব
```

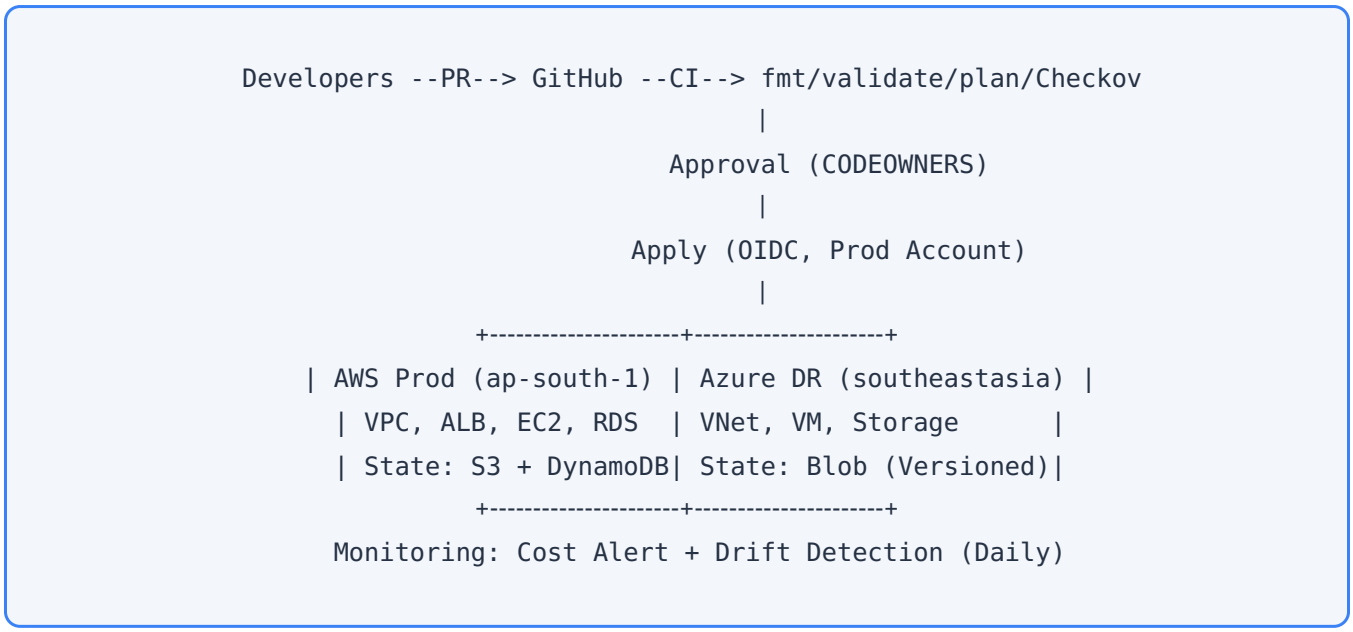
REAL WORLD: TechNova-তে Infracost যুক্ত করার পর একটি PR-এ দেখা গেল NAT Gateway ও বড় RDS-এর জন্য মাসিক খরচ ৪০০ ডলার বাড়বে। দলটি ডিজাইন বদলে খরচ অর্ধেক করেছিল — Apply-এর আগে, বিল আসার পরে নয়।

WARNING: Infracost-এর হিসাব আনুমানিক — আসল বিল AWS/Azure/GCP-এর Cost Explorer-এ দেখুন। Infracost সিদ্ধান্ত নিতে সাহায্য করে, বিলের বিকল্প নয়।

২৪.৯ Change Management, Audit ও Documentation

Change Management: সব পরিবর্তন PR-এর মাধ্যমে, Plan Review ও Approval সহ। Production-এ পরিবর্তনের সময়সূচি (Change Window) মেনে চলুন। Audit: CloudTrail (AWS), Activity Log (Azure) বা Cloud Audit Logs (GCP) চালু রাখুন — কে কখন কী করেছে তার রেকর্ড থাকবে। Terraform-এর State ও Plan-ও Audit-এর অংশ। Documentation: প্রতিটি Module-এর README লিখুন — উদ্দেশ্য, Input, Output, উদাহরণ। Architecture Decision Record (ADR) লিখুন — কেন এই সিদ্ধান্ত নেওয়া হয়েছিল।

২৪.১০ TechNova Production IaC Reference Architecture



এই Architecture-এর মূল বৈশিষ্ট্য: Git-ই সত্যের উৎস, CI-তে সব যাচাই, Prod-এ Approval বাধ্যতামূলক, State Versioned ও Locked, খরচ ও Drift নিয়মিত পর্যবেক্ষণ।

২৪.১১ Operations Runbook Outline

১. দৈনিক: Drift Detection Job-এর ফলাফল দেখুন। ২. সাপ্তাহিক: Cost Report দেখুন, অপ্রয়োজনীয় Resource খুঁজুন। ৩. মাসিক: State Backup যাচাই করুন, Access Review করুন। ৪. জরুরি: State Lock আটকে গেলে — নিশ্চিত হয়ে Unlock; Apply ব্যর্থ হলে — Plan দেখে, State যাচাই করে আবার চালান; Region বন্ধ হলে — DR Runbook অনুসরণ করুন। ৫. সব ঘটনার পরে Postmortem লিখুন — দোষ নয়, শিক্ষা।

২৪.১২ সাধারণ ভুল ও Troubleshooting

সমস্যা	কারণ	সমাধান
Prod-এ ভুল Apply	Approval ছাড়া Apply	CI-তে Approval Gate বাধ্যতামূলক করুন
State হারিয়ে গেছে	Versioning ছিল না	Backup থেকে ফিরিয়ে আনুন
খরচ হঠাৎ বেড়েছে	বড় Resource চালু আছে	Cost Explorer দেখে Right-Size করুন

TROUBLESHOOTING: State নষ্ট হলে আতঙ্কিত হবেন না। প্রথমে Bucket-এর Versioning থেকে আগের সংস্করণ ফিরিয়ে আনুন, তারপর `terraform plan` দিয়ে যাচাই করুন। কখনো State File হাতে সম্পাদনা করবেন না।

২৪.১৩ অধ্যায়ের সারাংশ

এই অধ্যায়ে আপনি Production IaC-এর Repository গঠন, Governance, Disaster Recovery, Cost Management, Change Management, Audit ও Documentation শিখেছেন। মূল শিক্ষা: Production মানে শুধু কোড নয় — নিয়ম, Recovery ও খরচের ব্যবস্থা একসাথে।

২৪.১৪ Knowledge Check

১. Environment Parity কেন দরকার? ২. State Versioning কেন গুরুত্বপূর্ণ? ৩. Infracost কী করে? ৪. CODEOWNERS কী? ৫. DR Runbook-এ কী থাকে?

২৪.১৫ Practical Assignment

TechNova-র জন্য একটি সম্পূর্ণ Production IaC পরিকল্পনা লিখুন: Repository গঠন, Naming ও Tagging নীতি, State Backup পরিকল্পনা, DR Runbook-এর রূপরেখা, Cost Budget ও Infracost ব্যবহারের পরিকল্পনা। একটি Reference Architecture Diagram আঁকুন এবং একটি Operations Runbook তৈরি করুন।